

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG

ĐỒ ÁN TỐT NGHIỆP

**THIẾT KẾ HỆ THỐNG GIÁM SÁT MÔI TRƯỜNG CABIN Ô TÔ SỬ
DỤNG CPU RISC-V TRÊN FPGA**

NGÀNH ĐIỆN TỬ - VIỄN THÔNG

Sinh viên: NGUYỄN TẤN LỢI
MSSV: 22161281
HUỲNH BẢO NGỌC
MSSV: 22161295

TP. HỒ CHÍ MINH – 06/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG

ĐỒ ÁN TỐT NGHIỆP

THIẾT KẾ HỆ THỐNG GIÁM SÁT MÔI TRƯỜNG CABIN
Ô TÔ SỬ DỤNG CPU RISC-V TRÊN FPGA

NGÀNH ĐIỆN TỬ - VIỄN THÔNG

Sinh viên: NGUYỄN TẤN LỢI
MSSV: 22161281
HUỲNH BẢO NGỌC
MSSV: 22161295

Hướng dẫn: ThS LÊ MINH

THÔNG TIN KHÓA LUẬN TỐT NGHIỆP

1. Thông tin sinh viên

Họ và tên sinh viên: Huỳnh Bảo Ngọc

MSSV: 22161295

Email: 22161295@hcmute.edu.vn

Điện thoại:

Họ và tên sinh viên: Nguyễn Tấn Lợi

MSSV: 22161281

Email: 22161281@hcmute.edu.vn

Điện thoại:

2. Thông tin đề tài

- Tên đề tài: Thiết Kế Hệ Thống Giám Sát Môi Trường Cabin Ô Tô Sử Dụng CPU Risc-V trên kit Fpga.
- Đơn vị quản lý: Bộ môn Kỹ Thuật máy tính – Viễn Thông, Khoa Điện Điện Tử, Trường Đại Học Công Nghệ Kỹ Thuật Tp. Hồ Chí Minh.
- Thời gian thực hiện: Từ ngày / / 20... đến ngày / / 20..
- Thời gian bảo vệ trước hội đồng: Ngày / / 20..

3. Lời cam đoan của sinh viên

Tôi cam đoan KLTN là công trình nghiên cứu của tôi, dưới sự hướng dẫn của Thạc sĩ Lê Minh. Kết quả công bố trong KLTN là trung thực và không sao chép từ bất kì công trình nào khác.

Tp. HCM, ngày ... tháng 06 năm 2026

SV thực hiện đồ án

(Ký và ghi rõ họ tên)

.....

Giảng viên hướng dẫn xác nhận quyền báo cáo đã được chỉnh sửa theo đề nghị được ghi trong biên bản của Hội đồng đánh giá Khóa luận tốt nghiệp.

.....

Tp. HCM, ngày ... tháng 06 năm 2026

Giảng viên hướng dẫn

(Ký, ghi rõ họ tên và học hàm – học vị)

Xác nhận của Bộ Môn

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: THIẾT KẾ HỆ THỐNG GIÁM SÁT MÔI TRƯỜNG CABIN Ô TÔ SỬ DỤNG CPU RISC-V TRÊN FPGA

Sinh viên: + Huỳnh Bảo Ngọc

MSSV: 22161295

+ Nguyễn Tấn Lợi

MSSV: 22161281

Hướng dẫn: ThS. Lê Minh

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

Đặt vấn đề rõ.. ..

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

Phương pháp thực hiện hợp lý.

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá / kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.

Kết quả phù hợp với mục tiêu đã đặt ra của đề tài.....

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

Kết luận phù hợp với đặt vấn đề.....

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.

Bố cục báo cáo rõ ràng.....

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

Làm việc nhóm hiệu quả.....

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

Có trích dẫn tài liệu tham khảo theo chỉ dẫn

8. Đánh giá về sự trùng lặp của đề tài

Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

Kết quả kiểm tra trùng lặp với công cụ Turnitin: báo cáo đạt yêu cầu với tỉ lệ trùng lặp dưới 40%.....

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Tinh thần làm việc nghiêm túc.....

Đề nghị của giảng viên hướng dẫn

Ghi rõ: “Báo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp”

Được phép bảo vệ khóa luận tốt nghiệp.....

Tp. HCM, ngày 24 tháng 6 năm 2026

Người nhận xét

(Ký và ghi rõ họ tên)


Lê Minh

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP
(Dùng cho giảng viên phản biện)

Đề tài:	THIẾT KẾ HỆ THỐNG GIÁM SÁT MÔI TRƯỜNG CABIN Ô TÔ SỬ DỤNG CPU RISC-V TRÊN FPGA		
Sinh viên:	Huỳnh Bảo Ngọc	MSSV:	22161295
	Nguyễn Tấn Lợi	MSSV:	22161281

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và tính ứng dụng [15]:
Đặt vấn đề rõ ràng, mục tiêu cụ thể [5]; đề tài có tính mới, cấp thiết [5]; đề tài có khả năng ứng dụng, tính sáng tạo [5].

2. Phương pháp thực hiện/ phân tích/ thiết kế [25]:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết [10]; có phân tích và đánh giá phù hợp [10]; có tính mới và tính sáng tạo [5]

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế [25]:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn [5]; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục [5].

4. Kết luận và đề xuất [10]:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn [5]; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục [5].

5. Hình thức trình bày, bố cục và chất lượng báo cáo [20]:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu [5]; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt [15].

6. Tài liệu trích dẫn [5]:

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

7. Đánh giá về sự trùng lặp của đề tài:

Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

8. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*:

Câu hỏi sinh viên phải trả lời trước hội đồng*:

Kết luận:	Cho bảo vệ ☐	Không cho bảo vệ*	☐	Điểm tổng:	/100
----------------------	--	------------------------------	--------------------------	-------------------	-------------

**Trường hợp giáo viên phản biện không cho bảo vệ, vui lòng ghi rõ lý do:*

Tp. HCM, ngày ... tháng năm 2026
Người phản biện

LỜI CẢM ƠN

Trong quá trình thực hiện đồ án tốt nghiệp với đề tài “Thiết kế hệ thống giám sát môi trường cabin ô tô sử dụng CPU RISC-V trên FPGA”, nhóm thực hiện đã nhận được nhiều sự hỗ trợ, hướng dẫn và động viên từ quý thầy cô, gia đình và bạn bè.

Trước hết, nhóm thực hiện xin bày tỏ lòng biết ơn sâu sắc đến ThS. Lê Minh, giảng viên hướng dẫn đã tận tình hỗ trợ, định hướng và góp ý trong suốt quá trình thực hiện đồ án. Những ý kiến chuyên môn, kinh nghiệm thực tế và sự hướng dẫn của thầy là cơ sở quan trọng giúp nhóm hoàn thiện nội dung nghiên cứu, thiết kế phần cứng, xây dựng phần mềm nhúng và trình bày báo cáo.

Nhóm thực hiện cũng xin gửi lời cảm ơn đến quý thầy cô Khoa Điện – Điện tử đã truyền đạt những kiến thức nền tảng và chuyên môn trong suốt quá trình học tập tại trường. Những kiến thức học tập tại trường là nền tảng quan trọng để nhóm có thể triển khai đề tài này.

Do kiến thức và kinh nghiệm còn hạn chế, báo cáo khó tránh khỏi những thiếu sót. Nhóm thực hiện rất mong nhận được sự góp ý từ quý thầy cô để đề tài được hoàn thiện hơn.

Xin chân thành cảm ơn!

TÓM TẮT

Trong bối cảnh các hệ thống nhúng và FPGA ngày càng được ứng dụng rộng rãi trong giám sát, điều khiển và xử lý dữ liệu, kiến trúc RISC-V với đặc điểm mở, đơn giản và dễ mở rộng trở thành một hướng tiếp cận phù hợp cho việc nghiên cứu và thiết kế CPU mềm trên FPGA. Việc tự xây dựng CPU RISC-V và tích hợp các ngoại vi trong một hệ thống SoC giúp người thiết kế hiểu rõ hơn mối liên hệ giữa phần cứng, phần mềm và cơ chế điều khiển ngoại vi trong hệ thống nhúng.

Đề tài “Thiết kế hệ thống giám sát môi trường cabin ô tô sử dụng CPU RISC-V trên FPGA” được thực hiện nhằm xây dựng một mô hình SoC nhúng trên kit Tang Nano 9K. Trong hệ thống, CPU RV32I đóng vai trò xử lý trung tâm, kết nối với SRAM và các ngoại vi như UART, I2C, PWM, GPIO, LED và Countdown Timer thông qua cơ chế memory-mapped I/O. Hệ thống sử dụng cảm biến AHT20 để đo nhiệt độ, độ ẩm, màn hình OLED SSD1309 để hiển thị dữ liệu, đồng thời sử dụng PWM để điều khiển quạt và servo.

Báo cáo trình bày quá trình thiết kế từ kiến trúc CPU RV32I, tích hợp SoC và ngoại vi, xây dựng chương trình nhúng C đến kiểm thử mô hình thực tế. Kết quả thực hiện cho thấy hệ thống có thể đọc dữ liệu cảm biến, tính toán các thông số môi trường, hiển thị lên OLED và điều khiển quạt, servo theo trạng thái hoạt động. Qua đó, đề tài chứng minh tính khả thi của việc xây dựng một hệ thống giám sát và điều khiển môi trường cabin ô tô dựa trên CPU RISC-V tự thiết kế trên FPGA.

MỤC LỤC

DANH MỤC HÌNH	XI
DANH MỤC BẢNG	XIII
DANH MỤC VIẾT TẮT.....	XIV
CHƯƠNG 1 TỔNG QUAN	1
1.1. Tổng quan	1
1.2. Mục tiêu đề tài	2
1.3. Phạm vi giới hạn đề tài	3
1.4. Phương pháp nghiên cứu	3
1.5. Đối tượng và phạm vi nghiên cứu	4
1.6. Bộ cục quyền báo cáo	4
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT.....	5
2.1. Tổng quan về kiến trúc RISC-V	5
2.1.1. Vi xử lý RISC-V	5
2.1.2. Các phương pháp triển khai kiến trúc RISC-V	6
2.2. Sự phát triển của kiến trúc RISC	8
2.3. Lịch sử phát triển của RISC-V	10
2.4. Giới thiệu ngôn ngữ mô tả phần cứng Verilog.....	11
2.5. Giới thiệu về kiến trúc tập lệnh và tập lệnh RV32I.....	13
2.5.1. Kiến trúc tập lệnh	13
2.5.2. Kiến trúc và tập lệnh RV32I.....	13
2.6 Các thành phần chính trong RISC_V	23
2.6.1 Bộ đếm chương trình (Program Counter)	23
2.6.2 Bộ nhớ lệnh (Instruction Memory).....	23
2.6.3 Bộ thanh ghi (Register File)	23

2.6.4 Bộ nhớ dữ liệu (Data Memory)	23
2.7 Cách thức hoạt động của CPU	24
2.8. Linh kiện sử dụng	25
2.8.1. Khối xử lý trung tâm	25
2.8.3. Khối cơ cấu chấp hành và điều khiển công suất	29
CHƯƠNG 3 THIẾT KẾ PHẦN CỨNG CỦA HỆ THỐNG.....	33
3.1. Yêu cầu hệ thống	33
3.1.1. Mục tiêu thiết kế	33
3.1.2. Yêu cầu chức năng	33
3.1.3. Yêu cầu phần cứng	33
3.1.4. Tiêu chí tổ chức thiết kế phần cứng	34
3.2. Thiết kế CPU RV32I	34
3.2.1. Kiến trúc tổng thể CPU RV32I	34
3.2.2. Nguyên lý hoạt động của CPU	37
3.2.3. Khối Program Counter	38
3.2.4. Khối Register File.....	39
3.2.5. Khối tạo giá trị tức thời	40
3.2.6. Khối ALU	41
3.2.7. Khối giải mã lệnh và điều khiển.....	42
3.2.8. Khối xử lý Load/Store và Reader	42
3.2.9. Giao tiếp bus của CPU	43
3.3. Thiết kế SoC và các ngoại vi.....	44
3.3.1. Kiến trúc tổng thể SoC	45
3.3.2. Bus hệ thống, ánh xạ bộ nhớ và nguyên tắc truy cập ngoại vi	46
3.3.3. Luồng giải mã địa chỉ	47

3.3.4. Khối SRAM.....	48
3.3.5. Khối LED	49
3.3.6. Khối UART	50
3.3.7. Khối Countdown Timer.....	51
3.3.8. Khối I2C	52
3.3.9. Khối PWM.....	53
3.3.10. Khối GPIO đọc nút nhấn	55
CHƯƠNG 4. THIẾT KẾ PHẦN MỀM NHÚNG CHẠY TRÊN CPU RISC-V	58
4.1. Tổng quan phần mềm nhúng	58
4.2. Quy trình biên dịch và tạo file khởi tạo bộ nhớ.....	58
4.3. Thiết kế lớp truy cập thanh ghi phần cứng.....	59
4.4. Hoạt động của các chương trình con	60
4.4.1. Giao tiếp I2C	60
4.4.2. Giao tiếp cảm biến AHT20.....	61
4.4.3. Giao tiếp OLED SSD1309	61
4.5. Xử lý dữ liệu cảm biến và công thức tính toán	61
4.6. Thuật toán điều khiển	62
4.7. Lưu đồ chương trình chính.....	64
CHƯƠNG 5 KẾT QUẢ	66
5.1. Kết quả mô hình thi công	66
5.2. Kết quả phần mềm thiết kế.....	67
5.3. Hoạt động của hệ thống.....	68
5.3.1. Kịch bản 1: Thời tiết đẹp (Trạng thái an toàn):.....	69
5.3.2. Kịch bản 2: Cảnh báo sớm (Chớm động sương).....	70
5.3.3. Kịch bản 3: Mờ trong kính nặng	71

5.3.4. Kịch bản 4: Ngoài trời mát mẻ	72
5.3.5. Kịch bản 5: Nắng nóng gay gắt	73
5.4. Đánh giá kết quả kiểm thử hệ thống	74
CHƯƠNG 6 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	76
6.1. Kết luận.....	76
6.2 Hướng phát triển.....	77
PHỤ LỤC	79
TÀI LIỆU THAM KHẢO	80

DANH MỤC HÌNH

Hình 2.1 Sơ đồ khối single risc-v [11]	6
Hình 2.2 Sự tăng trưởng về hiệu năng của bộ xử lý [13]	9
Hình 2.3. Thanh ghi lệnh trong kiến trúc RISC-V[4].	16
Hình 2.4. Thanh ghi lệnh I-type và thanh ghi lệnh tải trong kiến trúc RISC-V[4]	18
Hình 2.5. Thanh ghi lệnh lưu và thanh ghi lệnh S-type trong kiến trúc RISC-V [4]	19
Hình 2.6. Thanh ghi lệnh R-type trong kiến trúc RISC-V [4].....	20
Hình 2.7. Thanh ghi lệnh I-type trong kiến trúc RISC-V [4].....	20
Hình 2.8. Thanh ghi lệnh J-type trong kiến trúc RISC-V [4].....	21
Hình 2.9. Thanh ghi lệnh I-type trong kiến trúc RISC-V [4].....	22
Hình 2.10. Thanh ghi lệnh B-type trong kiến trúc RISC-V [4].....	22
Hình 2.11. Các thành phần chính trong RISC-V [11]	23
Hình 2.12. Kit FPGA Tang Nano 9K.....	26
Hình 2.13. Cảm biến nhiệt độ và độ ẩm AHT20.....	27
Hình 2.14. Màn hình OLED SSD1309.....	28
Hình 2.15. Quạt DC 5V	29
Hình 2.16. Động cơ Servo SG90.....	30
Hình 2.17. Module MOSFET D4184	31
Hình 3.1. Hình khối tổng thể CPU RV32I.	35
Hình 3.2. Hình chi tiết các khối bên trong CPU RV32I.....	36
Hình 3.3. Lưu đồ FSM hoạt động của CPU đa chu kỳ.....	37
Hình 3.4. Sơ đồ khối Program Counter	39
Hình 3.5. Sơ đồ tổ chức khối Register File trong CPU RV32I.	39
Hình 3.6. Sơ đồ khối tạo giá trị tức thời trong RV32I.	40
Hình 3.7. Sơ đồ khối ALU và các tín hiệu điều khiển.	41

Hình 3.8. Sơ đồ khối giải mã lệnh và điều khiển.	42
Hình 3.9. Sơ đồ khối xử lý lệnh Load/Store.....	43
Hình 3.10. Sơ đồ khối Reader xử lý dữ liệu đọc từ bus.	43
Hình 3.11. Cơ chế bắt tay mem_valid/mem_ready giữa CPU và bộ nhớ hoặc ngoại vi.	44
Hình 3.12. Sơ đồ khối tổng thể SoC RISC-V trên FPGA.	45
Hình 3.13. Luồng giải mã địa chỉ.	48
Hình 3.14. Sơ đồ khối SRAM/BRAM dùng cho chương trình và dữ liệu.	48
Hình 3.15. Sơ đồ khối LED.	49
Hình 3.16. Sơ đồ khối UART.....	50
Hình 3.17. Sơ đồ khối Countdown Timer dùng cho hàm delay.....	52
Hình 3.18. Sơ đồ khối I2C bit-bang điều khiển các đường SCL và SDA.....	52
Hình 3.19. Sơ đồ khối thanh ghi cấu hình PWM	54
Hình 3.20. Sơ đồ khối PWM Generator	54
Hình 3.21. Sơ đồ khối GPIO đọc trạng thái nút nhấn.	56
Hình 3.22. Sơ đồ nguyên lý.....	57
Hình 4.1. Lưu đồ chương trình chính của phần mềm nhúng.....	65
Hình 5.1. Mô hình thi công hệ thống giám sát môi trường cabin ô tô sử dụng CPU RISC- V trên FPGA.....	66
Hình 5.2. Kết quả hiển thị dữ liệu cảm biến và trạng thái điều khiển trên OLED.....	67
Hình 5.3. Kết quả mô phỏng kịch bản 1 – Thời tiết đẹp (Trạng thái an toàn)	70
Hình 5.4. Kết quả mô phỏng kịch bản 2 – Cảnh báo sớm (Chớm động sương)	71
Hình 5.5. Kết quả mô phỏng kịch bản 3 – Mờ trong kính nắng.....	72
Hình 5.6. Kết quả mô phỏng kịch bản 4 – Ngoài trời mát mẻ	73
Hình 5.7. Kết quả mô phỏng kịch bản 5 – Nắng nóng gay gắt	74

DANH MỤC BẢNG

Bảng 2.1: Phân tích sự khác biệt giữa hai kiến trúc CISC–RISC [12].....	9
Bảng 2.2. So sánh FSM Moore và FSM Mealy [11].....	12
Bảng 2.3. Bảng tập lệnh RV32I [14].....	14
Bảng 2.4. Thông số kỹ thuật Kit FPGA Tang Nano 9K	26
Bảng 2.5. Thông số kỹ thuật cảm biến AHT20	27
Bảng 2.6. Thông số kỹ thuật màn hình OLED SSD1309.....	28
Bảng 2.7. Thông số kỹ thuật quạt DC	29
Bảng 2.8. Thông số kỹ thuật động cơ Servo SG90	30
Bảng 2.9. Thông số kỹ thuật module MOSFET D4184.....	31
Bảng 2.10. Danh mục linh kiện	32
Bảng 3.1. Bản đồ địa chỉ các khối nhớ và ngoại vi trong SoC.....	46
Bảng 3.2. Bảng thanh ghi của khối UART.....	50
Bảng 3.3. Các thanh ghi của mỗi kênh PWM.	54
Bảng 4.1. Các thanh ghi ngoại vi được phần mềm nhúng sử dụng.....	59
Bảng 4.2. Các trạng thái điều khiển tự động của hệ thống.....	63
Bảng 5.1. Các kịch bản kiểm thử hệ thống giám sát môi trường cabin ô tô	68

DANH MỤC VIẾT TẮT

Từ viết tắt	Tên tiếng Anh	Ý nghĩa
ALU	Arithmetic Logic Unit	Khối số học và logic, thực hiện các phép toán số học, logic và so sánh trong CPU.
BRAM	Block Random Access Memory	Khối bộ nhớ RAM tích hợp sẵn bên trong FPGA.
CPU	Central Processing Unit	Bộ xử lý trung tâm, thực hiện việc đọc lệnh, giải mã lệnh và điều khiển các khối trong hệ thống.
FPGA	Field-Programmable Gate Array	Vi mạch logic khả trình, cho phép người thiết kế cấu hình phần cứng theo yêu cầu.
FSM	Finite State Machine	Máy trạng thái hữu hạn, dùng để mô tả các khối điều khiển tuần tự trong phần cứng.
GPIO	General-Purpose Input/Output	Cổng vào/ra số đa dụng, trong đề tài được dùng để đọc trạng thái nút nhấn.
I2C	Inter-Integrated Circuit	Chuẩn giao tiếp nối tiếp hai dây, sử dụng hai đường SDA và SCL để truyền dữ liệu giữa các IC ngoại vi.
ISA	Instruction Set Architecture	Kiến trúc tập lệnh, quy định các lệnh mà CPU có thể hiểu và thực thi.
LUT	Look-Up Table	Phần tử logic cơ bản trong FPGA, dùng để hiện thực các hàm logic tổ hợp.
OLED	Organic Light-Emitting Diode	Công nghệ màn hình sử dụng diode phát quang hữu cơ, trong đề tài dùng để hiển thị dữ liệu đo.
PC	Program Counter	Bộ đếm chương trình, lưu địa chỉ lệnh hiện tại hoặc lệnh kế tiếp của CPU.
PWM	Pulse Width Modulation	Phương pháp điều chế độ rộng xung, dùng để điều khiển quạt và servo.

RISC	Reduced Instruction Set Computer	Kiến trúc máy tính sử dụng tập lệnh rút gọn, đơn giản và dễ tối ưu phần cứng.
RISC-V	RISC Five	Kiến trúc tập lệnh mở dựa trên triết lý RISC, có khả năng mở rộng và không phụ thuộc bản quyền thương mại.
RTL	Register Transfer Level	Mức mô tả phần cứng dựa trên sự truyền dữ liệu giữa các thanh ghi.
RV32I	RISC-V 32-bit Base Integer Instruction Set	Tập lệnh số nguyên cơ sở 32 bit của kiến trúc RISC-V.
SoC	System on Chip	Hệ thống tích hợp trên một chip, bao gồm CPU, bộ nhớ, bus và các ngoại vi.
SRAM	Static Random Access Memory	Bộ nhớ truy cập ngẫu nhiên tĩnh, dùng để lưu chương trình, dữ liệu và stack trong hệ thống.
UART	Universal Asynchronous Receiver/Transmitter	Chuẩn truyền nhận nối tiếp bất đồng bộ, dùng để gửi thông tin gỡ lỗi giữa FPGA và máy tính.

CHƯƠNG 1 TỔNG QUAN

1.1. Tổng quan

Trong những năm gần đây, cùng với sự phát triển của công nghệ điện tử nhúng và vi xử lý, các hệ thống giám sát và điều khiển thông minh ngày càng được ứng dụng rộng rãi trong đời sống, đặc biệt là trong lĩnh vực ô tô. Cabin ô tô là không gian kín, chịu ảnh hưởng trực tiếp bởi môi trường bên ngoài, số lượng người ngồi trong xe, hệ thống điều hòa không khí, bức xạ mặt trời và trạng thái vận hành của phương tiện. Các tài liệu về an toàn trong xe cho thấy nhiệt độ trong khoang xe có thể tăng nhanh và gây nguy hiểm khi không được theo dõi, vì vậy việc giám sát liên tục các thông số như nhiệt độ và độ ẩm có ý nghĩa quan trọng đối với sự thoải mái, an toàn và khả năng phản ứng của hệ thống điều khiển [1].

Nhiệt độ và độ ẩm trong cabin ô tô là các yếu tố ảnh hưởng trực tiếp đến cảm giác nhiệt, sức khỏe và khả năng tập trung của người sử dụng xe. Trong các nghiên cứu về điều khiển nhiệt trong cabin, mục tiêu của hệ thống không chỉ là giữ một giá trị nhiệt độ cố định, mà còn phải duy trì trạng thái môi trường tạo cảm giác dễ chịu cho người ngồi trong xe [2]. Nhiệt độ quá cao có thể gây mệt mỏi, khó chịu, đặc biệt trong điều kiện thời tiết nắng nóng; trong khi độ ẩm không phù hợp có thể làm thay đổi cảm giác nhiệt, gây đọng sương trên kính, làm giảm tầm nhìn và chất lượng môi trường bên trong xe. Vì vậy, việc xây dựng một hệ thống có khả năng giám sát nhiệt độ, độ ẩm và hỗ trợ điều chỉnh môi trường cabin thông qua các cơ cấu chấp hành như quạt và servo là một hướng nghiên cứu có ý nghĩa thực tiễn.

Hiện nay, nhiều hệ thống giám sát môi trường được triển khai trên nền vi điều khiển hoặc vi xử lý thương mại. Tuy nhiên, trong phạm vi nghiên cứu và phát triển hệ thống nhúng chuyên dụng, FPGA kết hợp với CPU RISC-V mang lại một hướng tiếp cận linh hoạt hơn. FPGA cho phép mô tả và tổng hợp các khối phần cứng theo yêu cầu, trong khi RISC-V là kiến trúc tập lệnh mở, có cấu trúc module và phù hợp để triển khai các CPU mềm phục vụ học tập, nghiên cứu và tạo mẫu hệ thống số [3], [4].

Trên cơ sở đó, đề tài tập trung xây dựng hệ thống giám sát môi trường cabin ô tô sử dụng CPU RISC-V tích hợp trên kit FPGA Tang Nano 9K. Board Tang Nano 9K có xung nhịp 27 MHz, tài nguyên logic LUT4 và các giao tiếp cơ bản phù hợp cho việc

triển khai thử nghiệm CPU mềm và SoC quy mô nhỏ [5]. Hệ thống sử dụng cảm biến AHT20 để đo nhiệt độ và độ ẩm, màn hình OLED SSD1309 để hiển thị thông tin, giao tiếp với người dùng thông qua UART và nút nhấn, đồng thời điều khiển quạt và servo bằng các khối PWM. Các thiết bị AHT20 và SSD1309 đều phù hợp với hướng tích hợp ngoại vi qua I2C, trong đó I2C là bus hai dây gồm SDA và SCL, thường được dùng cho các IC ngoại vi tốc độ thấp trong hệ thống nhúng [6], [7], [8].

Đề tài giúp làm rõ quá trình thu thập và xử lý dữ liệu cảm biến, cơ chế ánh xạ bộ nhớ, giao tiếp ngoại vi và điều khiển thiết bị bằng phần mềm chạy trên CPU RISC-V. Về mặt ứng dụng, hệ thống mô phỏng một bài toán gần gũi trong lĩnh vực ô tô, đó là theo dõi trạng thái nhiệt ẩm trong cabin, phát hiện điều kiện bất thường và hỗ trợ điều chỉnh môi trường bên trong xe. Đây cũng là cơ sở để phát triển các chức năng nâng cao trong tương lai như giám sát điểm sương, chống đọng kính, điều khiển thông minh theo ngưỡng nhiệt ẩm hoặc tích hợp với hệ thống điều hòa tự động của xe.

1.2. Mục tiêu đề tài

Mục tiêu của đề tài là thiết kế và hiện thực một hệ thống giám sát môi trường cabin ô tô trên nền FPGA sử dụng CPU RISC-V, có khả năng đo lường, hiển thị, điều khiển và giao tiếp với người dùng một cách ổn định. Hệ thống cần đáp ứng được các chức năng cơ bản sau:

Đầu tiên, hệ thống phải có khả năng thu thập dữ liệu môi trường từ cảm biến nhiệt độ và độ ẩm AHT20. Dữ liệu đo được cần được đọc theo chu kỳ, xử lý trong phần mềm chạy trên CPU, sau đó chuyển đổi sang dạng dễ hiển thị và sử dụng trong các thuật toán điều khiển. Việc đo liên tục giúp hệ thống có thể phản ánh trạng thái môi trường gần thời gian thực, từ đó tạo cơ sở cho các phản ứng điều khiển thích hợp.

Thứ hai, hệ thống phải hiển thị thông tin môi trường trên màn hình OLED SSD1309. Nội dung hiển thị cần bao gồm nhiệt độ, độ ẩm, giá trị chênh lệch giữa các cảm biến hoặc giữa môi trường và ngưỡng tham chiếu, cùng với các cảnh báo khi điều kiện vượt ngưỡng cho phép. Màn hình hiển thị đóng vai trò là giao diện trực quan, giúp người dùng dễ quan sát tình trạng hệ thống mà không cần kết nối thêm thiết bị ngoài.

Thứ ba, hệ thống phải hỗ trợ tương tác người dùng thông qua nút nhấn GPIO và kênh UART phục vụ gỡ lỗi. Nút nhấn trên board cho phép thay đổi nhanh các mức hoạt động

của quạt và servo, trong khi UART được dùng để in thông báo trạng thái, hỗ trợ kiểm tra trong quá trình thử nghiệm.

Thứ tư, hệ thống phải điều khiển được các cơ cấu chấp hành như quạt và servo. Quạt được điều khiển bằng xung PWM với các mức độ rộng xung khác nhau tương ứng với các mức tốc độ gió, còn servo được điều khiển bằng PWM để mô phỏng cơ cấu đóng mở hoặc điều chỉnh hướng gió. Nhờ đó, hệ thống không chỉ dừng lại ở mức giám sát mà còn tiến đến mức phản ứng tự động đối với trạng thái môi trường.

Thứ năm, đề tài hướng đến việc xây dựng một hệ thống SoC hoàn chỉnh trên FPGA với CPU RISC-V làm trung tâm. CPU cần kết nối với các ngoại vi như SRAM, UART, I2C, PWM, GPIO, LED và bộ đếm thời gian thông qua cơ chế ánh xạ bộ nhớ. Cách tổ chức này giúp phần mềm có thể truy cập ngoại vi bằng các thao tác đọc ghi bộ nhớ, đúng với đặc trưng của hệ thống nhúng hiện đại.

Bên cạnh các mục tiêu chức năng, đề tài còn đặt ra mục tiêu học thuật là tìm hiểu sâu hơn về kiến trúc máy tính, phương pháp thiết kế SoC cho hệ thống nhúng với tài nguyên hạn chế và phương pháp tích hợp phần cứng với phần mềm.

1.3. Phạm vi giới hạn đề tài

Đề tài được triển khai ở mức mô hình nguyên mẫu trên kit FPGA Tang Nano 9K với tài nguyên phần cứng và thời gian thực hiện có giới hạn. Hệ thống sử dụng CPU RISC-V, SRAM dung lượng nhỏ, cảm biến AHT20 để đo nhiệt độ và độ ẩm, màn hình OLED SSD1309 để hiển thị số liệu cơ bản, cùng các khối PWM điều khiển quạt và servo nhằm mô phỏng chức năng điều tiết môi trường cabin [5], [6], [7]. Vì vậy, đề tài chưa hướng đến một bộ điều khiển ô tô thương mại hoàn chỉnh, chưa tích hợp các cảm biến mở rộng, giao diện đồ họa phức tạp, truyền thông CAN/LIN/OBD-II hay kết nối trực tiếp với hệ thống điện của xe thật. Phần mềm nhúng chạy theo vòng lặp chính, không sử dụng hệ điều hành thời gian thực hoặc các cơ chế xử lý phức tạp như cache, pipeline nâng cao và quản lý bộ nhớ. Trong phạm vi này, đề tài tập trung chứng minh tính khả thi của mô hình giám sát và điều khiển môi trường cabin trên nền RISC-V FPGA, đồng thời làm cơ sở cho các hướng phát triển mở rộng trong tương lai.

1.4. Phương pháp nghiên cứu

Trong quá trình thực hiện đề tài đã sử dụng một số phương pháp sau:

- Phương pháp tổng hợp tài liệu lý thuyết, những kiến thức đã được học.
- Phương pháp khảo sát những hệ thống thực tế trước đó.
- Phương pháp thiết kế và thực nghiệm: kiểm tra, đánh giá, nhận xét hoạt động của toàn hệ thống.
- Phương pháp tham khảo ý kiến từ người hướng dẫn.

1.5. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đề tài là hệ thống giám sát môi trường cabin ô tô trên nền FPGA sử dụng CPU RISC-V, bao gồm các khối cảm biến, hiển thị, giao tiếp và điều khiển chấp hành. Phạm vi nghiên cứu tập trung vào việc đo nhiệt độ và độ ẩm bằng cảm biến AHT20, hiển thị dữ liệu lên màn hình OLED, sử dụng UART cho gỡ lỗi, xử lý nút nhấn GPIO và điều khiển quạt, servo bằng PWM.

1.6. Bố cục quyển báo cáo

Bố cục của báo cáo được chia thành 6 chương như sau:

Chương 1: Tổng quan. Trình bày lý do chọn đề tài, mục tiêu, phạm vi giới hạn, phương pháp nghiên cứu và bố cục báo cáo.

Chương 2: Cơ sở lý thuyết. Trình bày các kiến thức nền tảng về RISC-V, tập lệnh RV32I, FPGA, Verilog và các giao tiếp ngoại vi sử dụng trong hệ thống.

Chương 3: Thiết kế phần cứng của hệ thống. Trình bày kiến trúc CPU RV32I, SoC, bus hệ thống, ánh xạ bộ nhớ và các ngoại vi như SRAM, UART, I2C, PWM, GPIO, LED và Countdown Timer.

Chương 4: Thiết kế phần mềm nhúng chạy trên CPU RISC-V. Trình bày cấu trúc chương trình C, quá trình khởi tạo hệ thống, đọc cảm biến, xử lý dữ liệu, hiển thị OLED và điều khiển quạt, servo.

Chương 5: Kết quả. Trình bày mô hình thi công, kết quả hiển thị, các kịch bản kiểm thử và đánh giá hoạt động của hệ thống.

Chương 6: Kết luận và hướng phát triển. Tổng kết kết quả đạt được, nêu hạn chế và đề xuất các hướng cải tiến trong tương lai.

CHƯƠNG 2 CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về kiến trúc RISC-V

2.1.1. Vi xử lý RISC-V

Kiến trúc máy tính truyền thống bao gồm hai loại chính: kiến trúc Von Neumann và kiến trúc Harvard[10], [11]. Trong khi kiến trúc Von Neumann lưu trữ cả dữ liệu và lệnh trong cùng một không gian bộ nhớ, thì kiến trúc Harvard chia bộ nhớ thành hai không gian riêng biệt để lưu trữ lệnh và dữ liệu. Sự khác biệt về bộ nhớ này cũng xuất hiện trong kiến trúc RISC, với bộ nhớ lệnh và bộ nhớ dữ liệu tách biệt nhau, giúp tăng hiệu suất xử lý và giảm độ trễ khi truy xuất thông tin.

Kiến trúc RISC (Reduced Instruction-Set Computing – tính toán với tập lệnh rút gọn) ra đời vào đầu những năm 1980, bắt nguồn từ các nghiên cứu tại Đại học California, Berkeley và Đại học Stanford. Mục tiêu ban đầu là đơn giản hóa tập lệnh của bộ vi xử lý để tăng tốc độ thực thi và hiệu quả sử dụng tài nguyên phần cứng [10]. Trong khi các kiến trúc CISC (Complex Instruction-Set Computing) như x86 sử dụng các lệnh phức tạp, RISC tập trung vào các lệnh đơn giản, dễ giải mã và thực thi nhanh chóng.

Một đại diện tiêu biểu của kiến trúc RISC hiện đại là RISC-V, được phát triển vào năm 2010 tại Đại học California, Berkeley. RISC-V là kiến trúc tập lệnh mở, miễn phí bản quyền, cho phép các nhà nghiên cứu, doanh nghiệp và tổ chức giáo dục tự do sử dụng và tùy biến theo nhu cầu mà không bị ràng buộc bởi các giấy phép thương mại.

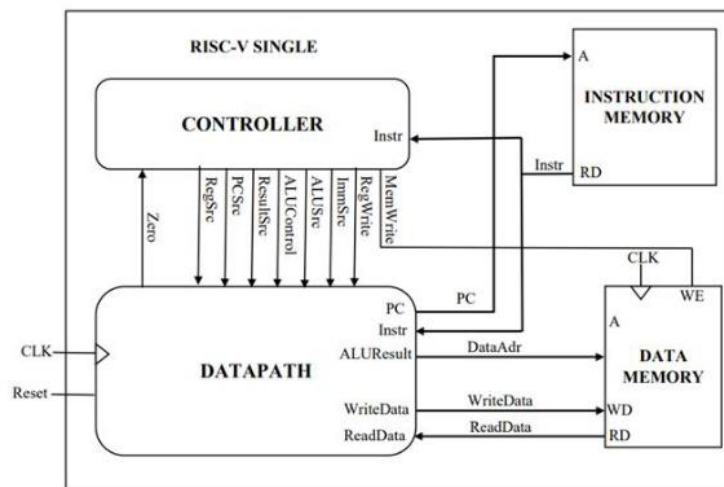
Ưu điểm của kiến trúc RISC-V:

- Đơn giản và dễ triển khai: Tập lệnh rút gọn giúp giảm độ phức tạp trong thiết kế phần cứng và phần mềm.
- Hiệu suất cao: Các lệnh đơn giản cho phép thực thi nhanh hơn, đặc biệt trong các hệ thống nhúng và thiết bị di động.
- Tiêu thụ năng lượng thấp: Phù hợp với các ứng dụng yêu cầu tiết kiệm năng lượng như IoT.
- Tính mở và linh hoạt: RISC-V không bị ràng buộc bản quyền, dễ dàng mở rộng và tùy biến.
- Hỗ trợ cộng đồng mạnh mẽ: Được nhiều tổ chức nghiên cứu và công ty công nghệ lớn ủng hộ.

Nhược điểm của kiến trúc RISC-V:

- Thiếu hỗ trợ phần mềm thương mại: So với các kiến trúc lâu đời như x86, hệ sinh thái phần mềm của RISC-V vẫn đang trong quá trình phát triển.
- Hiệu suất chưa tối ưu trong một số ứng dụng phức tạp: Với các tác vụ yêu cầu xử lý phức tạp, kiến trúc CISC có thể mang lại hiệu quả cao hơn.
- Cần thời gian để phổ biến: Dù tiềm năng lớn, RISC-V vẫn cần thời gian để được chấp nhận rộng rãi trong ngành công nghiệp.

Sơ đồ kiến trúc RISC-V Single:



Hình 2.1 Sơ đồ khối single risc-v [11]

Sơ đồ kiến trúc RISC gồm 4 khối chính: khối bộ nhớ lệnh (Instruction memory), khối điều khiển (Controller), khối đường dẫn dữ liệu (Datapath) và khối bộ nhớ dữ liệu (Data memory). Với đặc điểm bộ nhớ như đã giới thiệu ở trên, chức năng chính của khối điều khiển là đưa ra tín hiệu để khối đường dẫn dữ liệu thực thi đúng thời điểm.

2.1.2. Các phương pháp triển khai kiến trúc RISC-V

Một kiến trúc tập lệnh như RISC-V có thể được hiện thực bằng nhiều tổ chức vi kiến trúc khác nhau. Mặc dù cùng thực thi một tập lệnh, các bộ xử lý có thể khác nhau đáng kể về cách tổ chức đường dẫn dữ liệu, khối điều khiển, số chu kỳ thực thi lệnh, mức độ song song và hiệu năng tổng thể. Các phương pháp triển khai phổ biến gồm chu kỳ đơn, đa chu kỳ, đường ống và superscalar.

- **Kiến trúc đơn chu kỳ (Single-cycle)**

Trong kiến trúc chu kỳ đơn, mỗi lệnh được thực thi hoàn toàn trong một chu kỳ xung nhịp. Toàn bộ các bước xử lý như nạp lệnh, giải mã lệnh, đọc thanh ghi, thực thi phép toán, truy xuất bộ nhớ và ghi kết quả đều diễn ra trong cùng một chu kỳ.

Ưu điểm của phương pháp này là cấu trúc đơn giản, dễ thiết kế, dễ mô phỏng và phù hợp với mục tiêu học tập hoặc các hệ thống có quy mô nhỏ. Tuy nhiên, do chu kỳ xung nhịp phải đủ dài để hoàn thành lệnh phức tạp nhất, các lệnh đơn giản cũng phải chờ hết cùng một khoảng thời gian. Điều này làm giảm hiệu quả sử dụng thời gian và khiến hiệu năng tổng thể thấp hơn so với các phương pháp triển khai nâng cao.

- **Kiến trúc đa chu kỳ (Multicycle)**

Trong kiến trúc đa chu kỳ, quá trình thực thi lệnh được chia thành nhiều bước nhỏ, mỗi bước được thực hiện trong một chu kỳ xung nhịp riêng biệt. Các giai đoạn như nạp lệnh, giải mã, thực thi, truy xuất bộ nhớ và ghi kết quả có thể được thực hiện tuần tự qua nhiều chu kỳ.

Cách triển khai này cho phép mỗi loại lệnh sử dụng số chu kỳ phù hợp với độ phức tạp của nó. Những lệnh đơn giản có thể hoàn thành trong ít chu kỳ hơn, trong khi các lệnh truy xuất bộ nhớ hoặc rẽ nhánh có thể cần nhiều chu kỳ hơn. Ngoài ra, kiến trúc đa chu kỳ giúp tiết kiệm tài nguyên phần cứng nhờ khả năng tái sử dụng các khối chức năng trong nhiều giai đoạn khác nhau. Tuy nhiên, hệ thống điều khiển phức tạp hơn so với kiến trúc chu kỳ đơn, và thông lượng xử lý lệnh vẫn bị giới hạn do các lệnh thường được thực thi tuần tự.

- **Kiến trúc đường ống (Pipelined)**

Kiến trúc đường ống chia quá trình thực thi lệnh thành nhiều giai đoạn, thường gồm năm giai đoạn cơ bản: nạp lệnh, giải mã lệnh, thực thi, truy xuất bộ nhớ và ghi kết quả. Trong mô hình này, nhiều lệnh có thể được xử lý đồng thời, mỗi lệnh nằm ở một giai đoạn khác nhau của pipeline.

Ưu điểm chính của kiến trúc đường ống là cải thiện thông lượng xử lý. Sau khi pipeline được lấp đầy, bộ xử lý có thể hoàn thành gần như một lệnh trong mỗi chu kỳ xung nhịp. Tuy nhiên, việc triển khai pipeline làm tăng độ phức tạp của thiết kế do cần xử lý các vấn đề như xung đột dữ liệu, xung đột điều khiển, xung đột tài nguyên và tác động của các lệnh rẽ nhánh. Ngoài ra, kiến trúc này cần bổ sung các thanh ghi trung gian

giữa các giai đoạn và các cơ chế hỗ trợ như forwarding, stall hoặc flush để đảm bảo tính đúng đắn khi thực thi chương trình.

- **Kiến trúc superscalar**

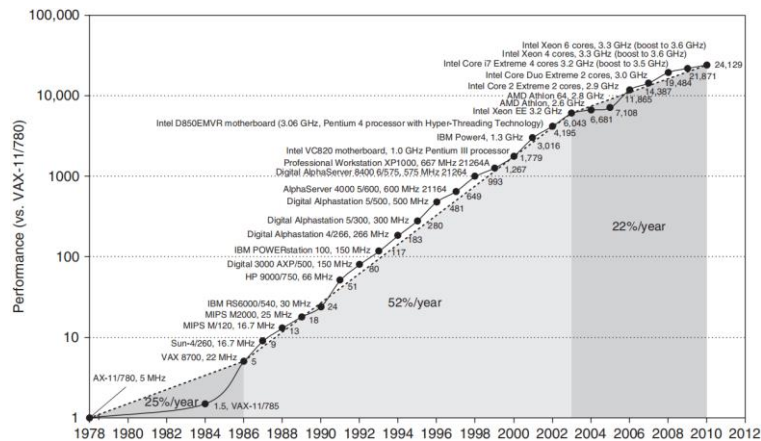
Superscalar là hướng triển khai vi xử lý có khả năng phát hành và thực thi nhiều lệnh trong cùng một chu kỳ xung nhịp. Để đạt được điều này, bộ xử lý superscalar thường được trang bị nhiều đơn vị thực thi hoạt động song song, cho phép xử lý đồng thời nhiều lệnh độc lập.

Kiến trúc superscalar thường kết hợp nhiều kỹ thuật nâng cao như pipelining, multiple issue, thực thi ngoài thứ tự và cơ chế dự đoán nhánh. Nhờ đó, hiệu năng có thể được cải thiện đáng kể so với kiến trúc pipeline đơn giản. Tuy nhiên, đổi lại là độ phức tạp phần cứng rất cao, yêu cầu nhiều tài nguyên logic, cơ chế điều phối lệnh phức tạp, cũng như các khối kiểm soát phụ thuộc dữ liệu và trạng thái chương trình. Vì vậy, superscalar thường phù hợp với các bộ xử lý hiệu năng cao hơn là các hệ thống nhúng nhỏ hoặc các thiết kế FPGA có tài nguyên hạn chế.

2.2. Sự phát triển của kiến trúc RISC

Trước năm 1980, phần lớn bộ xử lý thương mại sử dụng kiến trúc CISC với nhiều lệnh phức tạp nhằm giảm kích thước chương trình. Tuy nhiên, khảo sát trên các ứng dụng thực tế cho thấy phần lớn phần mềm chỉ sử dụng một nhóm nhỏ lệnh đơn giản, trong khi các lệnh phức tạp dù tốn nhiều tài nguyên phần cứng lại rất ít được dùng. Cùng với sự cải tiến của trình biên dịch và công nghệ bộ nhớ, những hạn chế của CISC như độ dài lệnh thay đổi, nhiều chế độ địa chỉ và bộ giải mã phức tạp trở thành rào cản đối với pipeline hiệu quả.

Từ những quan sát này, nhiều nhóm nghiên cứu hướng tới tập lệnh đơn giản, đều đặn và dễ pipeline, dẫn đến sự hình thành kiến trúc RISC. Ba dự án tiên phong gồm: IBM 801 (giảm độ phức tạp tập lệnh), Berkeley RISC (nhiều thanh ghi, lệnh đơn giản, pipeline hiệu quả) và Stanford MIPS (pipeline 5 giai đoạn, delay slot). Những dự án này chứng minh rằng thiết kế dựa trên tập lệnh đơn giản có thể đạt hiệu năng vượt trội so với CISC cùng thời.



Hình 2.2 Sự tăng trưởng về hiệu năng của bộ xử lý [13]

Từ thực nghiệm các hệ thống RISC đời đầu, Patterson & Hennessy đã tổng kết bốn nguyên lý nền tảng:

- Đơn giản giúp thiết kế đều đặn.
- Tập lệnh nhỏ cho phép thực thi nhanh.
- Ưu tiên tối ưu hóa các trường hợp phổ biến.
- Hiệu quả thiết kế cần sự cân bằng giữa hiệu năng và chi phí phần cứng.

Sự đối lập giữa hai triết lý CISC–RISC thể hiện rõ ở các đặc điểm: CISC có lệnh dài biến đổi, nhiều chế độ định địa chỉ và thao tác trực tiếp trên bộ nhớ; RISC sử dụng lệnh độ dài cố định, định dạng đơn giản, kiến trúc load–store và phần lớn lệnh thực thi trong một chu kỳ.

Bảng 2.1: Phân tích sự khác biệt giữa hai kiến trúc CISC–RISC [12]

CISC	RISC
Tập trung tối ưu phần cứng	Tập trung tối ưu phần mềm và trình biên dịch
Độ dài lệnh thay đổi	Độ dài lệnh cố định (single-word instruction)
Định dạng lệnh đa dạng	Giải mã theo trường cố định (fixed-field decoding)
Cho phép thao tác trực tiếp trên bộ nhớ	Kiến trúc load–store (chỉ lệnh LOAD/STORE mới truy cập bộ nhớ)
Hỗ trợ nhiều lệnh phức tạp	Lệnh đơn giản, thực thi nhanh

Một lệnh có thể cần nhiều chu kỳ	Phần lớn lệnh thực thi trong một chu kỳ xung
Tập lệnh lớn, nhiều lệnh phức tạp	Tập lệnh nhỏ, chỉ chứa các lệnh cơ bản
Lệnh có dạng “memory-to-memory”, LOAD/STORE tích hợp trong lệnh	Lệnh dạng “register-to-register”, LOAD và STORE là lệnh riêng
Nhiều transistor dùng cho phần cứng hỗ trợ lệnh phức tạp	Nhiều transistor dành cho mở rộng số lượng thanh ghi

Trên nền tảng đó, RISC-V ra đời năm 2010 tại UC Berkeley như một ISA mở, miễn phí bản quyền và dễ mở rộng. Với tập lệnh cơ sở nhỏ gọn (RV32I) cùng nhiều phần mở rộng tùy chọn, RISC-V kế thừa trực tiếp tinh thần của Berkeley RISC và nhanh chóng được ứng dụng rộng rãi trong hệ thống nhúng, IoT, vi điều khiển và các bộ xử lý hiệu năng cao. Việc sử dụng RISC-V trong đồ án phù hợp xu hướng thiết kế hiện đại, nơi tính đơn giản, linh hoạt và khả năng mở rộng được ưu tiên.

2.3. Lịch sử phát triển của RISC-V

RISC-V là kiến trúc tập lệnh thế hệ mới được phát triển tại Đại học California, Berkeley, kế thừa hơn ba thập kỷ nghiên cứu về kiến trúc RISC. Mặc dù chỉ mới xuất hiện trong thập niên gần đây, RISC-V nhanh chóng trở thành một ISA quan trọng nhờ tính mở, sự đơn giản và khả năng mở rộng linh hoạt.

Giai đoạn hình thành tư tưởng RISC (1980–1990): các dự án RISC-I (1981) và RISC-II (1983) tại UC Berkeley chứng minh rằng tập lệnh đơn giản, nhiều thanh ghi và pipeline rõ ràng giúp tăng hiệu năng và giảm độ phức tạp phần cứng. Đây là nền tảng cho các ISA thương mại như MIPS và là tiền đề trực tiếp cho RISC-V (Sarah L. Harris, David Money Harris, 2021).

Nhu cầu về một ISA mở (2000–2010): trong giai đoạn này, các ISA phổ biến đều có hạn chế: x86 quá phức tạp, ARM/MIPS yêu cầu cấp phép, còn nhiều ISA học thuật không còn phù hợp với SoC hiện đại. Do đó, xuất hiện nhu cầu về một ISA hiện đại, tinh gọn, miễn phí bản quyền và có khả năng tùy chỉnh cao.

Sự ra đời của RISC-V (2010–2011): năm 2010, Patterson và Asanović khởi động dự án RISC-V với ba mục tiêu: đơn giản – dễ hiện thực, module hóa theo phần mở rộng, và hoàn toàn mở. Đến năm 2011, phiên bản RV32I đầu tiên được hoàn thiện.

Công bố và phát triển hệ sinh thái (2014–2016): năm 2014, RISC-V được công bố rộng rãi cùng các công cụ phần mềm như GCC/LLVM, Spike, assembler và hỗ trợ Linux/FreeBSD. Năm 2015, RISC-V Foundation được thành lập với sự tham gia của nhiều tập đoàn lớn, đánh dấu bước chuyển sang tiêu chuẩn công nghiệp.

Thương mại hóa và tăng trưởng mạnh (2017–nay): từ 2017, RISC-V được thương mại hóa mạnh mẽ: SiFive ra mắt SoC đầu tiên, Western Digital áp dụng cho controller HDD/SSD và từ 2019–2024, RISC-V xuất hiện trong IoT, MCU, thiết bị đeo, drone, SSD controller và các bộ xử lý dùng RVV cho AI/HPC.

Lý do RISC-V phát triển nhanh: RISC-V được ưa chuộng nhờ ISA mở miễn phí bản quyền, thiết kế đơn giản dễ triển khai trên FPGA/ASIC, khả năng module hóa phù hợp từ nhúng đến HPC/AI và hệ sinh thái phần mềm ngày càng lớn mạnh

2.4. Giới thiệu ngôn ngữ mô tả phần cứng Verilog

Verilog HDL (Hardware Description Language) là một ngôn ngữ mô tả phần cứng được sử dụng rộng rãi trong lĩnh vực thiết kế và phát triển các hệ thống số. Ngôn ngữ này cho phép người thiết kế mô tả cấu trúc và hành vi của phần cứng điện tử như mạch logic, bộ điều khiển, bộ nhớ và bộ xử lý. Verilog không chỉ hỗ trợ việc xây dựng mô hình phần cứng ở mức trừu tượng cao mà còn có thể chuyển đổi trực tiếp thành mạch vật lý thông qua quá trình tổng hợp.

Verilog được sử dụng để mô tả, mô phỏng và kiểm tra các thiết kế phần cứng trước khi triển khai trên các nền tảng như FPGA hoặc sản xuất dưới dạng chip ASIC. Nhờ khả năng mô phỏng, người thiết kế có thể phát hiện và sửa lỗi từ giai đoạn sớm, giúp giảm chi phí và thời gian phát triển. Đồng thời, Verilog đóng vai trò quan trọng trong việc đảm bảo tính chính xác và độ tin cậy của hệ thống số.

Một trong những đặc điểm nổi bật của Verilog là khả năng mô tả các khối phần cứng hoạt động song song. Khác với các ngôn ngữ lập trình phần mềm hoạt động theo trình tự, Verilog phản ánh đúng bản chất của phần cứng, nơi nhiều khối logic có thể xử lý tín hiệu cùng lúc. Ngoài ra, ngôn ngữ này hỗ trợ nhiều mức độ mô tả khác nhau, bao

gồm mức cổng logic, mức truyền thanh ghi (RTL) và mức hành vi, giúp người thiết kế linh hoạt trong quá trình phát triển hệ thống.

- Máy trạng thái hữu hạn (FSM) trong Verilog

Máy trạng thái hữu hạn là một mô hình thiết kế logic tuần tự được sử dụng phổ biến trong các hệ thống số. FSM mô tả hoạt động của mạch thông qua một tập hữu hạn các trạng thái, trong đó hệ thống chuyển từ trạng thái này sang trạng thái khác dựa trên tín hiệu vào, điều kiện điều khiển và xung nhịp. Trong Verilog, FSM thường được sử dụng để thiết kế các khối điều khiển có trình tự rõ ràng như bộ điều khiển giao tiếp I2C, UART, SPI, mạch khởi tạo OLED, bộ điều phối bus hoặc các khối điều khiển ngoại vi.

Về nguyên lý, FSM thường được phân thành hai dạng chính là Moore và Mealy. Đối với FSM Moore, tín hiệu đầu ra chỉ phụ thuộc vào trạng thái hiện tại của hệ thống. Cách thiết kế này có ưu điểm là đầu ra ổn định, dễ kiểm soát và phù hợp với các khối điều khiển cần độ tin cậy cao. Đối với FSM Mealy, tín hiệu đầu ra phụ thuộc đồng thời vào trạng thái hiện tại và tín hiệu đầu vào. Nhờ đó, FSM Mealy có thể phản ứng nhanh hơn với sự thay đổi của tín hiệu vào, nhưng việc thiết kế và kiểm tra cần cẩn thận hơn để tránh nhiễu hoặc thay đổi đầu ra không mong muốn.

Bảng 2.2. So sánh FSM Moore và FSM Mealy [11]

Tiêu chí	FSM Moore	FSM Mealy
Phụ thuộc đầu ra	Chỉ phụ thuộc trạng thái hiện tại	Phụ thuộc trạng thái hiện tại và đầu vào
Độ ổn định đầu ra	Ổn định hơn	Có thể thay đổi nhanh theo đầu vào
Tốc độ phản hồi	Thường chậm hơn một nhịp	Phản hồi nhanh hơn
Độ phức tạp khi thiết kế	Dễ kiểm soát	Cần kiểm soát điều kiện tổ hợp kỹ hơn
Ứng dụng phù hợp	Bộ điều khiển tuần tự, giao thức đơn giản	Điều khiển cần phản hồi nhanh theo tín hiệu vào

Trong đề tài này, FSM được sử dụng như một phương pháp tổ chức điều khiển phần cứng trong Verilog. Các trạng thái được dùng để mô tả từng bước hoạt động của mạch, chẳng hạn như chờ tín hiệu bắt đầu, phát lệnh, đọc dữ liệu, ghi dữ liệu, kiểm tra

điều kiện hoàn thành và quay lại trạng thái chờ. Cách tiếp cận này giúp thiết kế phần cứng trở nên rõ ràng, dễ mô phỏng, dễ kiểm thử và thuận lợi hơn khi tích hợp nhiều ngoại vi vào hệ thống SoC trên FPGA.

2.5. Giới thiệu về kiến trúc tập lệnh và tập lệnh RV32I

2.5.1. Kiến trúc tập lệnh

Kiến trúc tập lệnh (ISA) là một tập hợp các chỉ thị mà bộ vi xử lý có thể hiểu và thực thi. Nó đóng vai trò như một giao diện giữa phần cứng và phần mềm, xác định cách phần mềm điều khiển phần cứng để thực hiện các tác vụ tính toán. ISA quy định các loại lệnh, định dạng lệnh, chế độ địa chỉ, và tập thanh ghi mà bộ xử lý sử dụng.

2.5.2. Kiến trúc và tập lệnh RV32I

2.5.2.1. Kiến trúc RV32I

RV32I là một tập con cơ bản của kiến trúc RISC-V, được thiết kế cho các bộ vi xử lý 32-bit. Tập lệnh này bao gồm các chỉ thị cần thiết để thực hiện các phép toán số học, logic, điều khiển luồng chương trình, và truy cập bộ nhớ. Với thiết kế đơn giản, nhất quán và dễ mở rộng.

Trong kiến trúc RISC-V, mô hình lập trình viên đóng vai trò là cầu nối giữa phần mềm và phần cứng, định nghĩa cách các thanh ghi và bộ đếm chương trình (PC) được sử dụng trong quá trình thực thi lệnh. Phiên bản RV32I cung cấp 32 thanh ghi tổng quát 32-bit, cùng với một thanh ghi PC, tạo nên một môi trường đơn giản nhưng hiệu quả để triển khai các thuật toán và hệ thống phần mềm. Mặc dù không có các thanh ghi chuyên biệt cho con trỏ ngăn xếp hay địa chỉ trả về, RISC-V vẫn tuân theo quy ước phần mềm chuẩn để đảm bảo khả năng tương thích và tối ưu hóa hiệu năng. Dưới đây sẽ trình bày chi tiết về cấu trúc thanh ghi, vai trò của từng thanh ghi, và các quy ước phần mềm liên quan trong mô hình lập trình viên của RV32I.

Mô hình lập trình viên cho tập lệnh cơ sở RV32I:

- x0 mặc định giá trị là 0 nó chỉ đọc không thể ghi được
- x1(ra - return address) thường được dùng để lưu trữ địa chỉ trả về
- x2(sp - stack pointer) là con trỏ ngăn xếp dùng trong việc quản lý ngăn xếp.
- x3(gp - global pointer) là con trỏ toàn cục trỏ đến dữ liệu toàn cục

- x4(tp – thread pointer) là con trỏ luồng được sử dụng trong các chương trình đa luồng để trỏ tới dữ liệu cụ thể của luồng
- x5 đến x7(t0-t2-temporaries) và x28 đến x31(t3-t6-temporaries) thanh ghi tạm thời sử dụng để lưu trữ dữ liệu tạm thời trong quá trình tính toán
- x8 (s0/fp – saved register/ frame pointer) có thể được sử dụng như một thanh ghi lưu trữ dữ liệu lâu dài (dữ liệu không đổi sau các lệnh gọi hàm) hoặc là con trỏ khung trỏ tới khung ngăn xếp hiện tại.
- x9(s1- saved registers) và x18 đến x27(s2-s11 - saved registers) lưu trữ dữ liệu lâu dài không thay đổi sau các lệnh gọi hàm.
- x10 đến x17 (a0-a7 - argument register) là các thanh ghi được sử dụng cho việc truyền đối số cho hàm và trả về giá trị từ hàm.

Trong phạm vi đề tài, CPU được thiết kế theo tập lệnh cơ sở RV32I, tập trung vào các nhóm lệnh cần thiết để chạy chương trình nhúng như số học, logic, nạp giá trị tức thời, điều khiển luồng và truy cập bộ nhớ. Đối với nhóm truy cập bộ nhớ, phần cứng CPU có khối xử lý Load/Store để hỗ trợ các thao tác đọc/ghi theo byte, half-word và word. Tuy nhiên, trong chương trình ứng dụng của hệ thống giám sát môi trường cabin, các thao tác truy cập bộ nhớ chủ yếu sử dụng lệnh LW và SW. Các lệnh như FENCE, PAUSE, ECALL và EBREAK không phải là trọng tâm sử dụng trong chương trình ứng dụng của đề tài này.

Dưới đây là bảng tập lệnh RV32I

Bảng 2.3. Bảng tập lệnh RV32I [14]

RV32I Base Instruction Set

imm[31:12]				rd	0110111	LUI	
imm[31:12]				rd	0010111	AUIPC	
imm[20 10:1 11 19:12]				rd	1101111	JAL	
imm[11:0]			rs1	000	rd	1100111	JALR
imm[12 10:5]	rs2	rs1	000	imm[4:1 11]	1100011	BEQ	
imm[12 10:5]	rs2	rs1	001	imm[4:1 11]	1100011	BNE	
imm[12 10:5]	rs2	rs1	100	imm[4:1 11]	1100011	BLT	
imm[12 10:5]	rs2	rs1	101	imm[4:1 11]	1100011	BGE	
imm[12 10:5]	rs2	rs1	110	imm[4:1 11]	1100011	BLTU	
imm[12 10:5]	rs2	rs1	111	imm[4:1 11]	1100011	BGEU	
imm[11:0]			rs1	000	rd	0000011	LB
imm[11:0]			rs1	001	rd	0000011	LH
imm[11:0]			rs1	010	rd	0000011	LW
imm[11:0]			rs1	100	rd	0000011	LBU
imm[11:0]			rs1	101	rd	0000011	LHU
imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	SB	
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	SH	
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	SW	
imm[11:0]			rs1	000	rd	0010011	ADDI
imm[11:0]			rs1	010	rd	0010011	SLTI
imm[11:0]			rs1	011	rd	0010011	SLTIU
imm[11:0]			rs1	100	rd	0010011	XORI
imm[11:0]			rs1	110	rd	0010011	ORI
imm[11:0]			rs1	111	rd	0010011	ANDI
0000000		shamt	rs1	001	rd	0010011	SLLI
0000000		shamt	rs1	101	rd	0010011	SRLI
0100000		shamt	rs1	101	rd	0010011	SRAI
0000000		rs2	rs1	000	rd	0110011	ADD
0100000		rs2	rs1	000	rd	0110011	SUB
0000000		rs2	rs1	001	rd	0110011	SLL
0000000		rs2	rs1	010	rd	0110011	SLT
0000000		rs2	rs1	011	rd	0110011	SLTU
0000000		rs2	rs1	100	rd	0110011	XOR
0000000		rs2	rs1	101	rd	0110011	SRL
0100000		rs2	rs1	101	rd	0110011	SRA
0000000		rs2	rs1	110	rd	0110011	OR
0000000		rs2	rs1	111	rd	0110011	AND
fm	pred	succ	rs1	000	rd	0001111	FENCE
1000	0011	0011	00000	000	00000	0001111	FENCE.TSO
0000	0001	0000	00000	000	00000	0001111	PAUSE
000000000000			00000	000	00000	1110011	ECALL
000000000001			00000	000	00000	1110011	EBREAK

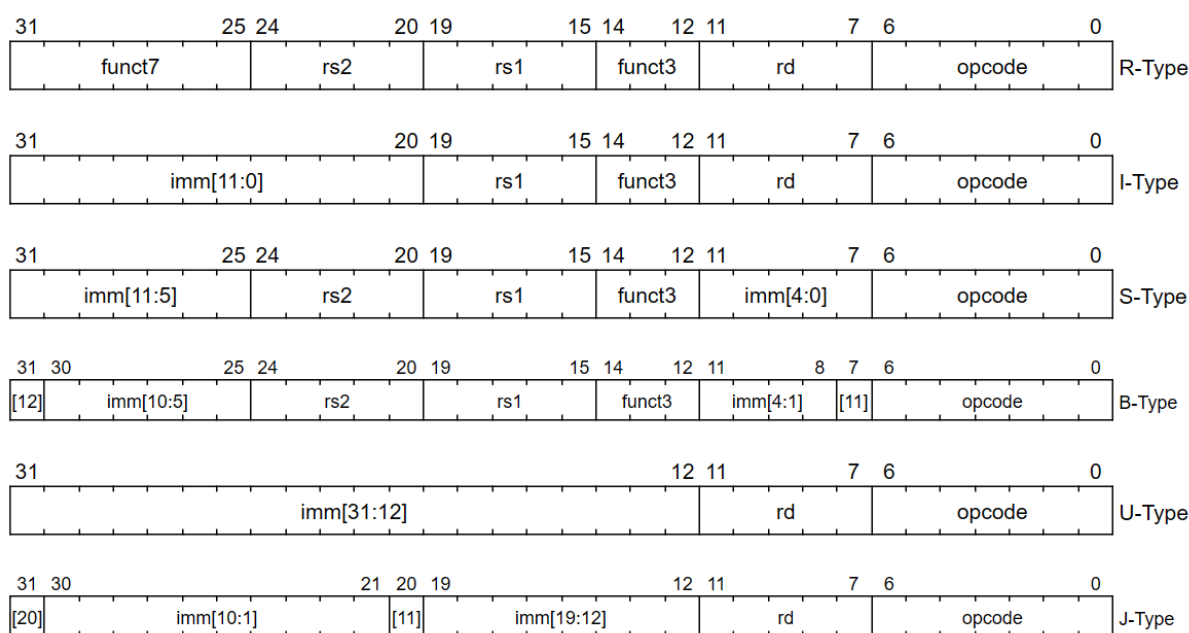
2.5.2.2. Định dạng lệnh

Tập lệnh RV32I sử dụng các định dạng lệnh cố định 32-bit nhằm đơn giản hóa quá trình giải mã và thực thi trong phần cứng. Có bốn định dạng chính: R-type, I-type, S-type, và U-type, mỗi định dạng phục vụ cho một nhóm lệnh cụ thể như toán học, truy cập bộ nhớ, hoặc điều khiển luồng chương trình. Các lệnh phải được căn chỉnh theo biên 4 byte trong bộ nhớ, và việc sai lệch địa chỉ sẽ gây ra ngoại lệ. Việc giữ nguyên vị trí

các trường thanh ghi (rs1, rs2, rd) trong tất cả các định dạng giúp giảm độ phức tạp trong thiết kế phần cứng và tăng hiệu suất giải mã lệnh.

Ngoài bốn định dạng cơ bản, RV32I còn có hai biến thể định dạng là B-type và J-type, được sử dụng cho các lệnh rẽ nhánh có điều kiện và nhảy không điều kiện. Các biến thể này sử dụng cách mã hóa đặc biệt cho trường hằng số (immediate), nhằm tối ưu hóa không gian opcode và đơn giản hóa phần cứng. Ví dụ, trong B-type, trường hằng số được mã hóa để biểu diễn độ lệch nhánh theo bội số của 2 byte, trong khi J-type dùng độ lệch nhảy theo bội số của 2 byte nhưng với độ rộng lớn hơn. Việc mã hóa hằng số theo cách này giúp giảm độ phức tạp của mạch phần cứng và tăng hiệu quả trong quá trình biên dịch.

Thanh ghi lệnh có dạng 32 bit và có 6 loại lệnh chính là R,I,S,B,U và J :



Hình 2.3. Thanh ghi lệnh trong kiến trúc RISC-V[4].

Tương ứng với mỗi loại thanh ghi lệnh ta có thể thực hiện nhiều phép toán hay logic khác nhau

Các định dạng lệnh trong kiến trúc RV32I:

- R-type: sử dụng các phép toán logic, số học trên thanh ghi. Các lệnh dạng này thực hiện phép toán giữa hai thanh ghi nguồn (rs1, rs2) và lưu kết quả vào thanh ghi đích (rd). Như ADD, SUB, AND, OR, XOR.

- I-type: dùng cho các lệnh thao tác với giá trị tức thời (immediate) hoặc truy xuất bộ nhớ. Một thanh ghi nguồn (rs1) được kết hợp với một giá trị tức thời (imm) để thực hiện phép toán, kết quả được lưu vào thanh ghi đích (rd). Các lệnh phổ biến gồm: ADDI, ORI, ANDI, LW, JALR.
- S-type: dùng cho các lệnh lưu dữ liệu từ thanh ghi vào bộ nhớ. Hai thanh ghi (rs1, rs2) được sử dụng để xác định địa chỉ và dữ liệu cần lưu, trong khi imm xác định offset địa chỉ. Như: SW, SB, SH.
- B-type: dùng cho các lệnh rẽ nhánh có điều kiện. Hai thanh ghi nguồn được so sánh, và nếu điều kiện đúng, chương trình sẽ nhảy đến địa chỉ được tính từ giá trị imm. Ví dụ: BEQ, BNE, BLT, BGE.
- U-type: dùng để nạp một giá trị tức thời 20 bit vào phần cao của thanh ghi đích. Thường dùng để khởi tạo địa chỉ hoặc hằng số lớn. Ví dụ: LUI, AUIPC.
- J-type: dùng cho lệnh nhảy không điều kiện đến một địa chỉ được tính từ giá trị imm. Thanh ghi đích (rd) lưu địa chỉ trả về để hỗ trợ nhảy có liên kết. Ví dụ: JAL.

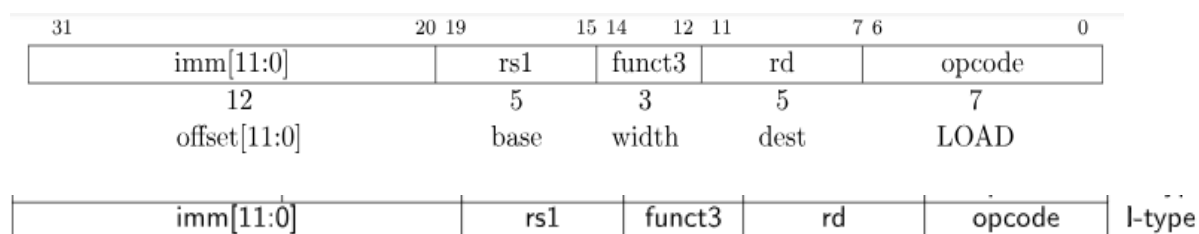
Các trường trong lệnh RV32I để tạo thành một thanh ghi định dạng lệnh:

- Opcode : phân biệt giữa các loại lệnh
- Rd(register destination): gồm 5 bit lưu trữ địa chỉ thanh ghi đích để nhận giá trị trả về trong phép toán.
- Rs1, Rs2(register source): gồm 5 bit lưu trữ địa chỉ thanh ghi nguồn để đưa giá trị vào để tính toán
- Imm(immediate value): gồm 12 bit giá trị để mở rộng dấu(sign-extend) lên 32 bit phù hợp cho tính toán hoặc một giá trị số nguyên được mã hóa trực tiếp trong lệnh, không cần truy xuất từ thanh ghi.
- Funct3: Gồm 3 bit dùng để phân biệt các biến thể của lệnh có cùng opcode. Như ADDI, ORI, ANDI đều có cùng opcode nhưng khác nhau ở funct3.
- Funct7: Gồm 7 bit chỉ xuất hiện trong định dạng R-type. Dùng để phân biệt thêm các lệnh có cùng opcode và funct3. Như ADD và SUB có cùng opcode và funct3, nhưng khác nhau ở funct7.

Trong kiến trúc RISC (Reduced Instruction Set Computer), việc giao tiếp giữa khối thanh ghi và khối bộ nhớ dữ liệu đóng vai trò quan trọng trong quá trình xử lý thông tin. Hai lệnh cơ bản và thiết yếu để thực hiện việc này là lệnh tải (Load Word – LW) và

lệnh lưu trữ (Store Word – SW). Lệnh LW có chức năng tải một từ dữ liệu (thường là 32 bit) từ bộ nhớ chính vào một thanh ghi cụ thể, thông qua địa chỉ được tính bằng cách cộng một offset với giá trị trong thanh ghi cơ sở. Ngược lại, lệnh SW dùng để lưu một từ dữ liệu từ thanh ghi vào bộ nhớ tại địa chỉ xác định. Khối thanh ghi đảm nhiệm các tác vụ tính toán như số học, logic, so sánh và tính địa chỉ, đồng thời lưu trữ tạm thời các giá trị trong quá trình xử lý. Trong khi đó, khối bộ nhớ dữ liệu lưu trữ các giá trị đầu vào, kết quả trung gian và kết quả cuối cùng của chương trình. Kiến trúc RISC có thể thực hiện các lệnh một cách nhanh chóng, chính xác và dễ dàng mở rộng, đồng thời giúp lập trình viên kiểm soát luồng dữ liệu rõ ràng hơn trong quá trình phát triển phần mềm.

- Hình dưới đây thể hiện thanh ghi lệnh tải giống với thanh ghi lệnh I- type

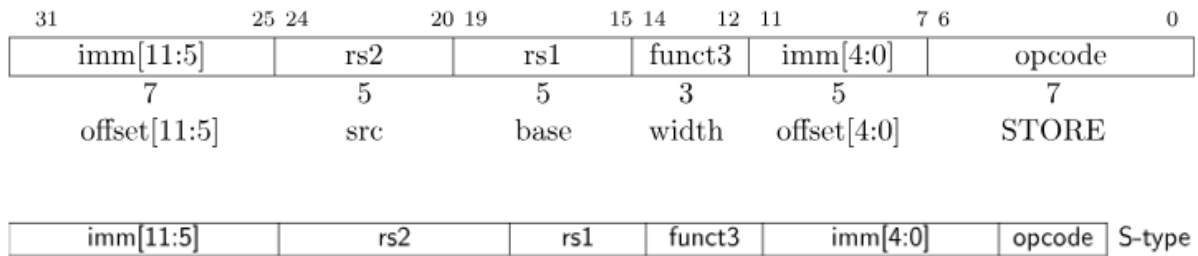


Hình 2.4. Thanh ghi lệnh I-type và thanh ghi lệnh tải trong kiến trúc RISC-V[4]

Funct3	Opcode	Lệnh assembly	Mô tả
000	0000011	Lb x5,12(x7)	Tải 1 byte từ địa chỉ bộ nhớ tính bằng giá trị trong thanh ghi x7 cộng với offset 12 và lưu giá trị đó vào thanh ghi x5.
001	0000011	Lh x5,12(x7)	Tải 2 byte từ địa chỉ bộ nhớ tính bằng giá trị trong thanh ghi x7 cộng với offset 12 và lưu giá trị đó vào thanh ghi x5.
010	0000011	Lw x5, -12(x7)	Tải 4 byte từ địa chỉ bộ nhớ tính bằng giá trị trong thanh ghi x7 cộng với offset (-12) và lưu giá trị đó vào thanh ghi x5.
100	0000011	Lbu x5,12(x7)	Tải 1 byte từ địa chỉ bộ nhớ tính bằng giá trị trong thanh ghi x7 cộng với offset 12 và lưu giá trị đó vào thanh ghi x5.

101	0000011	Lhu x5,12(x7)	Tải 2 byte từ địa chỉ bộ nhớ tính bằng giá trị trong thanh ghi x7 cộng với offset 12 và lưu giá trị đó vào thanh ghi x5.
-----	---------	---------------	--

- Hình dưới đây thể hiện thanh ghi lệnh lưu giống với định dạng lệnh S-type



Hình 2.5. Thanh ghi lệnh lưu và thanh ghi lệnh S-type trong kiến trúc RISC-V [4]

Lệnh	Funct3	Opcode	Mô tả
Sb x5, 12(x7)	000	0100011	Lưu 1 byte lấy giá trị từ thanh ghi x5 lưu vào địa chỉ vật lý được tính bằng giá trị trong thanh ghi x7 cộng với offset là 12
Sh x5, 12(x7)	001	0100011	Lưu 2 byte lấy giá trị từ thanh ghi x5 lưu vào địa chỉ vật lý được tính bằng giá trị trong thanh ghi x7 cộng với offset là 12
Sw x5, 12(x7)	010	0100011	Lưu 4 byte lấy giá trị từ thanh ghi x5 lưu vào địa chỉ vật lý được tính bằng giá trị trong thanh ghi x7 cộng với offset là 12

- Hình dưới đây thể hiện thanh ghi lệnh R- type



Hình 2.6. Thanh ghi lệnh R-type trong kiến trúc RISC-V [4]

Funct7	Funct3	Opcode	Lệnh	Lệnh assembly	Mô tả lệnh
0000000	000	0110011	Add	Add x3, x1, x2	Cộng giá trị của thanh ghi x1 và x2, lưu vào x3. ($x3 = x1 + x2$)
0100000	000	0110011	Sub	Sub x3, x1, x2	giá trị x1 trừ x2, lưu kết quả vào x3. ($x3 = x1 - x2$)
0000000	001	0110011	Sll	Sll x3, x1, x2	Dịch trái giá trị của x1 theo số bit trong x2. ($x3 = x1 \ll (x2 \& 0x1F)$)
0000000	010	0110011	Slt	Slt x3, x1, x2	x1 và x2 là kiểu signed $x3 = (x1 < x2) ? 1 : 0$
0000000	011	0110011	Sltu	Sltu x3, x1, x2	x1 và x2 là kiểu unsigned $x3 = (x1 < x2) ? 1 : 0$
0000000	100	0110011	Xor	Xor x3, x1, x2	$x3 = x1 \oplus x2$
0000000	101	0110011	Srl	Srl x3, x1, x2	$x3 = x1 \gg (x2)$
0100000	101	0110011	Sra	Sra x3, x1, x2	$x3 = x1 \gg (x2)$ giữ bit dấu
0000000	110	0110011	Or	Or x3, x1, x2	$x3 = x1 x2$
0000000	111	0110011	And	And x3, x1, x2	$x3 = x1 \& x2$

Ta có các lệnh như Sra và sub có bit funct7 thứ 6 là mức 1 nên sẽ phân biệt giữa 2 phép toán có dấu trong cùng 1 nhóm lệnh.

- Hình dưới đây thể hiện thanh ghi lệnh I-type

imm[11:0]	rs1	funct3	rd	opcode	I-type
-----------	-----	--------	----	--------	--------

Hình 2.7. Thanh ghi lệnh I-type trong kiến trúc RISC-V [4]

Funct3	Opcode	Lệnh	Lệnh assembly	Mô tả lệnh
000	0010011	Addi	addi x3, x1, 10	$x3 = x1 + 10$
010	0010011	Slti	slti x3, x1, 5	r1, r2 là kiểu signed $x3 = (x1 < 5) ? 1 : 0$

011	0010011	Sltiu	sltiu x3, x1, 5	r1, r2 là kiểu unsigned $x3 = (x1 < 5) ? 1 : 0$
100	0010011	Xori	xori x3, x1, 15	$x3 = x1 \oplus (0xA)$
110	0010011	Ori	ori x3, x1, 0xFF	$x3 = x1 \mid (0xFF)$
111	0010011	Andi	andi x3, x1, 255	$x3 = x1 \& (0xA)$

Lệnh I không có funct7 và rs2 thay vào đó 12bit cao của lệnh sẽ là giá trị tức thời. Giá trị này sẽ được mở rộng thành 32bit và sử dụng trực tiếp trong các phép toán thay vì lấy giá trị từ một thanh ghi khác. Điều này làm cho việc tính toán trở nên nhanh hơn.

Với các lệnh dịch trong lệnh I thì 5bit cuối của imm sẽ được dùng để làm lượng dịch chuyển cho phép dịch. Khi thực hiện dịch 5bit này sẽ được mở rộng lên thành 32bit để thực hiện phép tính.

Cuối cùng là lệnh loại J hay lệnh nhảy hoặc lệnh nhánh không điều kiện. Lệnh J bao gồm lệnh JALR và JAL. Lệnh JAL sẽ nhảy tới địa chỉ đích được tính toán bằng cách cộng giá trị pc hiện tại với giá trị imm, địa chỉ lệnh sau lệnh nhánh trong tập lệnh (PC+4) sẽ được lưu trữ vào thanh ghi rd. Lệnh JALR tương tự như lệnh JAL nhưng cách tính toán địa chỉ đích khác hoàn toàn với JAL. Địa chỉ đích của lệnh JALR sẽ được tính bằng cách cộng giá trị trong thanh ghi rs1 với giá trị imm và địa chỉ của lệnh ngay sau lệnh nhánh trong tập lệnh (PC+4) sẽ được lưu vào thanh ghi đích rd.

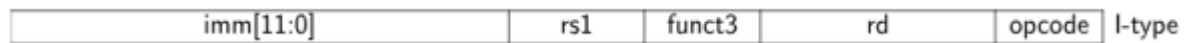
- Hình dưới đây thể hiện thanh ghi lệnh J-type



Hình 2.8. Thanh ghi lệnh J-type trong kiến trúc RISC-V [4]

Opcode	Lệnh	Mô tả
1101111	Jal x5, 20	Nhảy tới địa chỉ PC+20 và lưu giá trị tại địa chỉ kế tiếp vào x5

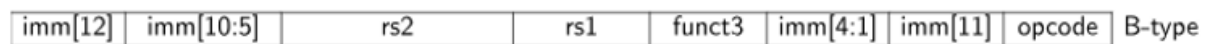
ngoài ra lệnh Jalr được cấu thành từ thanh ghi lệnh I-type



Hình 2.9. Thanh ghi lệnh I-type trong kiến trúc RISC-V [4]

Funct3	Opcode	Lệnh	Mô tả
000	1100111	Jalr x7,x8,20	Nhảy tới địa chỉ x8+20 và lưu giá trị tại địa chỉ kế tiếp vào x7

- Hình dưới đây thể hiện thanh ghi lệnh B-type

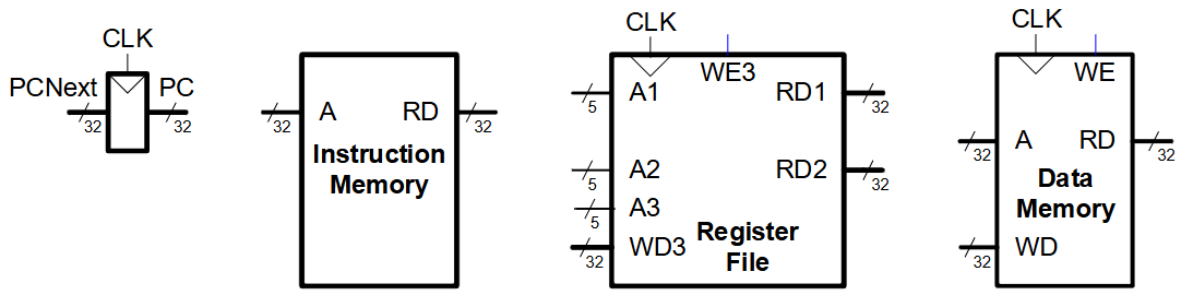


Hình 2.10. Thanh ghi lệnh B-type trong kiến trúc RISC-V [4]

Funct3	Opcode	Lệnh	Mô tả
000	1100011	beq x1, x2, label	Nếu $x1 == x2$ thì nhảy đến label
001	1100011	bne x1, x2, label	Nếu $x1 \neq x2$ thì nhảy đến label
100	1100011	blt x1, x2, label	r1 ,r2 là kiểu signed Nếu $x1 < x2$, nhảy đến label.
101	1100011	bge x1, x2, label	r1 ,r2 là kiểu signed Nếu $x1 \geq x2$, nhảy đến label
110	1100011	bltu x1, x2, label	r1, r2 là kiểu Nếu $x1 < x2$, nhảy đến label.
111	1100011	bgeu x1, x2, label	Nếu $x1 \geq x2$ (không dấu), nhảy đến label.

Định dạng lệnh B hay lệnh nhánh có điều kiện. Nếu điều kiện đúng PC sẽ được cập nhật đến địa chỉ nhánh được tính bằng PC hiện tại cộng với giá trị immediate đã được mở rộng dấu.

2.6 Các thành phần chính trong RISC_V



Hình 2.11. Các thành phần chính trong RISC-V [11]

Phần này trình bày bốn thành phần cơ bản thường xuất hiện trong một bộ xử lý RISC-V, bao gồm bộ đếm chương trình, bộ nhớ lệnh, bộ thanh ghi và bộ nhớ dữ liệu. Đây là những khối nền tảng tạo nên đường đi dữ liệu của CPU, giúp bộ xử lý thực hiện tuần tự các giai đoạn lấy lệnh, giải mã lệnh, thực thi và ghi kết quả.

2.6.1 Bộ đếm chương trình (Program Counter)

- Là thanh ghi đặc biệt dùng để lưu địa chỉ của lệnh hiện tại đang được thực thi.
- Sau mỗi chu kỳ, PC thường được tăng lên 4 để trỏ đến lệnh kế tiếp (vì mỗi lệnh RISC-V dài 32 bit = 4 byte).
- Trong trường hợp nhảy (branch, jump), PC sẽ được cập nhật theo địa chỉ mới.

2.6.2 Bộ nhớ lệnh (Instruction Memory)

- Lưu trữ toàn bộ chương trình dưới dạng mã máy.
- CPU sẽ truy xuất bộ nhớ lệnh bằng địa chỉ từ PC để lấy lệnh cần thực thi.
- Đây là bộ nhớ chỉ đọc trong quá trình thực thi.

2.6.3 Bộ thanh ghi (Register File)

- Gồm 32 thanh ghi tổng quát (x0 đến x31), mỗi thanh ghi 32 bit.
- Là nơi lưu trữ tạm thời các dữ liệu và kết quả tính toán.
- Thanh ghi x0 luôn có giá trị 0, không thể ghi vào.

2.6.4 Bộ nhớ dữ liệu (Data Memory)

- Dùng để lưu trữ dữ liệu trong quá trình thực thi chương trình.

- Các lệnh như lw (load word) và sw (store word) sẽ truy cập vào đây.
- Có thể đọc và ghi dữ liệu trong runtime.
- Kết nối với các ngoại vi như VGA , I/O

2.7 Cách thức hoạt động của CPU

Bộ xử lý trung tâm (CPU) thực hiện chương trình thông qua chu trình xử lý lệnh. Mỗi lệnh được xử lý tuần tự qua các giai đoạn cơ bản bao gồm: lấy lệnh, giải mã lệnh, thực thi lệnh, truy cập bộ nhớ và ghi kết quả. Các giai đoạn này được đồng bộ bởi tín hiệu xung nhịp (Clock) và diễn ra liên tục trong suốt quá trình hoạt động của hệ thống:

- Bước 1: Lấy lệnh (Instruction Fetch - IF)

Trong giai đoạn này, CPU sử dụng giá trị hiện tại của bộ đếm chương trình (Program Counter - PC) để truy cập bộ nhớ lệnh và đọc lệnh tương ứng. Lệnh được nạp vào thanh ghi lệnh hoặc các thanh ghi trung gian bên trong bộ xử lý để chuẩn bị cho quá trình xử lý tiếp theo. Đồng thời, giá trị của PC được cập nhật nhằm trỏ đến địa chỉ của lệnh kế tiếp. Đối với các lệnh điều khiển luồng chương trình như nhảy (Jump) hoặc rẽ nhánh (Branch), giá trị PC có thể được thay đổi để chuyển hướng thực thi đến một địa chỉ mới.

- Bước 2: Giải mã lệnh (Instruction Decode - ID)

Lệnh được lưu trữ dưới dạng mã máy nhị phân, do đó cần được phân tích để xác định chức năng tương ứng. Trong giai đoạn này, bộ giải mã lệnh (Instruction Decoder) sẽ phân tách các trường thông tin của lệnh như mã thao tác (Opcode), thanh ghi nguồn, thanh ghi đích và giá trị tức thời (Immediate). Dựa trên kết quả giải mã, bộ điều khiển (Control Unit) tạo ra các tín hiệu điều khiển nhằm cấu hình hoạt động của các khối chức năng bên trong CPU.

- Bước 3: Thực thi lệnh (Execute - EX)

Ở giai đoạn thực thi, các đơn vị chức năng của CPU được kích hoạt theo các tín hiệu điều khiển đã được tạo ra trong giai đoạn giải mã. Đối với các lệnh số học và logic, khối ALU (Arithmetic Logic Unit) thực hiện các phép toán tương ứng. Đối với các lệnh điều khiển chương trình, địa chỉ đích và điều kiện rẽ nhánh được tính toán tại giai đoạn này.

- Bước 4: Truy cập bộ nhớ (Memory Access - MEM)

Đối với các lệnh yêu cầu trao đổi dữ liệu với bộ nhớ, CPU thực hiện thao tác đọc hoặc ghi dữ liệu trong giai đoạn này. Cụ thể, các lệnh tải dữ liệu (Load) sẽ đọc dữ liệu từ bộ nhớ vào bộ xử lý, trong khi các lệnh lưu dữ liệu (Store) sẽ ghi dữ liệu từ thanh ghi xuống bộ nhớ tại địa chỉ đã được xác định trước đó.

- Bước 5: Ghi kết quả (Write Back - WB)

Kết quả tính toán từ ALU hoặc dữ liệu đọc từ bộ nhớ sẽ được ghi trở lại thanh ghi đích trong tập thanh ghi của CPU. Sau khi hoàn thành giai đoạn này, việc thực thi lệnh được xem là kết thúc và bộ xử lý tiếp tục chuyển sang xử lý lệnh kế tiếp.

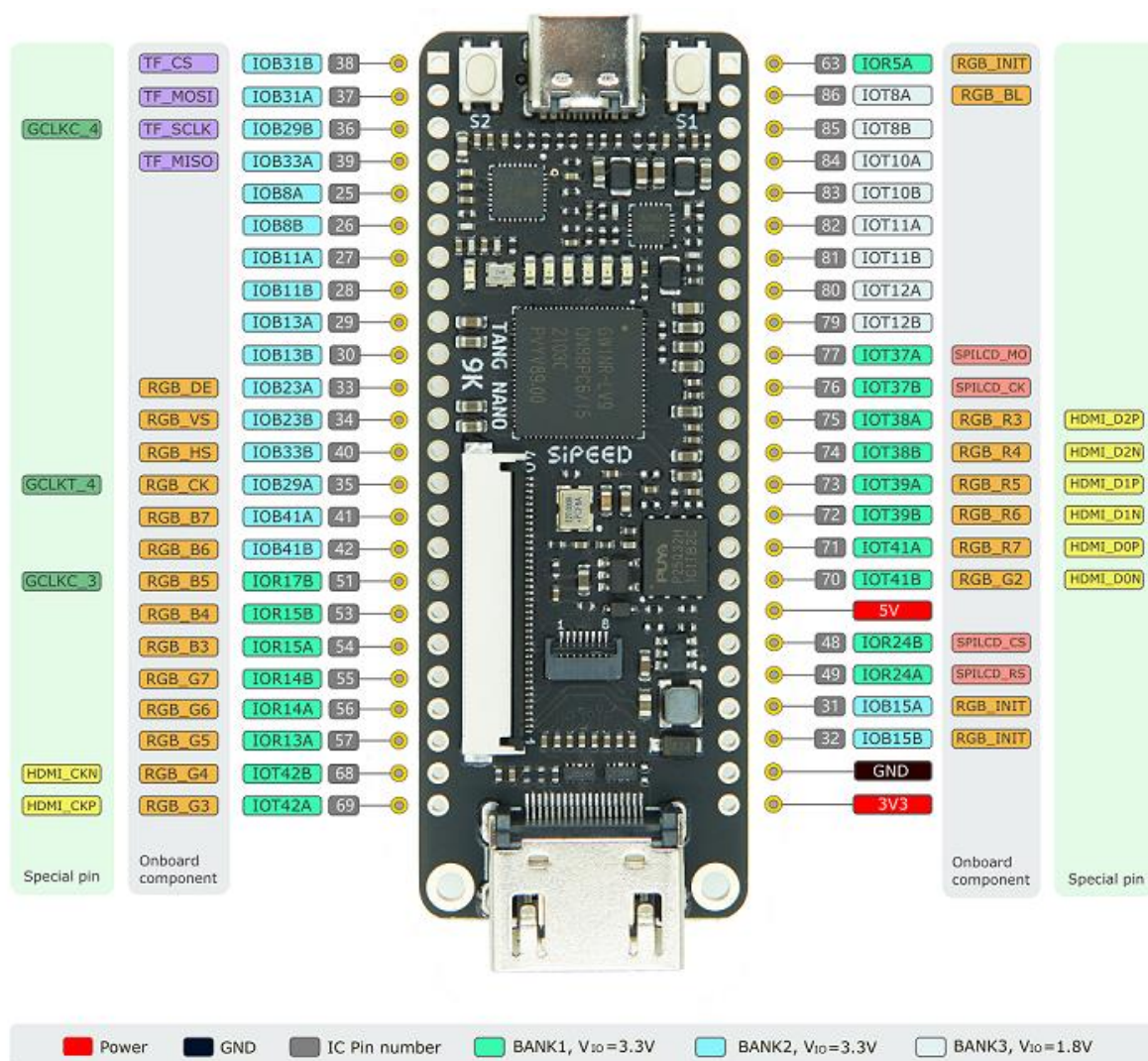
Chu trình xử lý lệnh được lặp lại liên tục trong suốt thời gian hoạt động của hệ thống. Hiệu năng của CPU phụ thuộc vào nhiều yếu tố như tần số xung nhịp, tổ chức kiến trúc phần cứng và các kỹ thuật tối ưu hóa được áp dụng. Trong các bộ xử lý hiện đại, những kỹ thuật như đường ống lệnh (Pipelining), thực thi song song và xử lý đa lõi (Multi-core Processing) được sử dụng nhằm nâng cao thông lượng và hiệu suất xử lý của hệ thống.

2.8. Linh kiện sử dụng

Để hiện thực hóa hệ thống giám sát và điều khiển môi trường cabin ô tô dựa trên kiến trúc RISC-V, đề tài sử dụng kết hợp nhiều linh kiện phần cứng từ vi mạch cốt lõi đến các cơ cấu chấp hành và linh kiện phụ trợ.

2.8.1. Khối xử lý trung tâm

Kit phát triển FPGA Tang Nano 9K: Đóng vai trò là trung tâm điều khiển của toàn bộ hệ thống. Kit FPGA này được sử dụng để tổng hợp và thực thi vi xử lý kiến trúc RISC-V lõi mềm (Soft-core SoC) do nhóm thiết kế. Tang Nano 9K là kit FPGA của hãng Sipeed sử dụng chip Gowin GW1NR-LV9QN88PC6/I5.



Hình 2.12. Kit FPGA Tang Nano 9K

Bảng 2.4. Thông số kỹ thuật Kit FPGA Tang Nano 9K

Thông số	Giá trị
Model	Sipeed Tang Nano 9K
FPGA	Gowin GW1NR-9
Số lượng LUT4	8.640
Số lượng Flip-Flop	6.480
Bộ nhớ BSRAM	468 Kbit
Bộ nhớ SPI Flash	32 Mbit
Số lượng PLL	2
Nguồn cấp	5V DC (USB Type-C)
Giao tiếp mở rộng	Header 2×24 chân GPIO

Thông số	Giá trị
Debug	USB-JTAG, USB-UART

2.8.2. Khối cảm biến và giao tiếp

- Cảm biến nhiệt độ, độ ẩm AHT20:

Sử dụng chuẩn giao tiếp I2C để thu thập dữ liệu toàn diện (môi trường ngoài, trong cabin, và kính lái), phục vụ bài toán tính toán điểm sương chống đọng kính.



Hình 2.13. Cảm biến nhiệt độ và độ ẩm AHT20

Bảng 2.5. Thông số kỹ thuật cảm biến AHT20

Thông số	Giá trị
Điện áp hoạt động	2.2V – 5.5V
Giao tiếp	I2C
Địa chỉ I2C	0x38
Dải đo nhiệt độ	-40°C đến +85°C
Sai số nhiệt độ	±0.3°C
Dải đo độ ẩm	0%RH – 100%RH
Sai số độ ẩm	±2%RH

- Màn hình OLED SSD1309:

Hoạt động như một giao diện trực quan với không gian hiển thị rộng, giúp hiển thị rõ nét các thông số nhiệt độ, độ ẩm và trạng thái hoạt động của hệ thống theo thời gian thực.

So với màn hình LCD truyền thống, OLED có ưu điểm độ tương phản cao, góc nhìn rộng, tốc độ đáp ứng nhanh và không cần đèn nền. Điều này giúp tiết kiệm năng

lượng đồng thời đảm bảo khả năng quan sát rõ ràng trong nhiều điều kiện ánh sáng khác nhau.



Hình 2.14. Màn hình OLED SSD1309

Bảng 2.6. Thông số kỹ thuật màn hình OLED SSD1309

Thông số	Giá trị
Driver IC	SSD1309
Kích thước màn hình	2.42 inch
Độ phân giải	128 × 64 pixel
Màu hiển thị	Trắng
Góc nhìn	> 160°
Điện áp hoạt động	3.3V – 5V DC
Giao thức giao tiếp	I2C
Vi điều khiển hỗ trợ	Arduino, 8051, MSP430, STM32, CSR và các nền tảng tương thích khác

2.8.3. Khối cơ cấu chấp hành và điều khiển công suất

- Quạt: Tạo luồng gió mô phỏng quá trình sấy kính.



Hình 2.15. Quạt DC 5V

Bảng 2.7. Thông số kỹ thuật quạt DC

Thuộc tính	Giá trị
Điện áp cấp	5V
Kích thước	50 x 50 x 12mm
Công suất	1W
Dòng hoạt động tối đa	150mA
Tốc độ quay	4762rpm
Đầu nối dây	XH2.54-2P

- Động cơ Servo: Được điều khiển bằng xung PWM. Servo 1 làm nhiệm vụ đóng/mở cửa lấy gió ngoài, Servo 2 làm nhiệm vụ điều khiển cửa gió sấy kính.



Hình 2.16. Động cơ Servo SG90

Bảng 2.8. Thông số kỹ thuật động cơ Servo SG90

Thuộc tính	Giá trị
Loại	Động cơ
Góc quay	180 độ
Mô men xoắn tối đa	2.4 Kg.cm
Điện áp làm việc	4.8 → 6V

- Module MOSFET D4184: Module MOSFET D4184 được sử dụng làm mạch điều khiển công suất cho quạt DC trong hệ thống. Do chân GPIO của FPGA chỉ cung cấp được dòng điện nhỏ nên không thể điều khiển trực tiếp tải công suất lớn. Module MOSFET nhận tín hiệu PWM từ FPGA và thực hiện đóng cắt nguồn cấp cho quạt với tần số cao, từ đó điều chỉnh tốc độ quay của quạt theo yêu cầu. Việc sử dụng MOSFET giúp nâng cao khả năng chịu dòng của mạch, bảo vệ FPGA và đảm bảo hệ thống hoạt động ổn định.



Hình 2.17. Module MOSFET D4184

Bảng 2.9. Thông số kỹ thuật module MOSFET D4184

Thông số	Giá trị
Loại module	Module điều khiển công suất MOSFET D4184
Điện áp cung cấp	6 – 28 VDC
Điện áp chịu tải tối đa	40 V
Dòng tải tối đa	50 A
Tín hiệu điều khiển	PWM hoặc mức logic
Kích thước	3.3 × 1.8 cm
Khối lượng	20 g
Chức năng	Điều khiển tải DC công suất lớn như quạt, động cơ DC, đèn LED công suất cao

2.8.1. Khối nguồn

- Nguồn 5V số 1: Cấp nguồn cho khối vi mạch FPGA Tang Nano 9K.
- Nguồn 5V số 2: Cung cấp dòng xả lớn để nuôi trực tiếp khối cơ cấu chấp hành tải nặng (Quạt và 2 Servo).

2.8.2. Linh kiện phụ trợ

- Tụ hóa 2200uF: ngăn chặn hiện tượng sụt áp khi động cơ và servo khởi động, giữ nguồn ổn định và bảo vệ mạch.
- Tụ gốm và điện trở: có tác dụng triệt tiêu nhiễu cao tần, phối hợp với điện trở tạo thành mạch RC giúp chống dội cho nút nhấn.

Bảng 2.10. Danh mục linh kiện

STT	Tên linh kiện	Vai trò	Số lượng
1	Kit FPGA Tang Nano 9K	Chạy CPU RISC-V, bộ xử lý trung tâm	1
2	Cảm biến AHT20	Giao tiếp I2C, đo nhiệt độ - độ ẩm	3
3	Màn hình OLED SSD1309	Hiển thị dữ liệu	1
4	Động cơ Servo	Điều khiển cửa lấy gió, cửa gió sấy kính	2
5	Quạt	Tạo luồng gió mô phỏng quá trình sấy kính	1
6	Module MOSFET D4184	Điều tốc PWM cho quạt	1
7	Nguồn Pin 5V	1 nguồn cấp cho FPGA, 1 nguồn tải động cơ	2
8	Nút nhấn 12x12mm	Input người dùng	3
9	Tụ điện hóa 2200uF	Lọc nguồn nguồn 2, chống sụt áp động cơ	1
10	Tụ gốm	Lọc nhiễu và Mạch RC chống dội nút nhấn	4
11	Điện trở	Mạch RC chống dội nút nhấn	5
12	Diode	Chống ngược dòng bảo vệ FPGA	1

CHƯƠNG 3 THIẾT KẾ PHẦN CỨNG CỦA HỆ THỐNG

3.1. Yêu cầu hệ thống

3.1.1. Mục tiêu thiết kế

Mục tiêu của chương này là trình bày quá trình thiết kế phần cứng cho hệ thống xử lý dựa trên kiến trúc RISC-V 32 bit. Hệ thống được xây dựng nhằm thực hiện các chương trình điều khiển cơ bản, đồng thời có khả năng giao tiếp với các ngoại vi thông qua cơ chế ánh xạ bộ nhớ.

Bộ xử lý trung tâm được thiết kế theo tập lệnh cơ sở RV32I của kiến trúc RISC-V. Đây là tập lệnh 32 bit đơn giản, phù hợp cho việc nghiên cứu, mô phỏng và triển khai trên FPGA. Việc lựa chọn RV32I giúp hệ thống có cấu trúc rõ ràng, dễ kiểm tra từng khối chức năng và thuận lợi khi mở rộng thêm ngoại vi.

Hệ thống SoC được triển khai trên nền tảng FPGA Tang Nano 9K. Trên FPGA, các thành phần chính như CPU, bộ nhớ và ngoại vi được mô tả bằng ngôn ngữ phần cứng, sau đó tổng hợp thành mạch số thực tế. Các ngoại vi được kết nối với CPU thông qua bus hệ thống và được truy cập bằng địa chỉ giống như truy cập bộ nhớ.

3.1.2. Yêu cầu chức năng

Hệ thống cần đáp ứng các yêu cầu chức năng cơ bản sau:

- Hỗ trợ thực thi các lệnh thuộc tập lệnh RV32I.
- Có khả năng đọc lệnh từ bộ nhớ chương trình.
- Có khả năng đọc và ghi dữ liệu vào bộ nhớ dữ liệu.
- Thực hiện được các phép toán số học, logic, dịch bit và so sánh.
- Hỗ trợ các lệnh điều khiển luồng chương trình như rẽ nhánh và nhảy.
- Cho phép CPU giao tiếp với các ngoại vi thông qua cơ chế Memory-Mapped I/O.
- Có thể mở rộng thêm các ngoại vi như LED, I2C, PWM và Timer tùy theo thiết kế thực tế.

3.1.3. Yêu cầu phần cứng

Phần cứng của hệ thống được thiết kế dựa trên các tài nguyên có sẵn của FPGA Tang Nano 9K. Xung nhịp hệ thống sử dụng nguồn clock 27 MHz trên board. Bộ nhớ chương trình và bộ nhớ dữ liệu được triển khai bằng tài nguyên bộ nhớ nội FPGA.

Các ngoại vi được tích hợp tùy theo yêu cầu của hệ thống. Trong thiết kế này, các ngoại vi được kết nối thông qua bus hệ thống, CPU có thể điều khiển bằng cách ghi dữ liệu vào các địa chỉ tương ứng.

3.1.4. Tiêu chí tổ chức thiết kế phần cứng

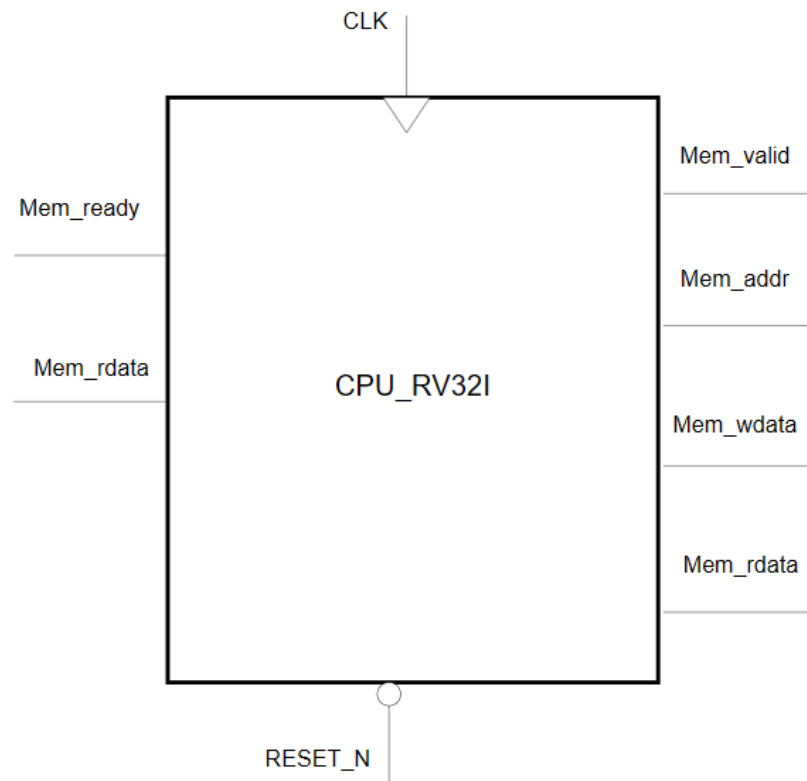
Khi triển khai trên FPGA có tài nguyên hạn chế, thiết kế được ưu tiên theo ba tiêu chí: đủ chức năng, dễ kiểm tra và dễ mở rộng. Đủ chức năng nghĩa là CPU phải thực thi được chương trình C tối giản, đọc ghi bộ nhớ và điều khiển các ngoại vi chính của đề tài. Dễ kiểm tra nghĩa là mỗi khối như UART, I2C, PWM, GPIO và timer có giao diện thanh ghi rõ ràng, có thể thử riêng trước khi ghép vào toàn hệ thống. Dễ mở rộng nghĩa là khi cần thêm ngoại vi mới, người thiết kế chỉ cần bổ sung module slave, cấp vùng địa chỉ và viết driver tương ứng trong firmware.

Thiết kế không chọn pipeline phức tạp vì trọng tâm của đề tài là xây dựng một SoC hoạt động ổn định trên FPGA nhỏ, không phải tối ưu tốc độ xử lý lệnh. Kiến trúc nhiều chu kỳ giúp giảm độ phức tạp điều khiển hazard, forwarding và flush. Đổi lại, CPI cao hơn pipeline, nhưng vẫn phù hợp với các tác vụ đọc cảm biến, cập nhật OLED và điều khiển PWM vì các tác vụ này không yêu cầu thông lượng tính toán lớn.

3.2. Thiết kế CPU RV32I

3.2.1. Kiến trúc tổng thể CPU RV32I

Trong hệ thống SoC được thiết kế, CPU RV32I đóng vai trò là khối xử lý trung tâm, thực hiện nhiệm vụ đọc lệnh, giải mã lệnh, xử lý dữ liệu và điều khiển quá trình truy cập bộ nhớ cũng như các ngoại vi. CPU được xây dựng theo kiến trúc xử lý đa chu kỳ, trong đó các giai đoạn thực thi lệnh được điều khiển tuần tự nhằm giảm độ phức tạp phần cứng và phù hợp với tài nguyên của FPGA. Hình 3.1 trình bày sơ đồ khối tổng thể của CPU RV32I, bao gồm các tín hiệu xung nhịp, reset và giao tiếp bộ nhớ theo cơ chế bắt tay giữa CPU và hệ thống bus.



Hình 3.1. Hình khối tổng thể CPU RV32I.

Hình 3.1 trình bày sơ đồ khối tổng thể của CPU RV32I được sử dụng trong hệ thống. CPU, được xây dựng theo kiến trúc xử lý đa chu kỳ.

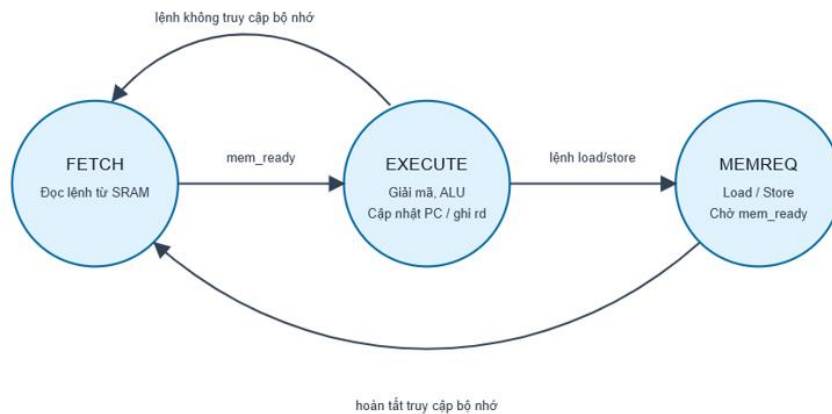
CPU gồm các thành phần chính như Program Counter, thanh ghi lệnh, khối giải mã lệnh, Register File, khối tạo giá trị tức thời, ALU, bộ điều khiển và khối giao tiếp bộ nhớ. Các khối này phối hợp với nhau để thực hiện quá trình lấy lệnh, giải mã lệnh, thực thi lệnh, truy cập bộ nhớ và ghi kết quả về thanh ghi.

CPU sử dụng giao tiếp bộ nhớ đơn giản gồm các tín hiệu `mem_valid`, `mem_ready`, `mem_addr`, `mem_wdata`, `mem_rdata` và `mem_wstrb`. Nhờ giao tiếp này, CPU có thể truy cập cả SRAM và các ngoại vi thông qua cùng một không gian địa chỉ.

Khi chương trình thực hiện một lệnh load hoặc store, địa chỉ do ALU tạo ra được đưa ra `mem_addr`. Khối giải mã địa chỉ ở top SoC sẽ quyết định giao dịch đó thuộc về vùng nhớ hay thuộc về một thanh ghi ngoại vi. Cách làm này giúp datapath của CPU giữ được cấu trúc gọn, còn phần mở rộng ngoại vi được xử lý ở mức SoC.

Với kiến trúc RV32I, mỗi lệnh có độ dài 32 bit và các trường lệnh như opcode, rd, rs1, rs2, funct3, funct7 được bố trí cố định. Đặc điểm này giúp khối decoder trong CPU dễ tách trường lệnh và tạo tín hiệu điều khiển. Trong đề tài, tập lệnh được tập trung vào nhóm số học - logic, load/store, branch và jump, đủ để chương trình C thực hiện vòng lặp điều khiển hệ thống.

3.2.2. Nguyên lý hoạt động của CPU



Hình 3.3. Lưu đồ FSM hoạt động của CPU đa chu kỳ.

CPU hoạt động theo máy trạng thái hữu hạn gồm ba trạng thái chính: FETCH, EXECUTE và MEMREQ.

Ở trạng thái FETCH, CPU phát địa chỉ Program Counter ra bus để đọc lệnh từ bộ nhớ. Khi bộ nhớ phản hồi bằng tín hiệu `mem_ready`, lệnh đọc được lưu vào thanh ghi lệnh để chuẩn bị giải mã.

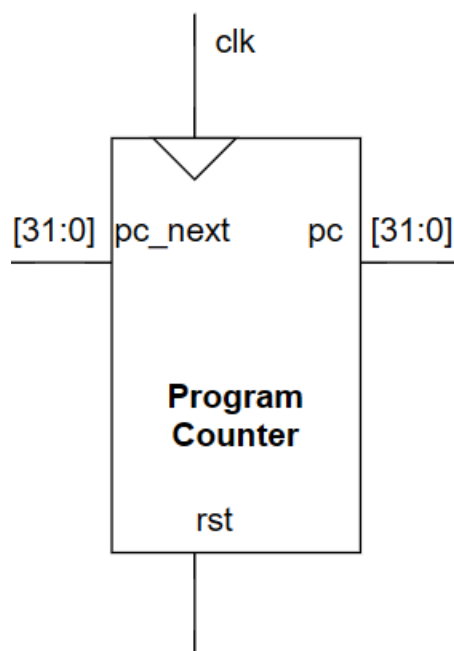
Ở trạng thái EXECUTE, CPU giải mã lệnh, đọc dữ liệu từ Register File, tạo giá trị immediate và điều khiển ALU thực hiện phép toán tương ứng. Với các lệnh không cần truy cập bộ nhớ, kết quả có thể được ghi về thanh ghi ngay sau giai đoạn này. Với các lệnh rẽ nhánh hoặc nhảy, CPU đồng thời tính toán giá trị Program Counter kế tiếp.

Ở trạng thái MEMREQ, CPU thực hiện các lệnh truy cập bộ nhớ như load và store. CPU phát địa chỉ, dữ liệu ghi và tín hiệu byte enable ra bus. Khi ngoại vi hoặc bộ nhớ trả về mem_ready, CPU hoàn tất giao dịch và quay lại lấy lệnh tiếp theo.

So với cách mô tả lý thuyết gồm các giai đoạn nạp lệnh, giải mã, thực thi, truy cập bộ nhớ và ghi kết quả, thiết kế thực tế gom các thao tác này vào ba trạng thái lớn để phù hợp với bus mem_valid/mem_ready. Trạng thái FETCH phụ trách lấy lệnh, EXECUTE phụ trách giải mã và xử lý phần lớn lệnh, còn MEMREQ chỉ được dùng khi lệnh cần truy cập SRAM hoặc ngoại vi. Nhờ vậy FSM không quá dài nhưng vẫn thể hiện được đầy đủ vòng đời của một lệnh trong CPU nhiều chu kỳ.

Khi mem_ready chưa được kích hoạt, CPU giữ nguyên yêu cầu trên bus thay vì tự tăng PC hoặc ghi kết quả sớm. Cơ chế chờ này rất quan trọng vì các ngoại vi không nhất thiết phản hồi trong cùng một chu kỳ. Ví dụ SRAM có thể trả về nhanh hơn, trong khi timer hoặc UART có thể cần thêm logic trạng thái trước khi báo hoàn tất giao dịch.

3.2.3. Khối Program Counter



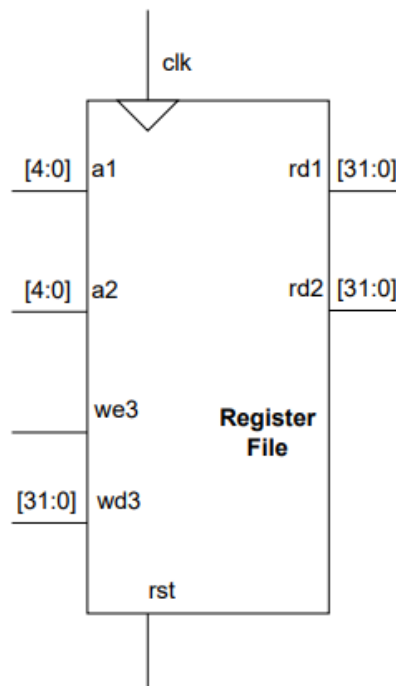
Hình 3.4. Sơ đồ khối Program Counter

Program Counter là thanh ghi lưu địa chỉ của lệnh đang được thực thi hoặc lệnh kế tiếp. Sau mỗi lệnh thông thường, PC được tăng thêm 4 do mỗi lệnh RV32I có độ dài 32 bit, tương ứng 4 byte.

Đối với các lệnh điều khiển luồng chương trình như branch, JAL hoặc JALR, giá trị PC không tăng tuần tự mà được thay đổi theo địa chỉ đích. Địa chỉ đích này được tính từ PC hiện tại, giá trị immediate hoặc giá trị trong thanh ghi nguồn tùy theo loại lệnh.

Trong thiết kế, PC được cập nhật dựa trên tín hiệu điều khiển sinh ra từ khối giải mã lệnh và kết quả so sánh từ ALU. Nhờ đó CPU có thể thực hiện đúng các lệnh nhảy và rẽ nhánh của tập lệnh RV32I.

3.2.4. Khối Register File



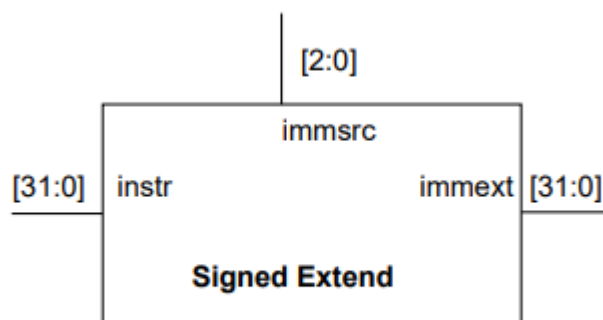
Hình 3.5. Sơ đồ tổ chức khối Register File trong CPU RV32I.

Register File là khối lưu trữ 32 thanh ghi tổng quát của kiến trúc RV32I. Mỗi thanh ghi có độ rộng 32 bit. CPU sử dụng hai cổng đọc để lấy dữ liệu từ hai thanh ghi nguồn rs1 và rs2, đồng thời sử dụng một cổng ghi để ghi kết quả về thanh ghi đích rd.

Dữ liệu ghi về Register File có thể đến từ nhiều nguồn khác nhau, bao gồm kết quả ALU, dữ liệu đọc từ bộ nhớ hoặc giá trị $PC + 4$ đối với lệnh nhảy có liên kết. Việc lựa chọn dữ liệu ghi về được điều khiển bởi tín hiệu `result_src`.

Theo đặc tả RISC-V, thanh ghi `x0` luôn có giá trị bằng 0. Vì vậy, nếu lệnh có ghi vào `x0`, giá trị của thanh ghi này vẫn không thay đổi. Quy tắc này giúp đơn giản hóa phần mềm và đảm bảo CPU hoạt động đúng theo kiến trúc RV32I.

3.2.5. Khối tạo giá trị tức thời

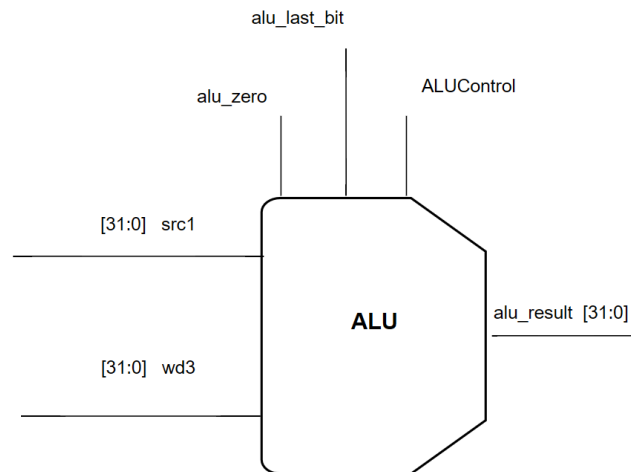


Hình 3.6. Sơ đồ khối tạo giá trị tức thời trong RV32I.

Khối tạo giá trị tức thời có nhiệm vụ tách các bit immediate từ lệnh 32 bit và mở rộng dấu thành dữ liệu 32 bit. Trong RV32I, mỗi định dạng lệnh có cách sắp xếp immediate khác nhau, vì vậy CPU cần một khối riêng để xử lý phần này.

Khối này hỗ trợ các dạng immediate của lệnh I-Type, S-Type, B-Type, U-Type và J-Type. Với lệnh tính toán immediate hoặc load, giá trị immediate được đưa vào ALU làm toán hạng thứ hai. Với lệnh store, immediate dùng để tính địa chỉ ghi bộ nhớ. Với lệnh branch và jump, immediate được dùng để tính địa chỉ PC kế tiếp.

3.2.6. Khối ALU



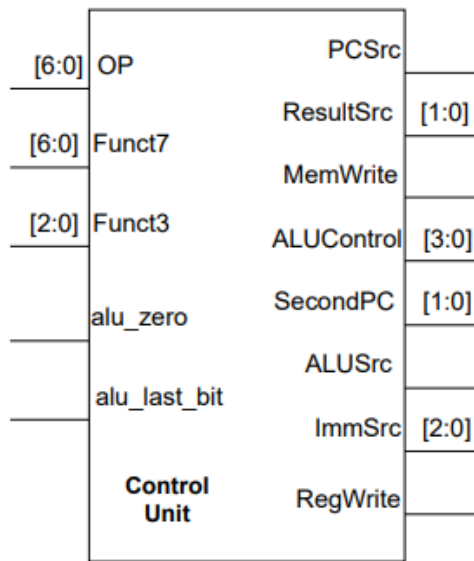
Hình 3.7. Sơ đồ khối ALU và các tín hiệu điều khiển.

ALU là khối thực hiện các phép toán số học và logic của CPU. Dữ liệu đầu vào của ALU gồm toán hạng thứ nhất từ Register File và toán hạng thứ hai được chọn giữa Register File hoặc giá trị immediate.

Các phép toán ALU hỗ trợ gồm cộng, trừ, AND, OR, XOR, dịch trái, dịch phải, so sánh có dấu và so sánh không dấu. Với lệnh load/store, ALU dùng để tính địa chỉ bộ nhớ. Với lệnh branch, ALU hỗ trợ so sánh hai toán hạng để xác định điều kiện rẽ nhánh.

Tín hiệu điều khiển ALU được tạo ra dựa trên các trường opcode, funct3 và funct7 của lệnh. Nhờ đó cùng một khối ALU có thể phục vụ nhiều nhóm lệnh khác nhau trong tập lệnh RV32I.

3.2.7. Khối giải mã lệnh và điều khiển



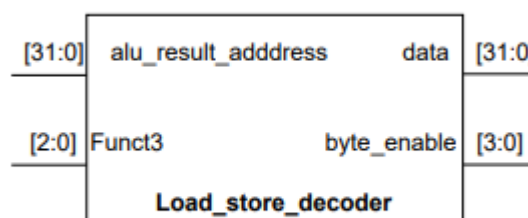
Hình 3.8. Sơ đồ khối giải mã lệnh và điều khiển.

Khối giải mã lệnh nhận lệnh 32 bit và tách các trường thông tin như opcode, rd, funct3, rs1, rs2 và funct7. Dựa trên các trường này, bộ điều khiển xác định loại lệnh và sinh ra các tín hiệu điều khiển cho toàn bộ CPU.

Các tín hiệu điều khiển chính gồm cho phép ghi thanh ghi, chọn nguồn toán hạng ALU, điều khiển đọc/ghi bộ nhớ, chọn nguồn dữ liệu ghi về Register File, chọn loại immediate và điều khiển cập nhật PC.

Trong thiết kế, khối điều khiển được tổ chức phù hợp với máy trạng thái nhiều chu kỳ. Điều này giúp CPU có thể dùng chung các khối phần cứng qua nhiều chu kỳ clock, giảm tài nguyên phần cứng so với thiết kế pipeline.

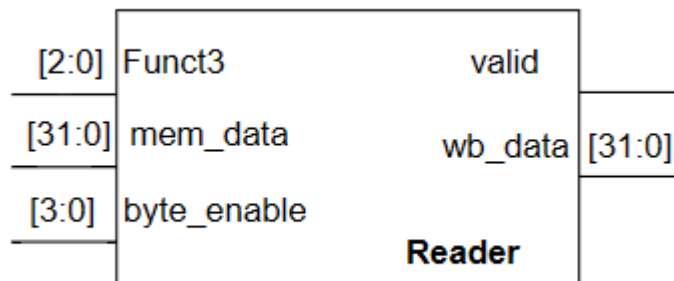
3.2.8. Khối xử lý Load/Store và Reader



Hình 3.9. Sơ đồ khối xử lý lệnh Load/Store.

Khối xử lý Load/Store có nhiệm vụ hỗ trợ CPU thực hiện các lệnh đọc và ghi bộ nhớ với nhiều kích thước dữ liệu khác nhau như byte, half-word và word. Trong thiết kế, chức năng này được chia thành hai phần chính: khối load_store_decoder và khối reader.

Khối load_store_decoder nhận địa chỉ do ALU tính toán, trường funct3 của lệnh và dữ liệu đọc từ thanh ghi nguồn rs2. Dựa trên hai bit thấp của địa chỉ, khối này xác định vị trí byte hoặc half-word cần truy cập trong word 32 bit. Khi thực hiện lệnh store, khối tạo tín hiệu byte_enable để xác định byte nào được ghi, đồng thời dịch dữ liệu ghi vào đúng vị trí trên bus mem_wdata.



Hình 3.10. Sơ đồ khối Reader xử lý dữ liệu đọc từ bus.

Khối reader được sử dụng cho các lệnh load. Sau khi dữ liệu 32 bit được đọc từ bus thông qua mem_rdata, khối reader dùng byte_enable để chọn đúng byte hoặc half-word cần lấy. Sau đó dữ liệu được dịch về các bit thấp và được mở rộng thành 32 bit. Tùy theo loại lệnh load, dữ liệu có thể được mở rộng dấu đối với LB, LH, hoặc mở rộng bằng 0 đối với LBU, LHU. Đối với lệnh LW, toàn bộ 32 bit dữ liệu được giữ nguyên.

Nhờ kết hợp load_store_decoder và reader, CPU có thể hỗ trợ đầy đủ các lệnh truy cập bộ nhớ cơ bản của RV32I như LB, LH, LW, LBU, LHU, SB, SH và SW trên cùng một bus dữ liệu 32 bit.

3.2.9. Giao tiếp bus của CPU



Hình 3.11. Cơ chế bắt tay mem_valid/mem_ready giữa CPU và bộ nhớ hoặc ngoại vi.

CPU sử dụng giao tiếp bus đơn giản theo cơ chế bắt tay giữa CPU và slave. Khi CPU cần truy cập bộ nhớ hoặc ngoại vi, CPU đưa địa chỉ lên mem_addr, dữ liệu ghi lên mem_wdata, byte enable lên mem_wstrb, đồng thời bật mem_valid.

Phía bộ nhớ hoặc ngoại vi sẽ kiểm tra địa chỉ và phản hồi bằng tín hiệu mem_ready khi giao dịch hoàn tất. Nếu là giao dịch đọc, dữ liệu được trả về CPU thông qua mem_rdata. Nếu là giao dịch ghi, dữ liệu từ CPU được ngoại vi hoặc SRAM chốt lại theo tín hiệu ghi.

Cơ chế mem_valid / mem_ready giúp CPU có thể làm việc với nhiều loại ngoại vi có thời gian phản hồi khác nhau. Trong hệ thống này, SRAM, LED, UART, Timer, I2C, PWM và GPIO đều được kết nối theo cơ chế này.

3.3. Thiết kế SoC và các ngoại vi

SoC là hệ thống được tích hợp trên cùng một chip hoặc cùng một nền tảng phần cứng. Trong một SoC, các khối chức năng như CPU, bộ nhớ, bus hệ thống, khối giải mã địa chỉ và các ngoại vi được kết nối với nhau để tạo thành một hệ thống xử lý hoàn chỉnh. Thay vì CPU hoạt động riêng lẻ, SoC cho phép CPU truy cập bộ nhớ, điều khiển ngoại vi và trao đổi dữ liệu với các khối phần cứng khác thông qua một cơ chế giao tiếp thống nhất.

Sau khi hoàn thành khối CPU RV32I, CPU được tích hợp với bộ nhớ và các ngoại vi để tạo thành một hệ thống SoC hoàn chỉnh trên FPGA. Trong hệ thống này, CPU đóng vai trò master duy nhất, còn SRAM và các ngoại vi như LED, UART, Countdown Timer, I2C, PWM và GPIO nút nhấn được ánh xạ vào không gian địa chỉ của CPU.

Trong phạm vi đề tài, SoC được hiểu là hệ thống tích hợp nhiều khối chức năng trên cùng một FPGA, bao gồm CPU, bộ nhớ, bus hệ thống, khối giải mã địa chỉ và các ngoại vi. CPU thực thi chương trình nhúng và phát sinh các yêu cầu đọc/ghi, trong khi các ngoại vi phản hồi thông qua cơ chế memory-mapped I/O. Cách tổ chức này giúp chương trình C có thể điều khiển phần cứng bằng các thao tác đọc/ghi đến các địa chỉ đã được quy định.

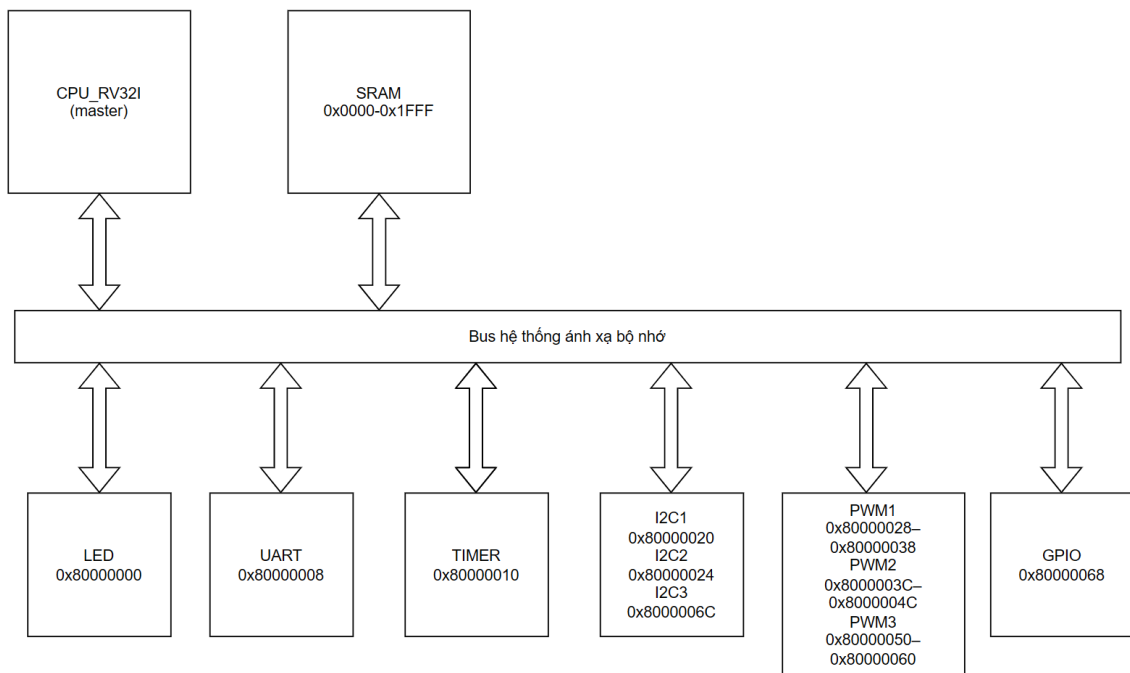
Các mục trong phần 3.3 được trình bày theo hướng phần cứng, gồm chức năng của khối, sơ đồ khối hoặc sơ đồ kết nối, thanh ghi/địa chỉ và nguyên lý hoạt động. Những

hình thể hiện một module cùng các tín hiệu vào/ra được gọi là sơ đồ khối; những hình thể hiện quan hệ giữa nhiều khối hoặc đường đi giao dịch được gọi là sơ đồ kết nối hoặc sơ đồ luồng.

3.3.1. Kiến trúc tổng thể SoC

Khối SoC tổng thể có nhiệm vụ kết nối CPU RV32I với bộ nhớ chương trình, bộ nhớ dữ liệu và các ngoại vi phục vụ cho bài toán giám sát môi trường cabin ô tô. CPU thực thi chương trình nhúng C, đọc cảm biến AHT20, cập nhật màn hình OLED, điều khiển quạt và servo bằng PWM, đồng thời gửi thông tin kiểm tra qua UART.

Hình 3.12 trình bày quan hệ giữa CPU, bus hệ thống, khối giải mã địa chỉ và các ngoại vi trong SoC. Từ sơ đồ có thể thấy CPU là master duy nhất của hệ thống, còn SRAM và các ngoại vi là các slave được chọn thông qua địa chỉ truy cập.



Hình 3.12. Sơ đồ khối tổng thể SoC RISC-V trên FPGA.

- SoC được chia thành hai nhóm địa chỉ chính. Vùng địa chỉ thấp bắt đầu từ 0x00000000 được dùng cho SRAM, còn vùng địa chỉ cao bắt đầu từ 0x80000000 được dùng cho các ngoại vi. Việc tách vùng như vậy giúp khối giải mã địa chỉ đơn giản, đồng thời giúp chương trình nhúng dễ định nghĩa các macro địa chỉ khi lập trình bằng C.

- **Nguyên lý hoạt động:** Sau khi reset, CPU bắt đầu nạp lệnh từ vùng bộ nhớ chương trình. Khi cần truy cập bộ nhớ hoặc ngoại vi, CPU đưa địa chỉ lên bus, phát tín hiệu mem_valid và chờ phản hồi mem_ready. Nếu địa chỉ thuộc SRAM, giao dịch được

chuyển đến bộ nhớ; nếu địa chỉ thuộc vùng ngoại vi, khối giải mã địa chỉ chọn module tương ứng và trả dữ liệu về CPU hoặc chốt dữ liệu ghi.

3.3.2. Bus hệ thống, ánh xạ bộ nhớ và nguyên tắc truy cập ngoại vi

Bus hệ thống là phần trung gian kết nối CPU RV32I với SRAM và các ngoại vi trong SoC. Khối này nhận các tín hiệu truy cập từ CPU như mem_valid, mem_addr, mem_wdata, mem_wstrb, sau đó xác định giao dịch thuộc về SRAM hay ngoại vi nào. Nhờ đó, nhiều ngoại vi có thể dùng chung một giao diện bus thống nhất.

Bảng 3.1. Bản đồ địa chỉ các khối nhớ và ngoại vi trong SoC.

Địa chỉ / vùng địa chỉ	Khối	Quyền truy cập	Chức năng
0x00000000– 0x00001FFF	SRAM	Đọc / ghi	Lưu chương trình, dữ liệu tạm và vùng stack.
0x80000000	LED	Đọc / ghi	Ghi trạng thái LED và đọc lại giá trị đã ghi.
0x80000008	UART_DI V	Ghi	Cấu hình bộ chia baudrate cho UART.
0x8000000C	UART_D ATA	Đọc / ghi	Ghi ký tự truyền hoặc đọc dữ liệu nhận được.
0x80000010	Countdown Timer	Đọc / ghi	Ghi giá trị đếm lùi và đọc trạng thái timer.
0x80000020	I2C1	Đọc / ghi	Điều khiển bus I2C cho OLED SSD1309 và cảm biến AHT20 S1.
0x80000024	I2C2	Đọc / ghi	Điều khiển bus I2C cho cảm biến AHT20 S2.
0x80000028– 0x80000038	PWM 1	Đọc / ghi	Nhóm thanh ghi PWM điều khiển quạt.
0x8000003C– 0x8000004C	PWM 2	Đọc / ghi	Nhóm thanh ghi PWM điều khiển servo thứ nhất.
0x80000050– 0x80000060	PWM 3	Đọc / ghi	Nhóm thanh ghi PWM điều khiển servo thứ hai.

0x80000068	GPIO Button	Đọc	Đọc trạng thái các nút nhấn điều khiển.
0x8000006C	I2C3	Đọc / ghi	Điều khiển bus I2C cho cảm biến AHT20 S3.

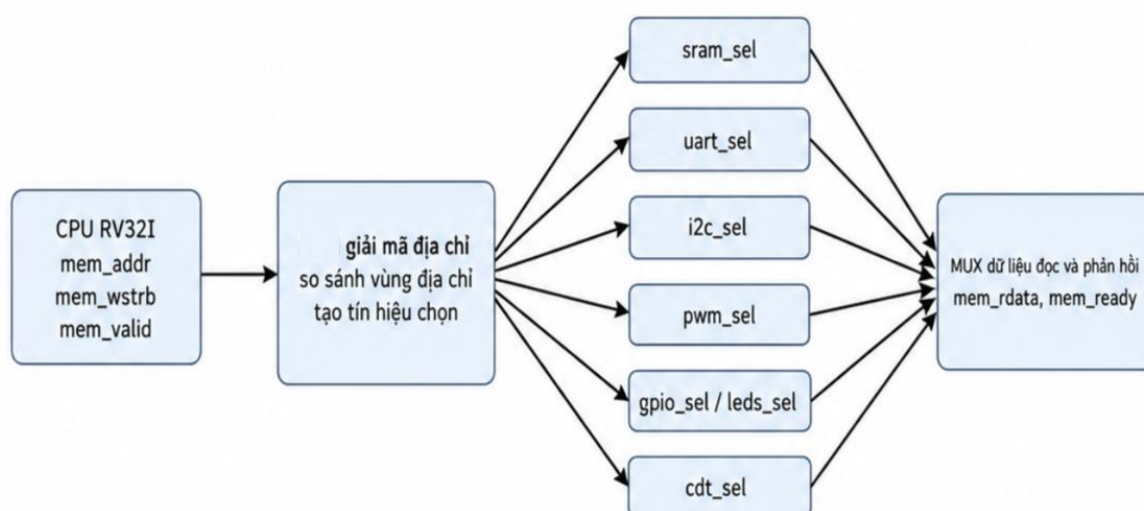
- **Thanh ghi địa chỉ:** Các địa chỉ trong bảng 3.13 là địa chỉ mà CPU nhìn thấy khi chạy chương trình nhúng. SRAM được đặt ở vùng địa chỉ thấp để chứa mã chương trình, dữ liệu và stack. Các ngoại vi như LED, UART, Timer, I2C, PWM và GPIO được đặt trong vùng địa chỉ cao bắt đầu từ 0x80000000. Chương trình nhúng chỉ cần đọc hoặc ghi đúng địa chỉ để điều khiển ngoại vi tương ứng.

- **Nguyên lý hoạt động:** Khi CPU phát giao tiếp bus, địa chỉ mem_addr được so sánh với bản đồ địa chỉ. Nếu địa chỉ nằm trong vùng SRAM, giao dịch được chuyển đến bộ nhớ. Nếu địa chỉ thuộc vùng 0x80000000 trở lên, giao dịch được chuyển đến ngoại vi tương ứng. Với thao tác ghi, dữ liệu trên mem_wdata được chốt vào thanh ghi ngoại vi; với thao tác đọc, dữ liệu từ ngoại vi được đưa về CPU thông qua mem_rdata.

3.3.3. Luồng giải mã địa chỉ

Giải mã địa chỉ giúp xác định giao dịch hiện tại của CPU thuộc về SRAM hay ngoại vi nào trong SoC và giúp nhiều module phần cứng có thể dùng chung một giao diện bus từ CPU.

- **Luồng giải mã địa chỉ:**



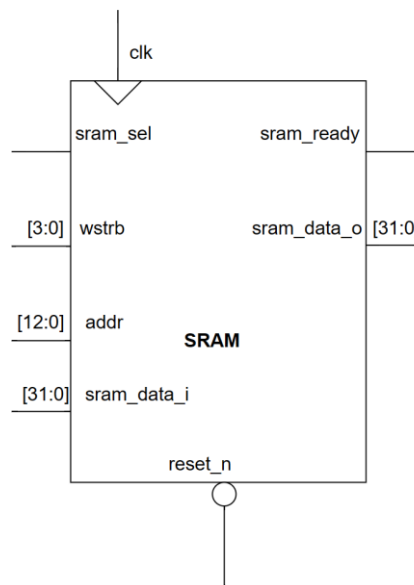
Hình 3.13. Luồng giải mã địa chỉ.

- **Nguyên lý hoạt động:** Khi mem_valid được kích hoạt, địa chỉ mem_addr được so sánh với từng vùng địa chỉ đã quy định. Chỉ module có địa chỉ khớp mới được cho phép phản hồi giao dịch. Với giao dịch đọc, dữ liệu từ module được chọn sẽ đi qua mạch chọn dữ liệu đọc để trả về mem_rdata. Với giao dịch ghi, chỉ module được chọn mới chốt dữ liệu từ mem_wdata. Việc chỉ cho một khối phản hồi tại một thời điểm giúp tránh xung đột dữ liệu trên bus.

3.3.4. Khối SRAM

SRAM/BRAM là bộ nhớ chính của SoC, dùng để lưu mã chương trình, dữ liệu toàn cục, biến tạm và vùng stack của chương trình nhúng. Trong thiết kế trên FPGA, bộ nhớ này được tổng hợp từ tài nguyên bộ nhớ nội của chip.

- **Sơ đồ khối:**



Hình 3.14. Sơ đồ khối SRAM/BRAM dùng cho chương trình và dữ liệu.

- **Thanh ghi địa chỉ:** SRAM được ánh xạ trong vùng 0x00000000 - 0x00001FFF, tương ứng dung lượng 8192 byte. Đây là vùng đọc/ghi, cho phép CPU nạp lệnh, đọc dữ liệu và ghi dữ liệu trong quá trình thực thi chương trình.

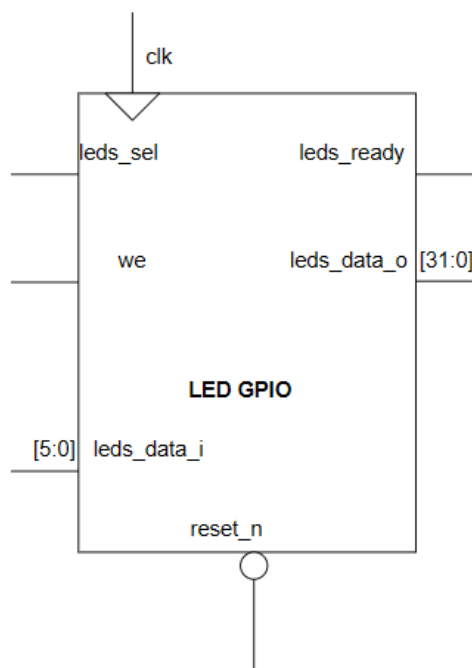
- **Nguyên lý hoạt động:** Khối SRAM nhận tín hiệu chọn sram_sel, địa chỉ addr[12:0], dữ liệu ghi sram_data_i[31:0] và tín hiệu wstrb[3:0]. Tín hiệu wstrb cho phép ghi theo từng byte trong một word 32 bit, nhờ đó bộ nhớ có thể hỗ trợ các thao tác

store byte, store half-word và store word của tập lệnh RV32I. Khi hoàn tất truy cập, khối trả về sram_ready và dữ liệu đọc sram_data_o[31:0].

3.3.5. Khối LED

Khối LED GPIO là ngoại vi đầu ra đơn giản, dùng để hiển thị trạng thái kiểm tra của hệ thống. Trong quá trình phát triển, LED giúp xác nhận CPU đã chạy chương trình, bus hoạt động đúng và chương trình đã đi đến các giai đoạn mong muốn.

- Sơ đồ khối:



Hình 3.15. Sơ đồ khối LED.

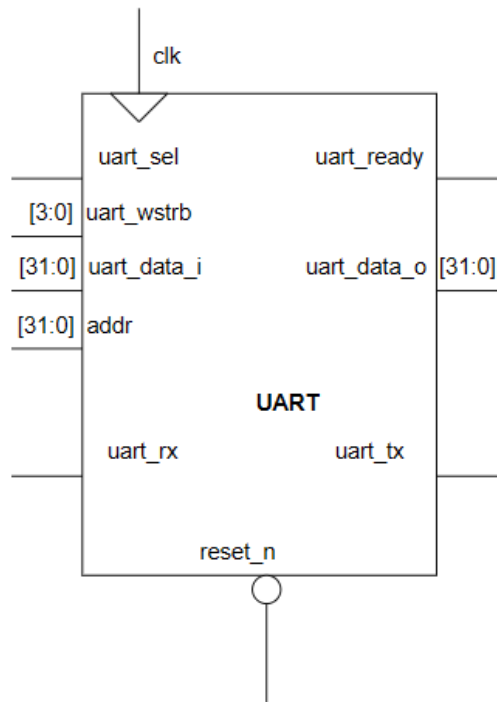
- **Thanh ghi địa chỉ:** Khối LED được ánh xạ tại địa chỉ 0x80000000. Các bit của thanh ghi LED được đưa ra các chân LED trên kit FPGA.

- **Nguyên lý hoạt động:** Khi chương trình nhúng ghi dữ liệu vào địa chỉ 0x80000000, phần cứng chốt dữ liệu vào thanh ghi LED. Giá trị thanh ghi này được duy trì cho đến lần ghi tiếp theo. Khi đọc lại địa chỉ LED, phần cứng có thể trả về giá trị hiện tại của thanh ghi thông qua leds_data_o[31:0] để hỗ trợ kiểm tra trong quá trình gỡ lỗi.

3.3.6. Khối UART

Khối UART dùng để truyền dữ liệu nối tiếp giữa FPGA và máy tính, phục vụ chủ yếu cho việc gỡ lỗi. Thông qua UART, chương trình nhúng có thể gửi ra terminal các thông báo như trạng thái khởi tạo, giá trị cảm biến, mức quạt hoặc vị trí servo.

- Sơ đồ khối:



Hình 3.16. Sơ đồ khối UART.

- **Thanh ghi địa chỉ:** UART sử dụng hai thanh ghi chính. Thanh ghi UART_DIV tại địa chỉ 0x80000008 dùng để cấu hình bộ chia baudrate. Thanh ghi UART_DATA tại địa chỉ 0x8000000C dùng để ghi ký tự cần truyền hoặc đọc dữ liệu nhận được nếu chức năng nhận được sử dụng.

Bảng 3.2. Bảng thanh ghi của khối UART.

Địa chỉ	Tên thanh ghi	Quyền truy cập	Chức năng
0x80000008	UART_DIV	R/W	Cấu hình bộ chia baudrate cho khối UART. Giá trị này quyết định tốc độ truyền dữ liệu nối tiếp.

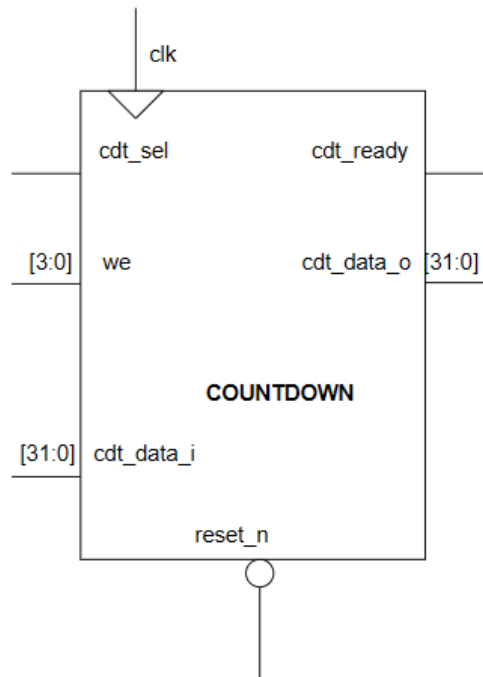
0x8000000C	UART_DATA	R/W	Ghi dữ liệu cần truyền qua chân uart_tx hoặc đọc dữ liệu nhận từ chân uart_rx nếu chức năng nhận được sử dụng.
------------	-----------	-----	--

- **Nguyên lý hoạt động:** Khối UART được CPU truy cập thông qua tín hiệu chọn uart_sel. Khi CPU ghi vào thanh ghi UART_DIV, giá trị bộ chia baudrate được cập nhật để tạo tốc độ truyền phù hợp. Khi CPU ghi vào thanh ghi UART_DATA, byte dữ liệu thấp trong uart_di[7:0] được đưa vào khối truyền UART và phát ra chân uart_tx. Khi CPU đọc vùng địa chỉ UART, dữ liệu phản hồi được đưa về thông qua uart_do[31:0], đồng thời uart_ready báo cho bus biết giao tiếp UART đã hoàn tất.

3.3.7. Khối Countdown Timer

Countdown Timer là bộ đếm lùi 32 bit, dùng để hỗ trợ tạo trễ trong chương trình nhúng. Khối này cần thiết cho các thao tác có yêu cầu thời gian như khởi tạo OLED, chờ cảm biến AHT20 đo xong, tạo khoảng nghỉ khi xử lý nút nhấn và hỗ trợ các hàm delay.

- Sơ đồ khối:



Hình 3.17. Sơ đồ khối Countdown Timer dùng cho hàm delay.

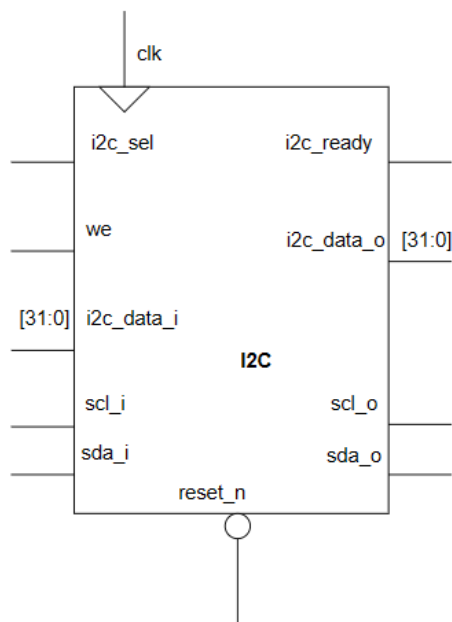
- **Thanh ghi / địa chỉ:** Countdown Timer được ánh xạ tại địa chỉ 0x80000010. Chương trình nhúng có thể ghi một giá trị đếm vào địa chỉ này và đọc lại giá trị hiện tại của bộ đếm. Khi giá trị đọc về bằng 0, khoảng thời gian chờ đã kết thúc.

- **Nguyên lý hoạt động:** Khi CPU ghi một giá trị khác 0 vào thanh ghi timer, bộ đếm bắt đầu giảm theo từng xung clock hoặc theo tick nội bộ của thiết kế. Trong lúc đó, firmware có thể polling thanh ghi timer cho đến khi giá trị về 0. Cách làm này giúp tạo delay ổn định hơn so với vòng lặp rỗng, đồng thời vẫn giữ thiết kế firmware đơn giản, không cần dùng ngắt.

3.3.8. Khối I2C

Khối I2C được sử dụng để giao tiếp với màn hình OLED SSD1309 và các cảm biến AHT20. Trong đề tài, phần cứng I2C cung cấp các thanh ghi để điều khiển và đọc trạng thái hai đường SCL, SDA; trình tự START, STOP, ghi byte, đọc byte và kiểm tra ACK được thực hiện bởi chương trình nhúng.

- **Sơ đồ khối:**



Hình 3.18. Sơ đồ khối I2C bit-bang điều khiển các đường SCL và SDA.

- **Thanh ghi địa chỉ:** Hệ thống sử dụng ba bus I2C độc lập: I2C1 tại 0x80000020, I2C2 tại 0x80000024 và I2C3 tại 0x8000006C. I2C1 dùng cho OLED SSD1309 và cảm biến AHT20 S1, I2C2 dùng cho cảm biến AHT20 S2, còn I2C3 dùng cho cảm biến

AHT20 S3. Việc tách thành nhiều bus giúp hệ thống sử dụng nhiều cảm biến AHT20 có cùng địa chỉ mà không bị xung đột trên một đường I2C chung.

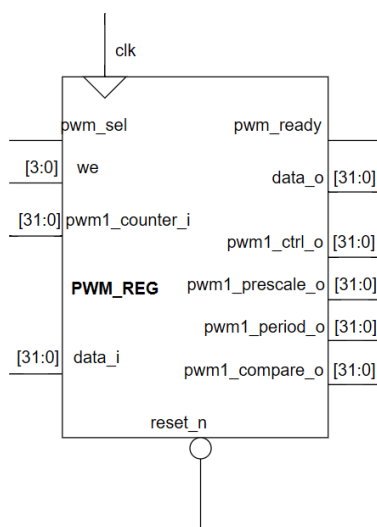
- **Nguyên lý hoạt động:** Khi chương trình ghi vào thanh ghi I2C, phần cứng cập nhật mức logic trên SCL và SDA. Khi chương trình đọc thanh ghi I2C, phần cứng trả về trạng thái hiện tại của các đường tín hiệu. Bằng cách ghi và đọc liên tiếp theo đúng thứ tự, chương trình tạo được các điều kiện START, STOP, truyền bit, đọc bit và kiểm tra ACK/NACK. Việc tách thành ba bus I2C giúp hệ thống có thể dùng nhiều cảm biến AHT20 cùng địa chỉ mà không bị xung đột trên cùng một bus.

3.3.9. Khối PWM

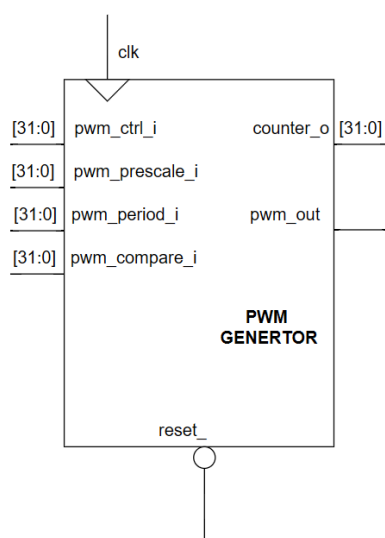
Khối PWM tạo tín hiệu xung có độ rộng thay đổi để điều khiển quạt và servo. Đối với quạt, PWM thay đổi duty cycle để thay đổi tốc độ quay. Đối với servo, PWM tạo xung có chu kỳ khoảng 20 ms và độ rộng xung mức cao thay đổi để mô phỏng vị trí cơ cấu cửa gió.

Trong thiết kế, phần PWM được chia thành hai nhóm khối: khối thanh ghi PWM dùng để CPU cấu hình các thông số điều khiển, và khối PWM Generator dùng để tạo dạng sóng PWM thật ở ngõ ra.

- **Sơ đồ khối:**



Hình 3.19. Sơ đồ khối thanh ghi cấu hình PWM



Hình 3.20. Sơ đồ khối PWM Generator

- **Thanh ghi địa chỉ:** Mỗi kênh PWM gồm các thanh ghi CTRL, PRESCALE, PERIOD, COMPARE và COUNTER. Thanh ghi CTRL dùng để bật/tắt kênh PWM, PRESCALE chia clock hệ thống thành tick thời gian nhỏ hơn, PERIOD xác định chu kỳ PWM, COMPARE xác định thời gian ngõ ra ở mức cao và COUNTER cho phép đọc giá trị bộ đếm hiện tại.

Bảng 3.3. Các thanh ghi của mỗi kênh PWM.

Offset	Thanh ghi	Quyền truy cập	Chức năng
+0x00	CTRL	Đọc/Ghi	Bit cho phép hoạt động và các bit điều khiển cơ bản của kênh PWM.
+0x04	PRESCALE	Đọc/Ghi	Giá trị chia clock để tạo tick đếm cho bộ PWM.
+0x08	PERIOD	Đọc/Ghi	Giá trị chu kỳ của tín hiệu PWM.
+0x0C	COMPARE	Đọc/Ghi	Ngưỡng so sánh quyết định thời gian mức tích cực trong một chu kỳ.
+0x10	COUNTER	Đọc	Giá trị bộ đếm hiện tại, dùng để quan sát hoặc kiểm tra hoạt động.

Ba kênh PWM được ánh xạ lần lượt tại các vùng 0x80000028-0x80000038, 0x8000003C-0x8000004C và 0x80000050-0x80000060. Với clock hệ thống 27 MHz, PRESCALE có thể đặt bằng 27 để tạo tick xấp xỉ 1 us.

- **Nguyên lý hoạt động:** Nguyên lý tạo PWM dựa trên bộ đếm và bộ so sánh. Khi COUNTER nhỏ hơn COMPARE, ngõ ra PWM ở mức tích cực; khi COUNTER lớn hơn hoặc bằng COMPARE, ngõ ra chuyển sang mức không tích cực. Khi COUNTER đạt PERIOD, bộ đếm quay về 0 và bắt đầu chu kỳ mới. Do phần cứng tự tạo xung sau khi được cấu hình, CPU chỉ cần cập nhật các thanh ghi khi muốn thay đổi tốc độ quạt hoặc vị trí servo.

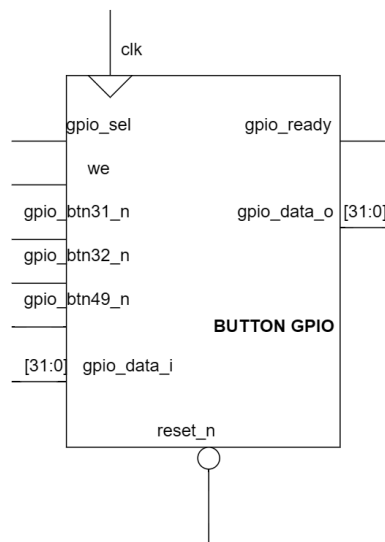
3.3.10. Khối GPIO đọc nút nhấn

Khối GPIO đọc nút nhấn là ngoại vi đầu vào của hệ thống. Khối này nhận trạng thái từ các nút nhấn vật lý và đưa trạng thái đó vào CPU dưới dạng các bit trong thanh ghi đọc. Chương trình nhúng sử dụng các bit này để thay đổi mức quạt, vị trí servo 1 và vị trí servo 2.

Trong phạm vi đề tài, khối GPIO được thiết kế theo dạng rút gọn, chỉ phục vụ chức năng đọc tín hiệu đầu vào từ nút nhấn. Khối này không triển khai đầy đủ các chức năng cấu hình như GPIO trong các SoC thương mại, chẳng hạn như chọn chiều vào/ra, bật/tắt điện trở kéo lên, kéo xuống hoặc cấu hình ngắt ngoài. Các chân nút nhấn được xem là tín hiệu đầu vào cố định, còn CPU chỉ đọc trạng thái của chúng thông qua thanh ghi dữ liệu đã được ánh xạ vào không gian địa chỉ.

Cách thiết kế này giúp giảm độ phức tạp phần cứng và phù hợp với yêu cầu của đề tài, vì hệ thống chỉ cần đọc trạng thái nút nhấn để thay đổi chế độ điều khiển chứ không cần xuất tín hiệu số đa dụng qua GPIO.

- **Sơ đồ khối:**



Hình 3.21. Sơ đồ khối GPIO đọc trạng thái nút nhấn.

- **Thanh ghi địa chỉ:** GPIO nút nhấn được ánh xạ tại địa chỉ 0x80000068. Đây là thanh ghi đọc trạng thái nút nhấn; bit0, bit1 và bit2 tương ứng với ba nút điều khiển của hệ thống. Các tín hiệu nút nhấn trên kit thường là tín hiệu mức thấp khi nhấn, do đó tầng phần cứng hoặc chương trình có thể cần đảo mức logic để thu được trạng thái nhấn theo quy ước của chương trình.

- **Nguyên lý hoạt động:** Tín hiệu từ nút nhấn vật lý được đưa vào khối GPIO Input, sau đó ánh xạ vào các bit trạng thái của thanh ghi GPIO. Khi chương trình cần kiểm tra nút nhấn, CPU thực hiện lệnh load tại địa chỉ 0x80000068. Giá trị đọc được được xử lý bằng phép kiểm tra bit. Để tránh một lần nhấn bị đếm nhiều lần, chương trình so sánh trạng thái nút hiện tại với trạng thái trước đó để phát hiện cạnh nhấn và có thể chèn một khoảng trễ ngắn nhằm giảm ảnh hưởng của dôi nút cơ khí.

3.4 Sơ đồ nguyên lý

Sau khi các khối phần cứng chính của hệ thống đã được thiết kế và mô tả ở các mục trước, sơ đồ nguyên lý được xây dựng nhằm thể hiện mối liên kết thực tế giữa kit FPGA Tang Nano 9K với các ngoại vi bên ngoài. Sơ đồ này bao gồm các kết nối đến cảm biến AHT20, màn hình OLED SSD1309, nút nhấn điều khiển, quạt DC, servo và các khối nguồn cần thiết. Thông qua sơ đồ nguyên lý, có thể thấy rõ cách các tín hiệu điều khiển từ FPGA được đưa đến từng thiết bị, đồng thời thể hiện vai trò của các giao tiếp như I2C, GPIO và PWM trong toàn bộ hệ thống.

CHƯƠNG 4. THIẾT KẾ PHẦN MỀM NHÚNG CHẠY TRÊN CPU RISC-V

4.1. Tổng quan phần mềm nhúng

Chương này trình bày quá trình thiết kế phần mềm nhúng chạy trên CPU RV32I trong hệ thống SoC đã xây dựng ở Chương 3. Phần mềm nhúng được viết bằng ngôn ngữ C, chạy trực tiếp trên CPU, không sử dụng hệ điều hành thời gian thực và không triển khai cơ chế ngắt phức tạp. Toàn bộ quá trình điều khiển phần cứng được thực hiện bằng cách đọc hoặc ghi các thanh ghi ngoại vi đã được ánh xạ vào không gian địa chỉ của CPU.

Trong hệ thống, chương trình nhúng có nhiệm vụ khởi tạo ngoại vi, đọc trạng thái nút nhấn, giao tiếp với các cảm biến AHT20 thông qua I2C, cập nhật dữ liệu lên màn hình OLED SSD1309, điều khiển quạt và hai servo bằng PWM, đồng thời gửi thông tin kiểm tra qua UART. Cách tổ chức này giúp kiểm chứng rõ mối liên hệ giữa CPU RISC-V, bus hệ thống, các ngoại vi phần cứng và chương trình điều khiển chạy trên CPU.

Do hệ thống giám sát nhiệt độ, độ ẩm và điều khiển quạt, servo không yêu cầu thời gian đáp ứng quá khắt khe như các hệ thống thời gian thực cứng, phần mềm được tổ chức theo vòng lặp chính kết hợp phương pháp polling. Chương trình lần lượt kiểm tra trạng thái các ngoại vi, xử lý dữ liệu và cập nhật đầu ra theo chu kỳ. Phương pháp này đơn giản, dễ kiểm thử và phù hợp với SoC tối giản trên FPGA.

4.2. Quy trình biên dịch và tạo file khởi tạo bộ nhớ

Chương trình nhúng được biên dịch bằng RISC-V GNU Toolchain. Bộ công cụ được chọn cho môi trường RISC-V bare-metal, tức là chương trình chạy trực tiếp trên phần cứng mà không cần hệ điều hành. Kiến trúc đích được cấu hình cho CPU RV32I và ABI 32 bit để bảo đảm mã máy sinh ra phù hợp với CPU RISC-V 32 bit đã thiết kế.

Quá trình biên dịch bắt đầu từ các file mã nguồn C và file khởi động assembly. File khởi động thực hiện bước khởi động tối thiểu như thiết lập stack pointer và chuyển quyền điều khiển sang chương trình chính. Sau đó, các file mã nguồn được biên dịch thành file object, liên kết với linker script để tạo file chương trình dạng ELF. Linker

script quy định vị trí đặt mã lệnh, dữ liệu và vùng stack trong không gian bộ nhớ SRAM của SoC.

Sau khi tạo file ELF, chương trình được chuyển sang dạng nhị phân và tiếp tục tạo file khởi tạo bộ nhớ. File này được nạp vào SRAM/BRAM của SoC để CPU có thể đọc lệnh và bắt đầu thực thi sau khi FPGA được cấu hình.

Mã nguồn C và startup.s → file object → file ELF → file nhị phân → file khởi tạo SRAM/BRAM

4.3. Thiết kế lớp truy cập thanh ghi phần cứng

Lớp truy cập thanh ghi phần cứng là phần thấp nhất của chương trình nhúng. Lớp này liên kết trực tiếp với bản đồ địa chỉ đã thiết kế trong Chương 3. Mỗi ngoại vi được gán một hoặc nhiều địa chỉ cố định trong không gian địa chỉ của CPU. Khi chương trình C đọc hoặc ghi vào các địa chỉ này, CPU sẽ phát giao dịch bus đến đúng module ngoại vi thông qua khối giải mã địa chỉ trong SoC.

Trong chương trình C, các địa chỉ ngoại vi thường được định nghĩa bằng macro hoặc con trỏ volatile. Từ khóa volatile giúp trình biên dịch không tối ưu bỏ các thao tác đọc/ghi thanh ghi, vì giá trị tại các địa chỉ này có thể thay đổi do phần cứng ngoại vi chứ không chỉ do chương trình C.

Ví dụ cách mô tả truy cập thanh ghi trong chương trình C:

```
#define REG32(addr) (*(volatile unsigned int *)(addr))
#define UART_DATA REG32(0x8000000C)
#define GPIO_BUTTON REG32(0x80000068)
#define PWM1_CMP REG32(0x80000034)
```

Bảng 4.1. Các thanh ghi ngoại vi được phần mềm nhúng sử dụng.

Địa chỉ	Tên/nhóm thanh ghi	Mục đích sử dụng trong chương trình
0x80000000	LED	Ghi trạng thái LED để kiểm tra hoạt động hệ thống.
0x80000008	UART_DIV	Cấu hình bộ chia baudrate cho UART.

0x8000000C	UART_DATA	Ghi ký tự cần truyền hoặc đọc dữ liệu nhận nếu chức năng nhận được sử dụng.
0x80000010	Countdown Timer	Tạo khoảng trễ hoặc mốc thời gian phục vụ chương trình.
0x80000020	I2C1	Điều khiển I2C cho OLED SSD1309 và cảm biến AHT20 thứ nhất.
0x80000024	I2C2	Điều khiển I2C cho cảm biến AHT20 thứ hai.
0x80000028-0x80000038	PWM1	Cấu hình PWM điều khiển quạt.
0x8000003C-0x8000004C	PWM2	Cấu hình PWM điều khiển servo thứ nhất.
0x80000050-0x80000060	PWM3	Cấu hình PWM điều khiển servo thứ hai.
0x80000068	GPIO Button	Đọc trạng thái các nút nhấn điều khiển.
0x8000006C	I2C3	Điều khiển I2C cho cảm biến AHT20 thứ ba.

Nhờ lớp truy cập thanh ghi, các driver phía trên không cần biết chi tiết bus Verilog bên dưới. Driver chỉ thực hiện thao tác đọc/ghi vào địa chỉ đã định nghĩa, còn phần cứng SoC chịu trách nhiệm giải mã địa chỉ, chọn ngoại vi tương ứng và phản hồi dữ liệu về CPU.

4.4. Hoạt động của các chương trình con

4.4.1. Giao tiếp I2C

Trong đề tài, giao tiếp I2C được hiện thực theo hướng điều khiển bằng chương trình nhúng. Phần cứng SoC cung cấp các thanh ghi để CPU điều khiển và đọc trạng thái hai đường tín hiệu SCL, SDA. Trong khi đó, các trình tự của giao thức I2C như tạo

điều kiện START, STOP, truyền dữ liệu, nhận dữ liệu và kiểm tra tín hiệu ACK được thực hiện bằng chương trình nhúng.

Cách thiết kế này giúp giảm độ phức tạp của phần cứng vì không cần xây dựng một bộ điều khiển I2C hoàn chỉnh bằng FSM riêng trong Verilog. Thay vào đó, chương trình nhúng chủ động tạo trình tự tín hiệu cần thiết để giao tiếp với các thiết bị ngoại vi. Trong hệ thống, khối giao tiếp I2C được sử dụng để trao đổi dữ liệu với cảm biến AHT20 và màn hình OLED SSD1309.

4.4.2. Giao tiếp cảm biến AHT20

Cảm biến AHT20 được sử dụng để đo nhiệt độ và độ ẩm trong hệ thống. Sau khi được cấp nguồn và khởi tạo, chương trình nhúng gửi lệnh đo đến cảm biến thông qua giao tiếp I2C. Sau một khoảng thời gian chờ để cảm biến hoàn thành quá trình chuyển đổi, chương trình đọc các byte dữ liệu thô trả về từ cảm biến.

Dữ liệu thô của AHT20 chứa thông tin độ ẩm và nhiệt độ dưới dạng các trường bit. Vì vậy, chương trình cần ghép các byte dữ liệu, tách phần dữ liệu tương ứng với độ ẩm và nhiệt độ, sau đó chuyển đổi sang giá trị vật lý. Các giá trị sau khi chuyển đổi được dùng để hiển thị trên OLED và làm dữ liệu đầu vào cho thuật toán đánh giá trạng thái môi trường cabin.

4.4.3. Giao tiếp OLED SSD1309

Màn hình OLED SSD1309 được sử dụng để hiển thị dữ liệu đo và trạng thái điều khiển. Driver OLED có nhiệm vụ gửi các lệnh khởi tạo màn hình, thiết lập chế độ hiển thị, xóa vùng hiển thị, đặt vị trí con trỏ và ghi dữ liệu ký tự. Vì màn hình giao tiếp qua I2C, driver OLED được xây dựng trên các hàm I2C đã trình bày ở trên.

Sau khi khởi tạo, chương trình cập nhật các dòng thông tin chính như nhiệt độ, độ ẩm của các cảm biến, mức quạt, trạng thái servo và cảnh báo nếu có. Việc cập nhật OLED được thực hiện theo chu kỳ trong vòng lặp chính. Chương trình chỉ cần chuyển giá trị số đo sang chuỗi ký tự, đặt vị trí hiển thị và gửi dữ liệu qua I2C.

4.5. Xử lý dữ liệu cảm biến và công thức tính toán

Cảm biến AHT20 trả về dữ liệu nhiệt độ và độ ẩm dưới dạng giá trị nhị phân thô 20 bit. Độ ẩm tương đối và nhiệt độ được chuyển đổi theo các công thức:

$$RH = \frac{S_{RH}}{2^{20}} * 100 \quad (\%) \quad (4.1)$$

$$T = \frac{S_T}{2^{20}} * 200 - 50 \quad (^\circ C) \quad (4.2)$$

Trong đó, S_RH là giá trị nhị phân thô 20 bit của độ ẩm và S_T là giá trị nhị phân thô 20 bit của nhiệt độ. Để giảm độ phức tạp khi chạy trên CPU RISC-V tối giản, chương trình biểu diễn nhiệt độ và độ ẩm ở dạng số nguyên nhân 10. Ví dụ, giá trị 253 tương ứng với 25,3 °C

Từ nhiệt độ và độ ẩm trong cabin, chương trình ước lượng nhanh điểm đọng sương theo công thức tuyến tính:

$$DPT = T_{\text{trong}} - \frac{100 - RH_{\text{trong}}}{5} \quad (4.3)$$

$$\Delta T = T_{\text{kinh}} - DPT \quad (4.4)$$

Trong đó, T_trong là nhiệt độ trong cabin, RH_trong là độ ẩm tương đối trong cabin và T_kinh là nhiệt độ tại khu vực kính. Giá trị delta biểu diễn khoảng cách giữa nhiệt độ kính và điểm đọng sương. Nếu delta càng nhỏ thì nguy cơ đọng sương càng cao. Trong chương trình, dữ liệu được lưu ở dạng nhân 10 nên ngưỡng delta <= 50 tương ứng với 5,0 °C. Khi delta <= 50, OLED hiển thị cảnh báo WARN; khi delta < 0, hệ thống hiển thị mức cảnh báo SUONG NANG.

Công thức DPT trên là công thức xấp xỉ nhanh, phù hợp cho mô hình thực nghiệm và logic điều khiển trong đề tài. Nếu cần độ chính xác cao hơn trong các ứng dụng thực tế, có thể thay bằng các mô hình tính điểm đọng sương chính xác hơn.

4.6. Thuật toán điều khiển

Thuật toán điều khiển của hệ thống được tổ chức theo hai chế độ chính là AUTO và MANUAL. Chế độ hiện tại được xác định bằng biến trạng thái is_manual_mode. Khi is_manual_mode = 0, hệ thống hoạt động ở chế độ AUTO; khi is_manual_mode = 1, hệ thống hoạt động ở chế độ MANUAL. Chương trình mặc định khởi động ở chế độ AUTO.

Ở chế độ AUTO, chương trình sử dụng dữ liệu nhiệt độ, độ ẩm và giá trị delta để lựa chọn trạng thái điều khiển phù hợp. Giá trị delta được tính từ hiệu giữa nhiệt độ khu vực kính và điểm đọng sương. Các giá trị nhiệt độ trong chương trình được biểu diễn theo dạng nhân 10, do đó delta = 50 tương ứng với 5.0°C.

Thuật toán AUTO được tổ chức theo các trạng thái ưu tiên như sau. Nếu nhiệt độ ngoài trời nhỏ hơn nhiệt độ trong cabin, hệ thống chuyển sang trạng thái ngoài trời mát và điều chỉnh servo để ưu tiên lấy gió ngoài. Nếu nhiệt độ trong cabin vượt ngưỡng nhiệt độ cao, hệ thống tăng mức quạt để hỗ trợ làm mát. Nếu $\Delta T \leq 0$, hệ thống xem đây là trạng thái sương nặng và đặt quạt, servo ở mức mạnh hơn. Nếu $\Delta T \leq 5.0^{\circ}\text{C}$, hệ thống xem đây là trạng thái sương nhẹ và điều khiển ở mức trung bình. Nếu không thuộc các trường hợp trên, hệ thống được xem là an toàn và duy trì mức điều khiển thấp.

Ở chế độ MANUAL, mức quạt, servo1 và servo2 được lấy theo giá trị người dùng chọn thông qua các nút nhấn. Khi người dùng nhấn nút điều khiển quạt hoặc servo trong lúc hệ thống đang ở AUTO, chương trình chuyển sang MANUAL và lưu lại trạng thái hiện tại làm giá trị điều khiển ban đầu. Sau đó, mỗi lần nhấn nút sẽ tăng mức tương ứng theo vòng lặp giá trị đã quy định. Khi người dùng nhấn giữ nút đủ lâu, bộ đếm thời gian nhấn giữ đạt ngưỡng đặt trước và hệ thống tự chuyển lại chế độ AUTO.

Sau khi xác định được mức quạt và vị trí servo ở chế độ AUTO hoặc MANUAL, chương trình cập nhật giá trị PWM tương ứng để điều khiển quạt, servo1 và servo2. Đồng thời, trạng thái hiện tại của hệ thống được cập nhật lên màn hình OLED để người dùng quan sát.

Bảng 4.2. Các trạng thái điều khiển tự động của hệ thống

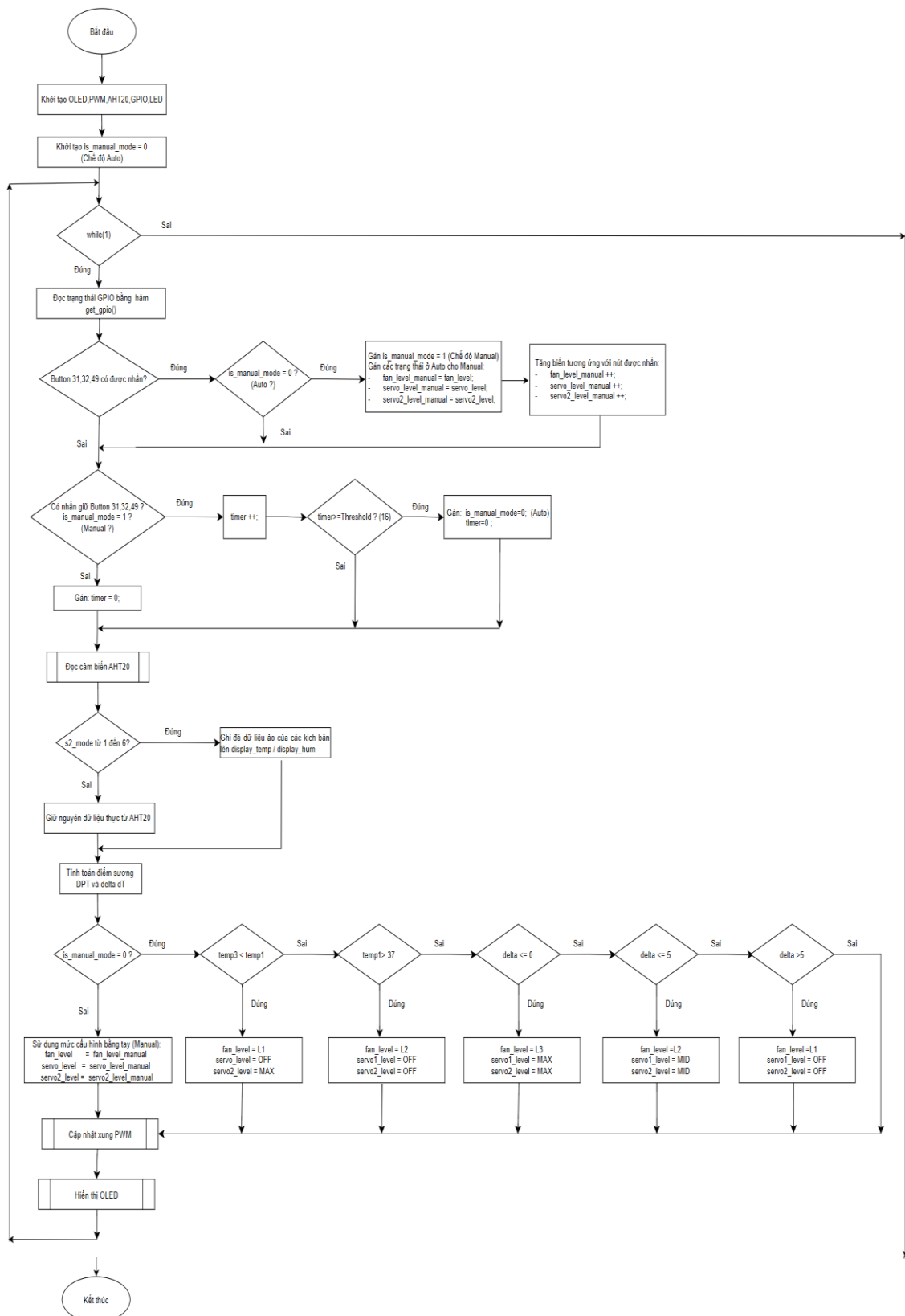
Trạng thái	Điều kiện	Ý nghĩa	Điều khiển
P1	$T_{\text{ngoài}} < T_{\text{trong}}$	Ngoài trời mát hơn cabin	Quạt L1, servo1 OFF, servo2 MAX
P2	$T_{\text{trong}} > 40^{\circ}\text{C}$	Nhiệt độ cabin cao	Quạt L2, servo1 OFF, servo2 OFF
P3	$\Delta T \leq 0^{\circ}\text{C}$	Nguy cơ đọng sương nặng	Quạt L3, servo1 MAX, servo2 MAX
P4	$\Delta T \leq 5^{\circ}\text{C}$	Cảnh báo sớm đọng sương	Quạt L2, servo1 MID, servo2 MID
P5	Các trường hợp còn lại	Trạng thái an toàn	Quạt L1, servo1 OFF, servo2 OFF

Trong bảng 4.2 trình bày các trạng thái điều khiển tự động của hệ thống. L1, L2 và L3 lần lượt là các mức thấp, trung bình và cao của quạt. Các trạng thái OFF, MID và

MAX của servo biểu diễn vị trí đóng, mở trung bình và mở lớn nhất của cửa gió. Các mức này được dùng để mô phỏng quá trình điều khiển quạt và hướng gió của hệ thống theo từng trạng thái môi trường.

4.7. Lưu đồ chương trình chính

Lưu đồ chương trình chính được trình bày ở Hình 4.1. Sau khi khởi tạo hệ thống, chương trình đi vào vòng lặp chính. Trong mỗi vòng lặp, chương trình kiểm tra nút nhấn, đọc dữ liệu cảm biến, tính toán điểm động sương và delta, lựa chọn chế độ điều khiển, cập nhật PWM và cập nhật OLED. Do chương trình chạy trong vòng lặp vô hạn, sau khi hoàn thành một chu kỳ xử lý, hệ thống quay lại đầu vòng lặp để tiếp tục giám sát môi trường.



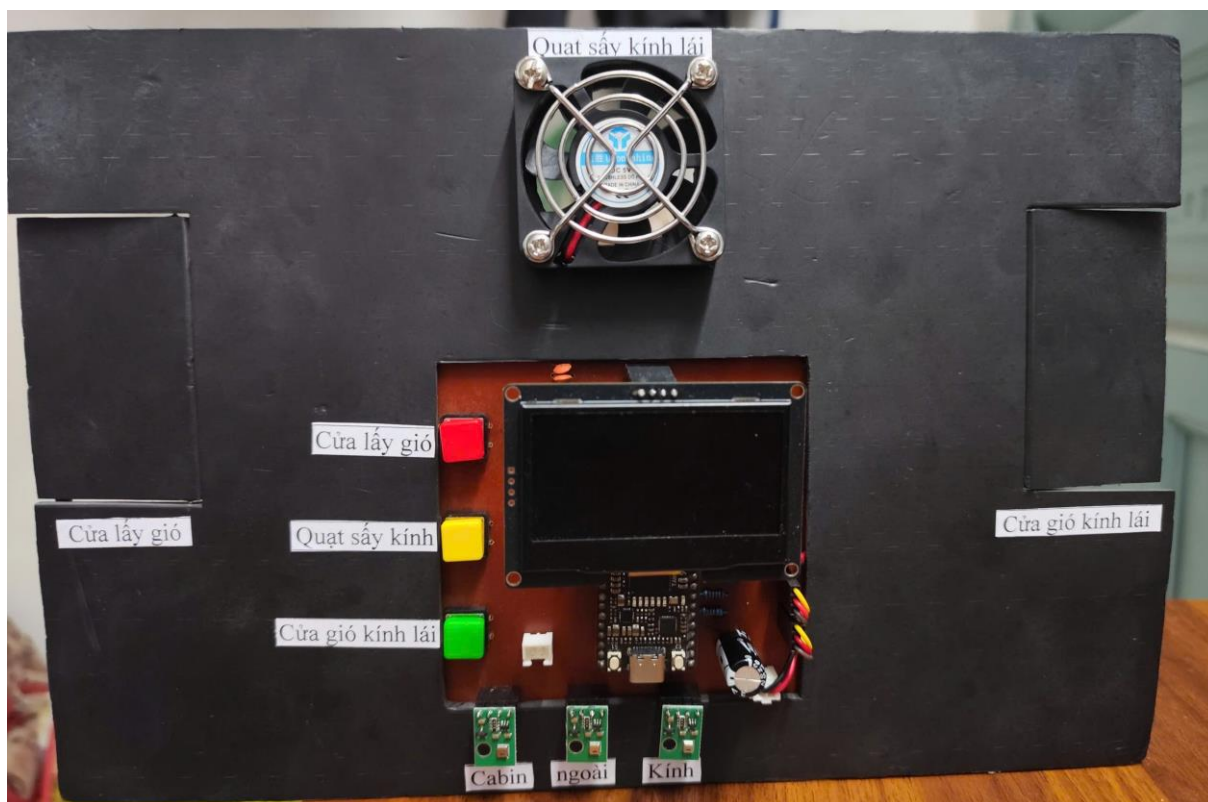
Hình 4.1. Lưu đồ chương trình chính của phần mềm nhúng.

CHƯƠNG 5 KẾT QUẢ

Chương này trình bày các kết quả đạt được sau khi tích hợp phần cứng SoC RISC-V, firmware nhúng và các thiết bị ngoại vi trên mô hình thử nghiệm. Nội dung chương tập trung vào kết quả mô hình thi công, kết quả phần mềm hiển thị và kịch bản kiểm chứng hoạt động của hệ thống. Các kết quả được trình bày dựa trên hình chụp mô hình thực tế và màn hình OLED trong quá trình vận hành hệ thống.

5.1. Kết quả mô hình thi công

Sau khi hoàn thành thiết kế phần cứng và phần mềm, hệ thống được triển khai trên kit FPGA Tang Nano 9K và kết nối với các ngoại vi gồm màn hình OLED, cảm biến AHT20, quạt, servo và các nút nhấn điều khiển. Hình 5.1 trình bày mô hình thực tế sau khi lắp ráp để kiểm tra hoạt động của toàn hệ thống.



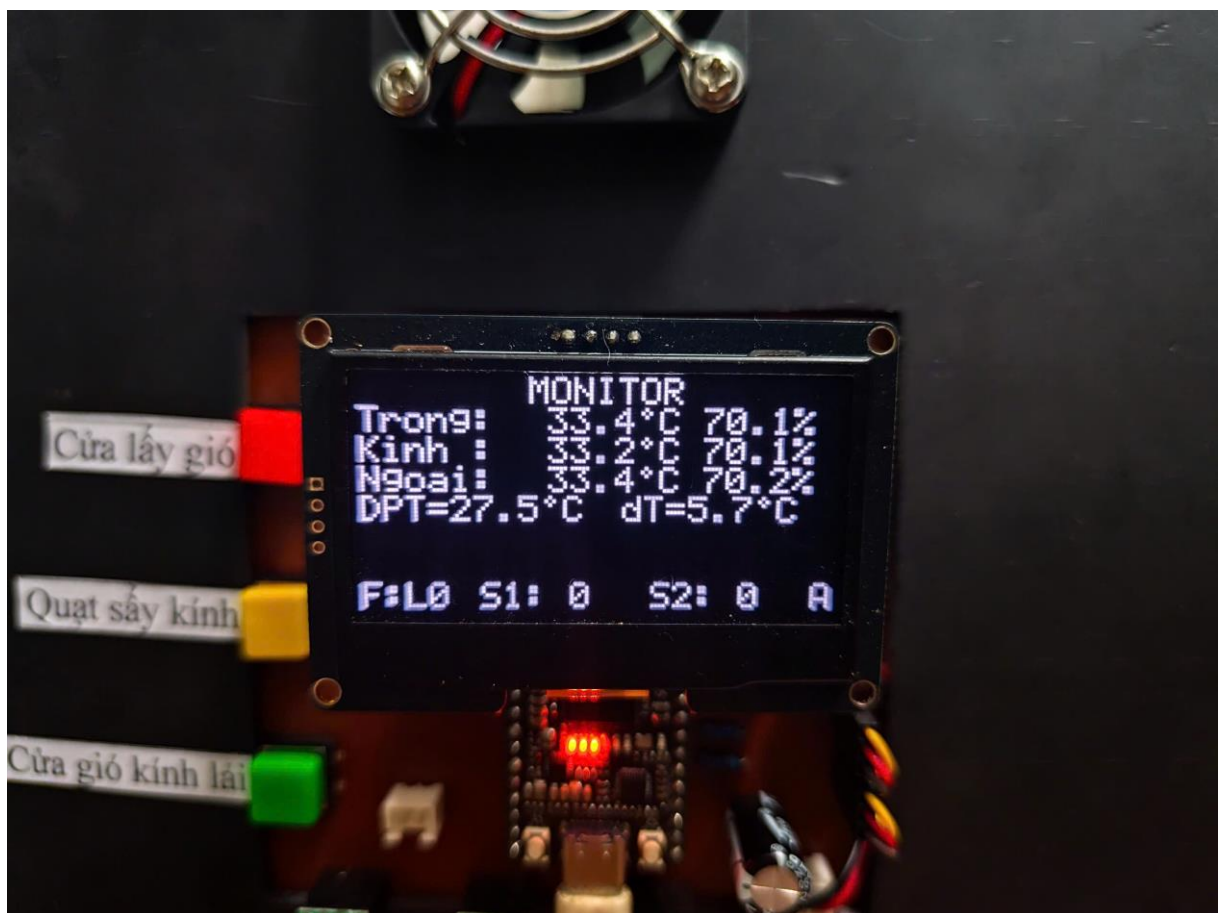
Hình 5.1. Mô hình thi công hệ thống giám sát môi trường cabin ô tô sử dụng CPU RISC-V trên FPGA.

Trong mô hình này, FPGA đóng vai trò là trung tâm xử lý, thực thi chương trình nhúng trên CPU RISC-V và điều khiển các ngoại vi thông qua cơ chế Memory-Mapped I/O. Các module cảm biến, OLED, quạt và servo được bố trí xung quanh board để phục

vụ kiểm tra từng chức năng. Việc mô hình có thể khởi động, hiển thị dữ liệu và phản ứng với nút nhấn cho thấy phần cứng và firmware đã được tích hợp thành một hệ thống hoàn chỉnh ở mức nguyên mẫu.

5.2. Kết quả phần mềm thiết kế

Phần mềm nhúng được biên dịch từ chương trình C và nạp vào bộ nhớ chương trình của SoC. Sau khi CPU reset, firmware thực hiện các bước khởi tạo I2C, OLED, cảm biến AHT20, PWM và GPIO, sau đó chuyển sang vòng lặp chính để đọc cảm biến, xử lý dữ liệu và cập nhật nội dung hiển thị. Hình 5.2 thể hiện kết quả hiển thị trên OLED khi hệ thống đang vận hành.



Hình 5.2. Kết quả hiển thị dữ liệu cảm biến và trạng thái điều khiển trên OLED.

Nội dung trên OLED thể hiện các giá trị đo từ các cảm biến AHT20, giá trị điểm sương, độ lệch nhiệt độ và trạng thái điều khiển của quạt, servo. Kết quả này cho thấy firmware đã đọc được dữ liệu từ cảm biến qua I2C, chuyển đổi dữ liệu thô sang dạng hiển thị và gửi chuỗi ký tự đến OLED SSD1309. UART trong hệ thống chỉ được sử

dụng cho mục đích gỡ lỗi và quan sát thông báo khởi động, không dùng để gửi lệnh điều khiển quạt hoặc servo.

5.3. Hoạt động của hệ thống

Để kiểm chứng hoạt động của hệ thống, các chức năng chính được kiểm tra theo trình tự từ thấp đến cao, bắt đầu từ việc xác nhận phần cứng cơ bản trước khi tiến đến các thuật toán điều khiển. Trước hết, LED trạng thái được sử dụng để xác nhận CPU đã khởi động thành công và thực thi chương trình nhúng. Sau đó, màn hình OLED được kiểm tra nhằm xác nhận giao tiếp I2C hoạt động ổn định. Tiếp theo, cảm biến AHT20 được cấu hình để đọc dữ liệu định kỳ và cập nhật liên tục các giá trị nhiệt độ, độ ẩm lên màn hình.

Từ dữ liệu nhiệt độ và độ ẩm thu được, chương trình sử dụng công thức tính điểm động sương đã trình bày ở Chương 4 để ước lượng nhanh giá trị DPT trong cabin.

Sau khi các khối phần cứng và phần mềm đã hoạt động đồng bộ, hệ thống tiếp tục được kiểm thử ở mức tích hợp. Các kịch bản mô phỏng được xây dựng nhằm đánh giá khả năng phản ứng của hệ thống trong nhiều điều kiện môi trường khác nhau, bao gồm trạng thái an toàn, cảnh báo sớm nguy cơ động sương, mờ kính nặng, ngoài trời mát và nắng nóng gay gắt.

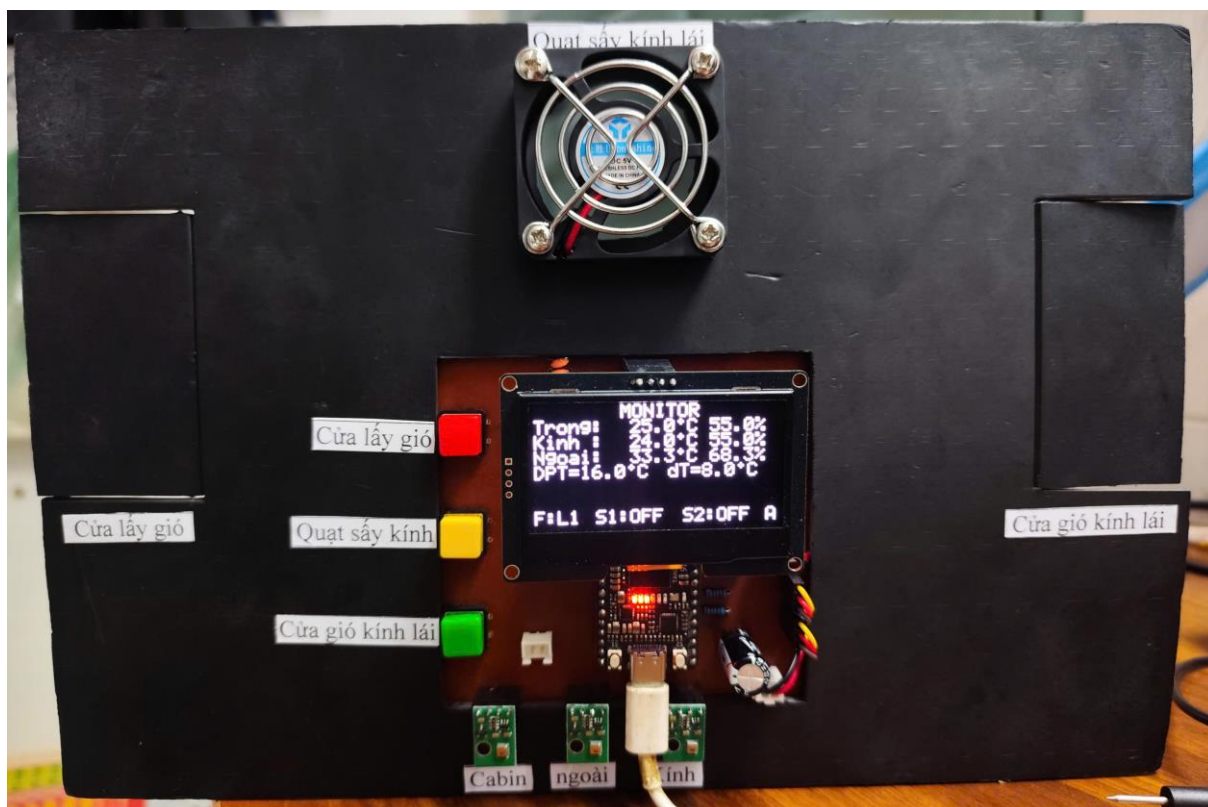
Bảng 5.1. Các kịch bản kiểm thử hệ thống giám sát môi trường cabin ô tô

STT	Tên kịch bản	Điều kiện kiểm thử chính	Delta	Fan	Servo 1	Servo 2	Mục tiêu kiểm thử
1	Thời tiết đẹp (An toàn)	RH thấp, kính khô ráo	80	1	OFF	OFF	Kiểm tra chế độ vận hành bình thường và tiết kiệm năng lượng
2	Cảnh báo sớm (Chớm mờ)	Nguy cơ động sương thấp	30	2	MID	MID	Kiểm tra cơ chế phòng ngừa chủ động
3	Mờ trong kính nặng	RH cao, nhiệt độ kính thấp hơn điểm sương	-20	3	MAX	MAX	Kiểm tra chế độ phá sương khẩn cấp

STT	Tên kịch bản	Điều kiện kiểm thử chính	Delta	Fan	Servo 1	Servo 2	Mục tiêu kiểm thử
4	Ngoài trời mát mẻ	T_ngoai thấp hơn T_trong	90	1	OFF	MAX	Kiểm tra chế độ tản dụng khí trời
5	Nắng nóng gay gắt	T_ngoai rất cao (>35°C)	110	2	OFF	OFF	Kiểm tra chế độ cách ly môi trường

5.3.1. Kịch bản 1: Thời tiết đẹp (Trạng thái an toàn):

Kịch bản này được xây dựng nhằm kiểm tra khả năng vận hành bình thường của hệ thống trong điều kiện môi trường thuận lợi, khi không tồn tại nguy cơ động sương trên kính lái. Các cảm biến ghi nhận nhiệt độ cabin và độ ẩm ở mức ổn định, dẫn đến nhiệt độ điểm sương thấp hơn đáng kể so với nhiệt độ bề mặt kính. Giá trị chênh lệch nhiệt độ đạt $\Delta T = 80$, lớn hơn ngưỡng an toàn được thiết lập trong thuật toán điều khiển. Trong trường hợp này, CPU RISC-V xác định kính lái đang ở trạng thái khô ráo và không cần thực hiện chức năng phá sương. Do đó, hệ thống chuyển sang chế độ tối ưu hóa năng lượng bằng cách duy trì tốc độ quạt ở mức thấp (Fan = 1), đồng thời đưa hai servo về trạng thái OFF. Cách điều khiển này giúp tập trung luồng khí làm mát cho khoang hành khách và giảm tải cho hệ thống điều hòa.

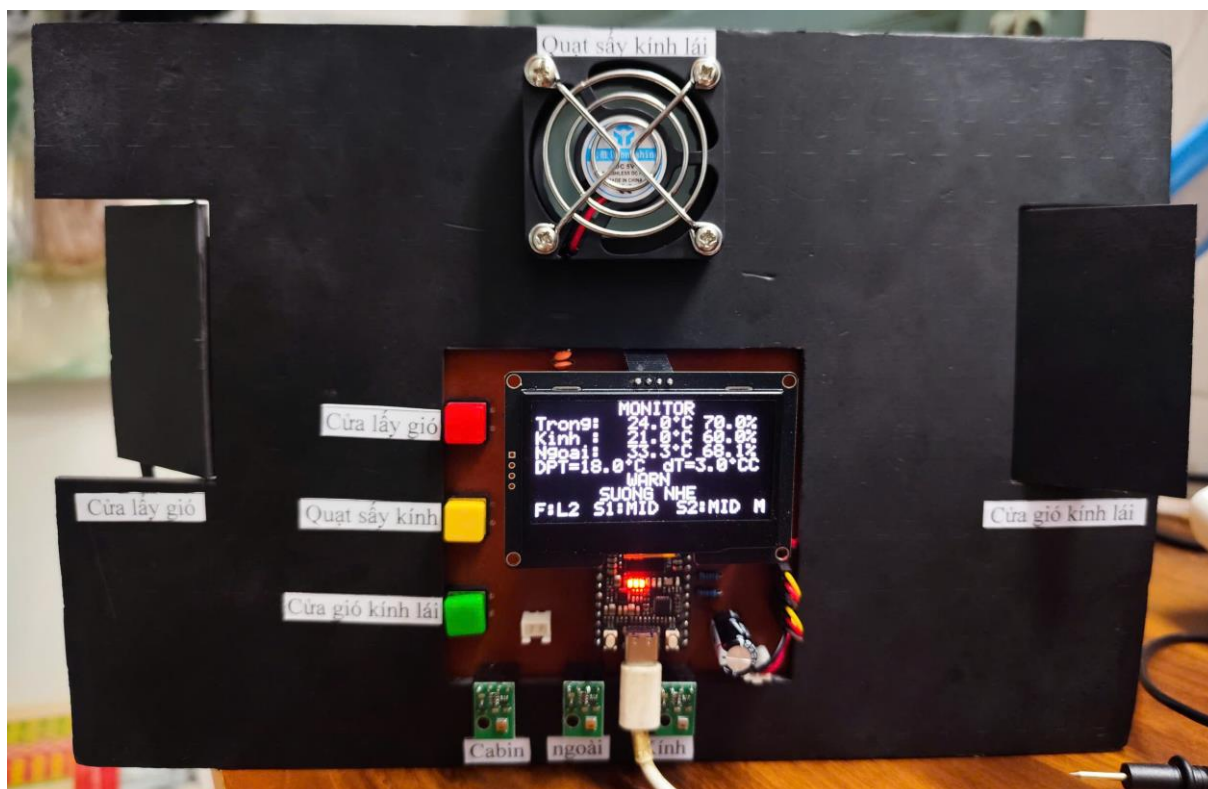


Hình 5.3. Kết quả mô phỏng kịch bản 1 – Thời tiết đẹp (Trạng thái an toàn)

Nhận xét: Kết quả thu được phù hợp với yêu cầu thiết kế. Hệ thống nhận diện chính xác trạng thái an toàn và đưa các cơ cấu chấp hành về chế độ tiết kiệm năng lượng.

5.3.2. Kịch bản 2: Cảnh báo sớm (Chớm động sương)

Kịch bản này được sử dụng để đánh giá khả năng phòng ngừa chủ động của hệ thống trước khi hiện tượng động sương thực sự xảy ra. Với giá trị $\Delta T = 30$, nhiệt độ kính lái vẫn cao hơn nhiệt độ điểm sương nhưng khoảng cách an toàn đã giảm đáng kể. CPU RISC-V nhận diện đây là vùng cảnh báo và kích hoạt chế độ điều khiển trung gian. Tốc độ quạt được nâng lên mức vừa ($Fan = 2$), trong khi hai servo được đặt ở vị trí MID nhằm tăng cường lưu thông không khí và giảm dần độ ẩm trong cabin.



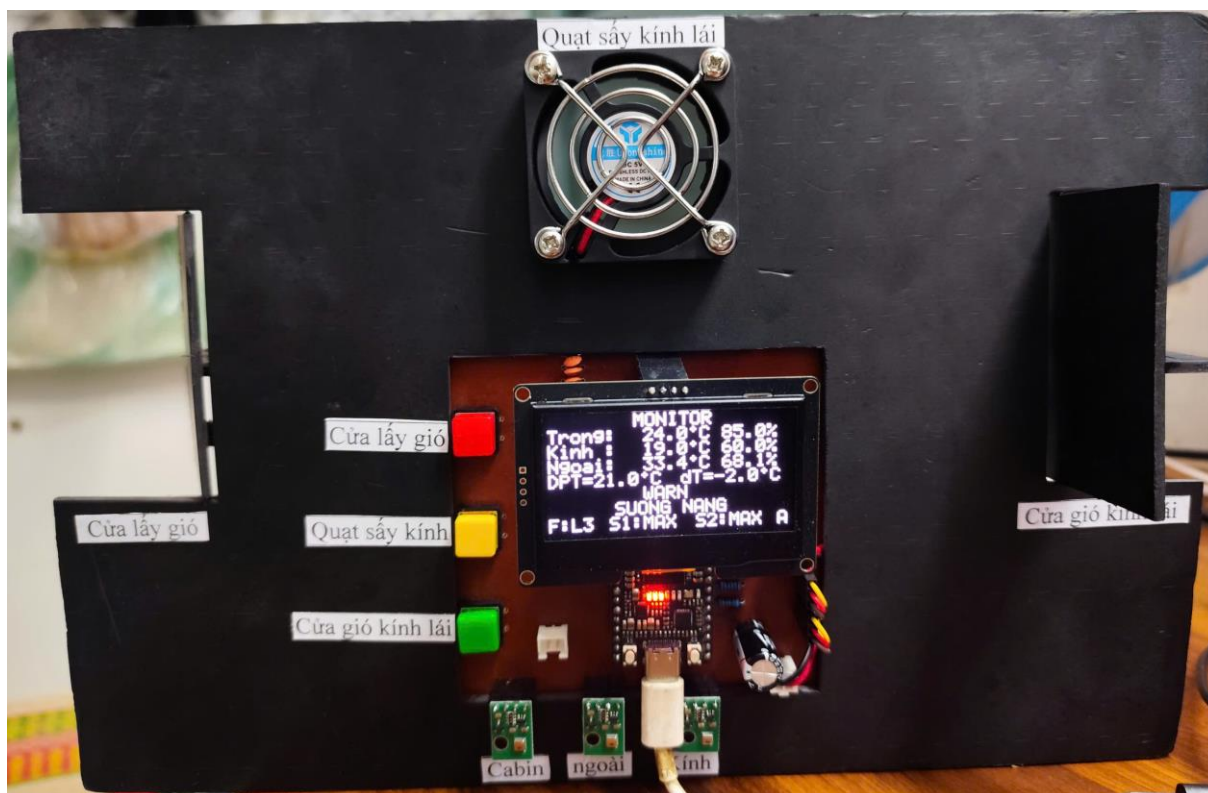
Hình 5.4. Kết quả mô phỏng kịch bản 2 – Cảnh báo sớm (Chớm động sương)

Nhận xét: Hệ thống có khả năng dự đoán nguy cơ động sương và thực hiện các biện pháp phòng ngừa trước khi tầm nhìn bị ảnh hưởng.

5.3.3. Kịch bản 3: Mờ trong kính nặng

Kịch bản này mô phỏng điều kiện thời tiết mưa hoặc môi trường có độ ẩm cao bên trong cabin. Khi độ ẩm tăng mạnh, nhiệt độ điểm sương tiến gần hoặc vượt quá nhiệt độ bề mặt kính lái, làm xuất hiện nguy cơ ngưng tụ hơi nước trên kính. Trong trường hợp này, giá trị $\Delta T = -20$ cho thấy nhiệt độ kính đã thấp hơn nhiệt độ điểm sương và hiện tượng động sương có thể xảy ra.

Sau khi xử lý dữ liệu cảm biến, CPU RISC-V xác định đây là tình huống nguy hiểm ảnh hưởng trực tiếp đến tầm nhìn của người lái. Thuật toán điều khiển lập tức chuyển sang chế độ phá sương khẩn cấp. Tốc độ quạt được tăng lên mức cao nhất (Fan = 3), đồng thời Servo 1 và Servo 2 được điều khiển đến vị trí MAX nhằm tạo lưu lượng khí lớn nhất hướng tới khu vực kính lái và tăng cường khả năng trao đổi không khí.



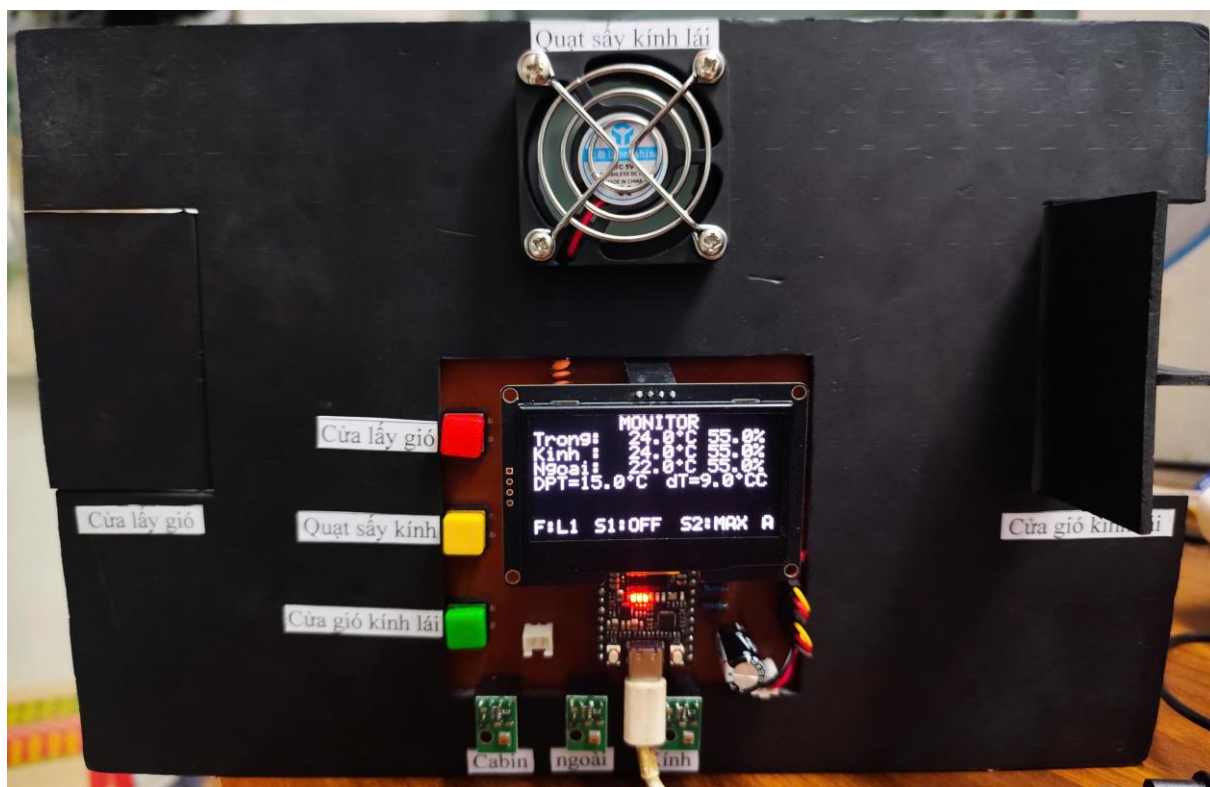
Hình 5.5. Kết quả mô phỏng kịch bản 3 – Mờ trong kính năng

Nhận xét: Hệ thống phản ứng đúng với điều kiện nguy hiểm, ưu tiên chức năng phá sương để đảm bảo tầm nhìn cho người điều khiển phương tiện.

5.3.4. Kịch bản 4: Ngoài trời mát mẻ

Kịch bản này mô phỏng điều kiện môi trường bên ngoài có nhiệt độ thấp hơn nhiệt độ bên trong cabin. Mặc dù kính lái vẫn ở trạng thái an toàn ($\Delta T = 90$), thuật toán nhận diện được tiềm năng tận dụng nguồn không khí tự nhiên để hỗ trợ làm mát.

CPU RISC-V duy trì tốc độ quạt ở mức thấp ($Fan = 1$), giữ Servo 1 ở trạng thái OFF và điều khiển Servo 2 đến vị trí MAX để tăng cường lấy gió ngoài. Giải pháp này giúp giảm tải cho hệ thống điều hòa và nâng cao hiệu quả sử dụng năng lượng.



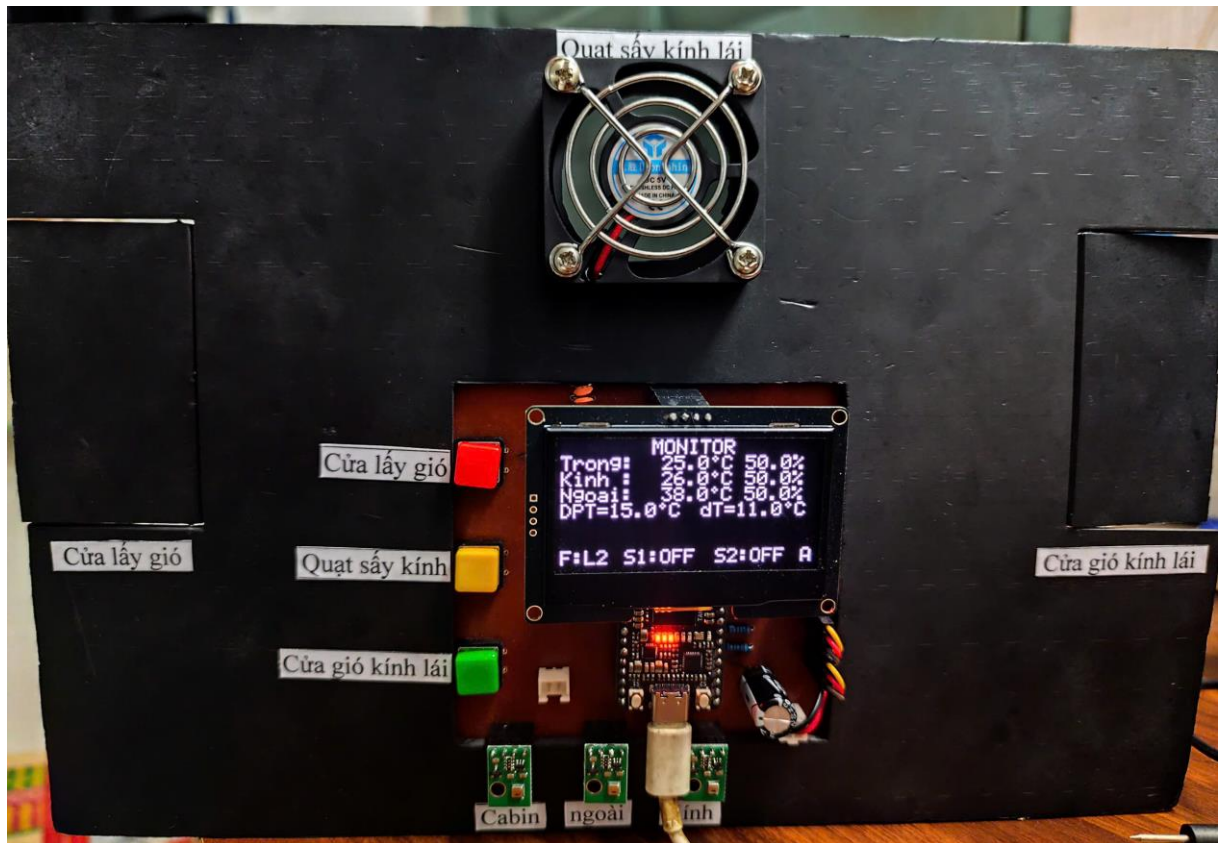
Hình 5.6. Kết quả mô phỏng kịch bản 4 – Ngoài trời mát mẻ

Nhận xét: Hệ thống khai thác hiệu quả điều kiện môi trường thuận lợi nhằm giảm tiêu thụ năng lượng mà vẫn đảm bảo sự thoải mái cho người sử dụng.

5.3.5. Kịch bản 5: Nắng nóng gay gắt

Kịch bản này được xây dựng để kiểm tra khả năng bảo vệ khoang cabin khi nhiệt độ môi trường bên ngoài tăng cao. Trong trường hợp này, nhiệt độ ngoài thất vượt ngưỡng 35°C trong khi kính lái vẫn duy trì trạng thái an toàn với $\Delta T = 110$.

CPU RISC-V xác định rằng việc lấy gió ngoài sẽ làm gia tăng tải nhiệt cho hệ thống điều hòa. Vì vậy, thuật toán chuyển sang chế độ cách ly môi trường bằng cách đặt Servo 2 ở trạng thái OFF để đóng hoàn toàn cửa lấy gió ngoài. Đồng thời, tốc độ quạt được tăng lên mức trung bình ($Fan = 2$) nhằm tăng cường lưu thông khí lạnh trong cabin.



Hình 5.7. Kết quả mô phỏng kịch bản 5 – Nắng nóng gay gắt

Nhận xét: Kết quả cho thấy hệ thống đã thực hiện đúng chiến lược điều khiển đề ra, hạn chế sự xâm nhập nhiệt từ môi trường bên ngoài và duy trì điều kiện nhiệt độ ổn định trong cabin.

5.4. Đánh giá kết quả kiểm thử hệ thống

Thông qua năm kịch bản kiểm thử đã xây dựng, hệ thống giám sát môi trường cabin ô tô sử dụng CPU RISC-V trên FPGA đã thể hiện khả năng hoạt động ổn định và đáp ứng đúng các yêu cầu thiết kế. Kết quả mô phỏng cho thấy các giá trị nhiệt độ, độ ẩm và tham số ΔT được xử lý chính xác, từ đó hệ thống đưa ra quyết định điều khiển phù hợp đối với quạt và các cơ cấu servo.

Trong các trường hợp kính lái an toàn, hệ thống ưu tiên các chiến lược tối ưu năng lượng như giảm tốc độ quạt, đóng cửa lấy gió ngoài hoặc tận dụng nguồn không khí tự nhiên từ môi trường. Ngược lại, khi xuất hiện nguy cơ đóng sương, thuật toán nhanh chóng chuyển sang chế độ phòng ngừa hoặc phá sương khẩn cấp nhằm đảm bảo tầm nhìn cho người lái.

Ngoài ra, kết quả mô phỏng cũng chứng minh khả năng thích ứng của hệ thống trước các điều kiện môi trường khác nhau như thời tiết mát mẻ, nắng nóng hoặc độ ẩm cao trong cabin. Các cơ cấu chấp hành đều thay đổi trạng thái đúng theo logic điều khiển đã đề xuất, không xuất hiện trường hợp điều khiển sai hoặc xung đột trạng thái.

Nhìn chung, các kết quả thu được đã xác nhận tính đúng đắn của thuật toán điều khiển và tính khả thi của việc triển khai hệ thống trên nền tảng CPU RISC-V tích hợp trong FPGA. Đây là cơ sở quan trọng để phát triển và triển khai hệ thống trên phần cứng thực tế trong các nghiên cứu tiếp theo.

CHƯƠNG 6 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Đề tài đã hoàn thành việc thiết kế và hiện thực hệ thống giám sát môi trường cabin ô tô sử dụng CPU RISC-V trên nền FPGA. Hệ thống được xây dựng theo hướng SoC, trong đó CPU RV32I đóng vai trò xử lý trung tâm, kết nối với SRAM, bus memory-mapped và các ngoại vi như LED, UART, Countdown Timer, I2C, GPIO và PWM. Việc tổ chức các ngoại vi theo bản đồ địa chỉ giúp chương trình nhúng có thể truy cập phần cứng thông qua các thao tác đọc/ghi thanh ghi, phù hợp với mục tiêu xây dựng một hệ thống nhúng quy mô nhỏ trên FPGA.

Về phần cứng, đề tài đã tích hợp được CPU RV32I với các khối ngoại vi cần thiết để phục vụ bài toán giám sát và điều khiển môi trường cabin. Các khối I2C được sử dụng để giao tiếp với cảm biến AHT20 và màn hình OLED SSD1309. Khối PWM được dùng để điều khiển quạt và hai servo, trong khi GPIO được dùng để đọc trạng thái nút nhấn. UART được sử dụng hỗ trợ quá trình kiểm tra và gỡ lỗi. Các khối phần cứng được thiết kế theo hướng đơn giản, phù hợp với tài nguyên của kit FPGA Tang Nano 9K và yêu cầu của mô hình thử nghiệm.

Về phần mềm, chương trình nhúng C đã thực hiện được các nhiệm vụ chính gồm khởi tạo hệ thống, đọc dữ liệu từ ba cảm biến AHT20, xử lý dữ liệu nhiệt độ và độ ẩm, tính toán điểm đọng sương, tính giá trị chênh lệch nhiệt độ, cập nhật thông tin lên OLED và điều khiển quạt, servo thông qua PWM. Chương trình hoạt động theo vòng lặp chính, kết hợp cơ chế polling để đọc cảm biến, xử lý nút nhấn và cập nhật trạng thái điều khiển.

Về chức năng đo và hiển thị, hệ thống đã đọc được dữ liệu nhiệt độ, độ ẩm từ các cảm biến AHT20. Dữ liệu sau khi đọc được chuyển đổi sang dạng số nguyên nhân 10 để thuận tiện cho việc hiển thị một chữ số thập phân trên OLED. Màn hình OLED hiển thị được thông tin từ các cảm biến, giá trị điểm đọng sương, giá trị delta, trạng thái quạt, trạng thái servo và chế độ điều khiển của hệ thống. Kết quả này cho thấy CPU RISC-V đã có thể thực thi chương trình nhúng, giao tiếp với ngoại vi và cập nhật dữ liệu hiển thị trong mô hình thực tế.

Về chức năng điều khiển, hệ thống đã tạo được tín hiệu PWM để điều khiển quạt và hai servo. Các mức điều khiển được thay đổi dựa trên trạng thái môi trường hoặc thông qua nút nhấn của người dùng. Ở chế độ tự động, chương trình lựa chọn mức quạt và vị trí servo dựa trên các điều kiện như nhiệt độ trong cabin, nhiệt độ ngoài cabin, điểm động sương và giá trị delta. Ở chế độ thủ công, người dùng có thể thay đổi mức quạt và servo bằng nút nhấn GPIO. Như vậy, các chức năng đo lường, xử lý, hiển thị và điều khiển đã được liên kết thành một vòng hoạt động hoàn chỉnh.

So với mục tiêu ban đầu, đề tài đã đạt được các nội dung chính: xây dựng được CPU RV32I chạy chương trình C, thiết kế được SoC có bộ nhớ và ngoại vi memory-mapped, đọc được cảm biến nhiệt độ - độ ẩm AHT20, hiển thị kết quả lên OLED, điều khiển quạt và servo bằng PWM, đồng thời kiểm tra được sự phối hợp giữa phần cứng và phần mềm trên FPGA. Các kết quả thực nghiệm cho thấy mô hình có khả năng hoạt động đúng theo các kịch bản kiểm thử đã xây dựng.

Tuy nhiên, hệ thống hiện tại vẫn còn một số hạn chế. Đề tài mới dừng ở mức mô hình nguyên mẫu, chưa thực hiện hiệu chuẩn cảm biến bằng thiết bị đo chuẩn, chưa đánh giá sai số đo trong nhiều điều kiện môi trường khác nhau và chưa kiểm tra độ ổn định trong thời gian dài. Chương trình nhúng vẫn hoạt động theo phương pháp polling nên thời gian phản hồi còn phụ thuộc vào chu kỳ đọc cảm biến và cập nhật hiển thị. Ngoài ra, giao tiếp I2C trong đề tài được điều khiển bằng phần mềm nên chưa tối ưu bằng một bộ điều khiển I2C phần cứng hoàn chỉnh.

6.2 Hướng phát triển

Hệ thống có thể được phát triển theo hướng nâng cao độ tin cậy của phần đo lường. Cảm biến AHT20 cần được hiệu chuẩn bằng thiết bị đo nhiệt độ, độ ẩm chuẩn. Các số liệu đo nên được thu thập trong nhiều lần thử nghiệm và trong nhiều điều kiện môi trường khác nhau để đánh giá sai số trung bình, sai số lớn nhất và độ ổn định của hệ thống.

Đối với kiến trúc phần cứng, hệ thống có thể được phát triển theo hướng nâng cấp CPU RISC-V và tối ưu kiến trúc SoC. CPU hiện tại được thiết kế theo hướng đơn giản, phù hợp để kiểm chứng chức năng và chạy chương trình nhúng cơ bản. Trong các phiên bản tiếp theo, CPU có thể được cải tiến bằng cách bổ sung kiến trúc pipeline nhằm

tăng thông lượng xử lý lệnh, giảm thời gian thực thi chương trình và nâng cao hiệu năng tổng thể của hệ thống. Bên cạnh đó, có thể bổ sung cơ chế xử lý ngắt để CPU phản hồi nhanh hơn với các sự kiện như nút nhấn, timer hoặc hoàn tất giao tiếp ngoại vi.

Ngoài việc nâng cấp CPU, bus ngoại vi cũng có thể được chuẩn hóa theo các chuẩn phổ biến như APB hoặc Wishbone. Khi các ngoại vi có giao diện thống nhất, việc tích hợp thêm module mới sẽ thuận lợi hơn, đồng thời các khối như UART, GPIO, Timer, I2C và PWM có thể được tái sử dụng trong các thiết kế SoC khác. Ngoài ra, khối I2C hiện tại có thể được nâng cấp thành bộ điều khiển I2C phần cứng có FSM riêng để giảm tải cho chương trình nhúng và cải thiện độ ổn định của giao tiếp. Đối với CPU RISC-V, có thể tiếp tục mở rộng tập lệnh hoặc cải tiến kiến trúc vi xử lý. Một số hướng phát triển có thể thực hiện gồm bổ sung cơ chế ngắt, tối ưu khối xử lý Load/Store, cải thiện giao tiếp bus hoặc phát triển kiến trúc pipeline ở mức đơn giản. Những cải tiến này giúp CPU hoạt động linh hoạt hơn và phù hợp với các ứng dụng nhúng phức tạp hơn.

Đối với chương trình nhúng, có thể bổ sung cơ chế ngắt cho GPIO, Timer hoặc I2C để giảm sự phụ thuộc vào polling. Khi có cơ chế ngắt, CPU không cần kiểm tra liên tục từng ngoại vi mà chỉ xử lý khi có sự kiện xảy ra, ví dụ như nút nhấn, hoàn tất truyền dữ liệu hoặc đến chu kỳ cập nhật cảm biến. Điều này giúp chương trình rõ ràng hơn và cải thiện khả năng đáp ứng khi số lượng tác vụ tăng lên.

Về thuật toán điều khiển, hệ thống có thể được phát triển theo hướng thông minh hơn bằng cách kết hợp nhiều thông số môi trường như nhiệt độ, độ ẩm, điểm động sương, nhiệt độ khu vực kính và nhiệt độ ngoài cabin. Thuật toán có thể được cải tiến để điều khiển quạt và servo mượt hơn, thay vì chỉ dùng các mức rời rạc. Ngoài ra, có thể bổ sung khả năng lưu dữ liệu đo, truyền dữ liệu ra máy tính hoặc xây dựng giao diện giám sát trực quan hơn.

Nhìn chung, đề tài đã tạo được nền tảng ban đầu cho việc xây dựng một hệ thống nhúng sử dụng CPU RISC-V trên FPGA. Từ nền tảng này, hệ thống có thể tiếp tục được mở rộng cả về phần cứng, phần mềm và thuật toán điều khiển để tiến gần hơn đến một hệ thống giám sát và điều khiển môi trường cabin hoàn chỉnh.

PHỤ LỤC

PHỤ LỤC A: MÃ NGUỒN CHƯƠNG TRÌNH

<https://github.com/ngocat/datn>

PHỤ LỤC B: HƯỚNG DẪN SỬ DỤNG SẢN PHẨM THI CÔNG

Để vận hành mô hình hệ thống giám sát môi trường cabin ô tô sử dụng CPU RISC-V trên FPGA, thực hiện các bước sau:

- Cấp nguồn cho kit FPGA Tang Nano 9K và các ngoại vi như cảm biến AHT20, màn hình OLED, quạt và servo.
- Sau khi hệ thống khởi động, màn hình OLED hiển thị thông tin trạng thái ban đầu.
- Hệ thống tự động đọc dữ liệu từ các cảm biến AHT20, tính toán điểm đọng sương, giá trị delta và hiển thị kết quả lên OLED.
- Ở chế độ AUTO, chương trình tự lựa chọn mức quạt và vị trí servo dựa trên điều kiện môi trường.
- Người dùng có thể sử dụng các nút nhấn để chuyển sang điều khiển thủ công hoặc thay đổi mức quạt, servo tùy theo chương trình đã thiết kế.

TÀI LIỆU THAM KHẢO

TÀI LIỆU THAM KHẢO

- [1] National Highway Traffic Safety Administration, “Child Heatstroke Prevention: Prevent Hot Car Deaths,” 2026. [Online]. Available: <https://www.nhtsa.gov/campaign/heatstroke>. [Accessed Jun. 8, 2026].
- [2] J. Brusey, D. Hintea, E. Gaura, and N. Beloe, “Reinforcement Learning-based Thermal Comfort Control for Vehicle Cabins,” arXiv, 2017. [Online]. Available: <https://arxiv.org/abs/1704.07899>. [Accessed Jun. 8, 2026].
- [3] RISC-V International, “About RISC-V / History of RISC-V,” 2026. [Online]. Available: <https://riscv.org/about/>. [Accessed Jun. 8, 2026].
- [4] RISC-V International, “Volume I: RISC-V Unprivileged ISA Specification,” 2026. [Online]. Available: <https://docs.riscv.org/reference/isa/unpriv/unpriv-index.html>. [Accessed Jun. 8, 2026].
- [5] Sipeed, “Tang Nano 9K Wiki,” 2026. [Online]. Available: <https://wiki.sipeed.com/hardware/en/tang/Tang-Nano-9K/Nano-9K.html>. [Accessed Jun. 8, 2026].
- [6] ASAIR, “AHT20 Product Manual,” 2020. [Online]. Available: https://files.seeedstudio.com/wiki/Grove-AHT20_I2C_Industrial_Grade_Temperature_and_Humidity_Sensor/AHT20-datasheet-2020-4-16.pdf. [Accessed Jun. 8, 2026].
- [7] Solomon Systech, “SSD1309 OLED/PLED Segment/Common Driver with Controller Datasheet,” 2011. [Online]. Available: <https://www.hpinfotech.ro/SSD1309.pdf>. [Accessed Jun. 22, 2026].
- [8] NXP Semiconductors, “UM10204 I2C-bus Specification and User Manual, Rev. 7.0,” 2021. [Online]. Available: https://www.nxp.com/documents/user_manual/UM10204.pdf. [Accessed Jun. 8, 2026].
- [9] IEEE, “IEEE Standard for Verilog Hardware Description Language,” IEEE Std 1364-2005, 2005. [Online]. Available: <https://standards.ieee.org/ieee/1364/3641/>. [Accessed Jun. 8, 2026].

- [10] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface: RISC-V Edition*. Cambridge, MA, USA: Morgan Kaufmann, 2018.
- [11] S. L. Harris and D. M. Harris, *Digital Design and Computer Architecture: RISC-V Edition*. Cambridge, MA, USA: Morgan Kaufmann, 2022.
- [12] S. Gul, N. Aftab, and A. Rani, “A Comparison between RISC and CISC Microprocessor Architectures,” *International Journal of Science Engineering and Advance Technology*, vol. 4, no. 5, May 2016.
- [13] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 5th ed. Waltham, MA, USA: Morgan Kaufmann, 2012.
- [14] Five EmbedDev, “RISC-V Instruction Set Manual, Volume I: RISC-V User-Level ISA, Priv-v1.12: RV32/64G Instruction Set Listings,” 2021. [Online]. Available: <https://five-embeddev.com/riscv-user-isa-manual/Priv-v1.12/instr-table.html>. [Accessed Jun. 8, 2026].